同等学力-软件工程-真题库-按题型分类

一、填空题

1. 类是对象的__抽象__；对象是类的__实例__。

解析：考察软件设计-面向对象方法

- **类**是对一类事物的抽象描述（如“汽车”类），定义了对象的属性和方法；
- **对象**是类的具体实例（如“我的红色轿车”），具备类所定义的特征和行为。
- 二者是抽象与具体的关系，对应面向对象方法的核心概念。

2. 面向对象方法中，对象之间的交互通过__消息传递__实现。

解析：考察软件设计-面向对象方法

面向对象方法中，对象之间的交互通过消息传递实现。

3. 面向对象 4+1 视图模型包括：逻辑视图、物理视图、开发视图、__进程视图__和__场景视图__。

解析：考察软件设计-面向对象方法

4+1 视图模型是面向对象架构设计的经典方法：

- **逻辑视图：**描述对象、类及关系（功能需求）；
- **物理视图：**描述硬件部署（如服务器、网络）；
- **开发视图：**描述代码结构（如模块、包）；
- **进程视图：**描述系统动态行为（如进程、线程交互）；
- **场景视图（即用例）：**串联其他 4 个视图，验证架构合理性。
- 进程视图补充了系统运行时的动态协作细节，是模型的重要组成部分。

二、选择题

1. 在面向对象方法中，信息隐蔽是通过对象的（①）来实现的。只有类的公共界面的成员才能成为使用类的操作，这是软件设计的（②）原则。当且仅当一个操作对类的实例用户有用时，它才是类公共界面的一个成员，这是软件设计的（③）原则。（④）体现了子类复用父类属性和方法的特性。

① A. 分类性 B. 继承性 C. 封装性 D. 共享性

② A. 过程抽象 B. 功能抽象 C. 信息隐蔽 D. 共享性

③ A. 标准调用 B. 最小界面 C. 高耦合 D. 高效率

④ A. 多态性 B. 继承性 C. 封装性 D. 抽象性

解析：考察软件设计-面向对象方法

淘宝店：Panda公众号文档源码库，原创

① C. 封装性

解析：封装是面向对象的核心特性，通过将数据（属性）和操作（方法）绑定在一起，并隐藏内部实现细节，仅对外提供公共接口。例如，一个银行账户类通过私有属性存储余额，通过公共方法（如 deposit()、withdraw()）控制访问，确保数据安全。

② C. 信息隐蔽

解析：信息隐蔽原则要求类的实现细节（如私有变量、内部方法）对外部不可见，仅暴露必要的公共接口。这降低了模块间的依赖，提高了可维护性。例如，Java 中的 private 和 protected 访问修饰符就是信息隐蔽的具体实现。

③ B. 最小界面

解析：最小界面原则（又称 “接口最小化原则”）强调类的公共接口应仅包含用户真正需要的方法，避免暴露无关功能。例如，一个 FileReader 类的公共接口应只包含 read()、close()等核心方法，而非内部实现细节。

④ B. 继承性

解析：继承是子类自动复用父类属性和方法的机制，支持代码复用和层次化设计。例如，“学生类” 继承 “人类” 的属性（如姓名、年龄）和方法（如 “说话”），并可扩展自己的特有行为（如 “学习”）。

2. 敏捷过程模型是一个（①）开发过程，消除了开发中不必要的（②）和提交物。它将开发活动：分析、设计、编码和测试（③）在一起，全过程采用（④）开发、反馈修正和反复测试的策略。

① A. 迭代增量 B. 矩阵式 C. 线性 D. 渐进性

② A. 产品 B. 步骤 C. 过程 D. 文档

③ A. 分离 B. 结合 C. 并行 D. 替代

④ A. 瀑布式 B. 迭代增量 C. 螺旋式 D. 喷泉式

解析：考察软件过程-软件生存周期模型

① A. 迭代增量

敏捷采用迭代增量的方式：将项目分解为多个短周期（Sprint），每个周期产出可运行的增量版本。例如，Scrum 框架中的 Sprint 即为典型的迭代周期。

② D. 文档

敏捷强调 “可工作的软件胜过详尽的文档”（敏捷宣言）。它并非完全排斥文档，而是消除过度的、不产生直接价值的文档（如冗长的需求规格说明书），更注重通过面对面沟通和可运行代码传递信息。

③ B. 结合

敏捷将分析、设计、编码和测试紧密结合：在每个迭代中，团队成员协同完成这些活动，而非按顺序分离（如瀑布模型）。例如，测试人员在需求分析阶段就参与编写测试用例，开发过程中持续执行测试。

④ B. 迭代增量

敏捷通过多次迭代逐步完善产品，每次迭代基于用户反馈调整方向。例如，每次 Sprint 结束后，团队通过评审会收集客户反馈，在下一迭代中修正问题或添加新功能，形成持续改进的闭环。

3. 软件需求分析阶段的核心输出是（ B ）。

A. 模块结构图 B. 需求规格说明书 C. 测试计划 D. 程序代码

淘宝店：PanuCode辉文档源码库 原创

解析：考察软件需求

- **模块结构图：**是软件设计阶段（概要设计）的输出，用于描述模块间的关系，不符合题意。
- **需求规格说明书：**是需求分析阶段的核心成果，详细记录用户需求和系统应具备的功能，符合大纲中“需求规约”的要求。
- **测试计划：**通常在软件设计阶段或测试阶段初期制定，用于规划测试活动，不属于需求分析阶段的输出。
- **程序代码：**是编码阶段的输出，与需求分析阶段无关。
- 因此，正确答案为 B 选项。

4. 结构化设计中，模块间通过参数传递基本数据类型的耦合属于（ A ）

A. 数据耦合
B. 控制耦合
C. 公共耦合
D. 内容耦合

解析：考察软件设计-结构化设计

- **数据耦合：**指模块间通过参数传递基本数据类型（如整数、字符串等），是耦合程度最低、最理想的耦合类型，符合题意。
- **控制耦合：**模块间传递的是控制信息（如标志位、开关量），接收模块需根据控制信息调整行为，耦合程度高于数据耦合。
- **公共耦合：**多个模块通过共享公共数据环境（如全局变量、公共数据结构）交互，耦合程度高，不利于维护。
- **内容耦合：**一个模块直接访问另一个模块的内部数据或代码，是耦合程度最高的类型，应避免使用。
- 因此，正确答案为 A 选项。

5. 以下不属于软件需求分析阶段任务的是（ C ）。

A. 确定软件的功能需求
B. 确定软件的性能需求
C. 确定软件的实现算法
D. 建立软件的逻辑模型

解析：
软件需求分析阶段的核心任务是明确用户需求，定义软件的功能、性能、接口等，并建立系统的逻辑模型（如数据流图、数据字典）。

- 选项 A、B、D 均属于需求分析阶段的任务：确定功能 / 性能需求、建立逻辑模型是需求分析的核心工作。
- 选项 C（确定实现算法）属于详细设计阶段的任务，该阶段需为每个模块设计具体的算法和数据结构，不属于需求分析范畴。

考点关联：对应考试大纲中“需求分析”部分，要求掌握需求阶段的任务与输出成果。

6. 软件详细设计阶段的主要任务是（ B ）。

淘宝店：PanuaCode程文档源码库，原创

A. 确定模块的接口
B. 确定模块的算法和数据结构
C. 确定软件的总体结构
D. 确定软件的开发环境

解析:
软件设计分为总体设计（概要设计）和详细设计两个阶段:

- **总体设计阶段**: 确定软件的总体结构（如模块划分、调用关系）和模块接口（选项 A、C），不涉及具体实现细节。
- **详细设计阶段**: 针对每个模块，设计其具体的**算法**（如流程图、N-S 图描述的处理逻辑）和**数据结构**（如内部使用的数据组织方式），确保模块功能可被程序员直接实现。
- 选项 D（开发环境）属于项目规划阶段的辅助性工作，与设计阶段无直接关联。

考点关联: 对应考试大纲中“软件设计”部分，要求区分概要设计与详细设计的任务，掌握结构化设计的分层思想。

7. 面向对象设计中，对象之间的通信通过（ C ）实现。
A. 继承
B. 封装
C. 消息传递
D. 多态

解析:
在面向对象设计中，**消息传递**是对象之间通信的唯一方式。对象通过发送和接收消息来协同完成任务，消息中包含接收对象要执行的方法及相关参数。

- 选项 A（继承）: 用于实现类之间的层次关系，子类复用父类的属性和方法，与对象通信无关。
- 选项 B（封装）: 将对象的属性和方法封装在类中，隐藏内部细节，属于对象的自我保护机制，非通信方式。
- 选项 D（多态）: 指不同类的对象对同一消息响应不同的行为，是消息传递的后续表现，而非通信本身。

考点关联: 对应考试大纲中“面向对象方法”部分，要求掌握面向对象的基本概念（封装、继承、多态、消息传递）及其作用。

8. 黑盒测试主要是根据（ B ）来设计测试用例。
A. 程序的内部逻辑
B. 程序的功能规格说明
C. 程序的代码结构
D. 程序的执行路径

解析:
黑盒测试（功能测试）的核心思想是将程序视为“黑盒”，**不关注内部逻辑和代码结构，仅依据软件的功能规格说明**（如需求文档中定义的输入输出关系、功能逻辑）设计测试用例。

- 选项 A、C、D 均属于**白盒测试**的依据: 白盒测试需针对程序的内部逻辑、代码结构（如分支、循环）、执行路径设计用例，确保覆盖代码的逻辑细节。

淘宝店：PanuaCode课文档源码库，原创

- 选项 B（功能规格说明）是黑盒测试的唯一依据，用于验证软件是否满足需求定义的功能。

考点关联：对应考试大纲中“软件测试”部分，要求区分黑盒测试与白盒测试的基本概念、方法及适用场景。

9. 软件项目管理中，用于估算项目规模的方法是（ B ）。

A. 甘特图
B. 功能点分析法
C. 挣值分析法
D. 因果图

解析：

- 选项 A（甘特图）：是项目进度管理工具，用于直观展示任务时间安排和依赖关系，不涉及规模估算。
- 选项 B（功能点分析法）：是软件项目规模估算的经典方法，通过统计系统的功能点（如用户输入、输出、查询、文件等）量化项目规模，**适用于需求阶段的早期估算。**
- 选项 C（挣值分析法）：属于项目进度和成本监控工具，通过实际成本与计划成本、进度的对比分析项目绩效，非规模估算方法。
- 选项 D（因果图）：即鱼骨图，用于质量管理中的故障原因分析，与规模估算无关。

考点关联：对应考试大纲中“软件项目管理”部分，要求掌握项目估算的基本方法（如功能点分析、代码行估算）及其适用场景。

10. 软件开发模型是指软件开发的全部过程、活动和任务的结构框架。主要的开发模型有：瀑布模型、演化模型、螺旋模型、喷泉模型等。螺旋模型将瀑布模型和演化模型相结合，并增加了(1)，它建立在(2)的基础上，沿螺旋自内向外每旋转一圈，就得到它的一个新版本。喷泉模型描述了(3)的开发模型，它体现了这种开发方法创建软件过程所固有的(4)和(5)的特征。

供选择的答案：

(1)：A. 系统工程 B. 风险分析 C. 设计评审 D. 进度控制
(2)：A. 模块划分 B. 子程序分解 C. 设计 D. 原型
(3)：A. 面向对象 B. 面向数据流 C. 面向数据结构 D. 面向事件驱动
(4)：A. 归纳 B. 推理 C. 迭代 D. 递归
(5)：A. 开发各阶段之间无间隙 B. 开发各阶段分界明显 C. 部分开发阶段分界明显 D. 开发过程不分段

解析：考察软件过程

(1) B. 风险分析(2) D. 原型(3) A. 面向对象(4) C. 迭代(5) A. 开发各阶段之间无间隙

- **螺旋模型的核心补充（第 1 空）：**螺旋模型在瀑布模型（线性阶段）和演化模型（迭代开发）的基础上，增加了**风险分析**环节。每轮螺旋开发都需评估技术、成本、进度等风险，通过风险驱动决策，因此 (1) 选 B。
- **螺旋模型的基础（第 2 空）：**螺旋模型的开发过程基于**原型迭代**：从核心功能原型出发，每旋转一圈（完成一次迭代）都完善原型，逐步逼近最终产品，因此 (2) 选 D。
- **喷泉模型的适用方法（第 3 空）：**喷泉模型是**面向对象开发**的典型模型，其名称源于面向对象方法中“分析、设计、实现”阶段的无缝衔接，如同喷泉的水循环，因此 (3) 选 A。

淘宝店：PandaCoding原文档源码库，原创

- **喷泉模型的过程特征（第 4 空）：**面向对象开发中，分析、设计、编码等活动常**迭代进行**（如先开发部分类，测试后再扩展），喷泉模型体现了这种迭代性，因此 (4) 选 C。
- **喷泉模型的阶段特征（第 5 空）：**与瀑布模型的严格阶段划分不同，喷泉模型中**各开发阶段无明显间隙**，分析、设计、实现可交叉进行（如边分析边设计），因此 (5) 选 A。

软件过程模型	模型描述
瀑布模型	将软件生存周期的各项活动规定为依固定顺序而连接的若干阶段工作，规定了每一阶段的输入，以及本阶段的工作成果，并作为输出传入下一阶段。
增量模型	该模型有一个假设，即需求可以分组，形成一个一个的增量；在这一条件下，可对每一增量实施瀑布式开发。
演化模型	是一种有弹性的过程模式，由一些小的开发步组成，每一步历经需求分析、设计、实现和验证，产生软件产品的一个增量，通过这些迭代，完成最终软件产品的开发。
螺旋模型	该模型将软件生存周期的活动分为四个可重复的阶段：规划、风险分析、开发和评估。
喷泉模型	该模型体现了软件创建所固有的迭代和无间隙的特征。

11．在面向对象程序设计中，允许不同类的对象对同一消息（方法调用）做出各自不同的响应，这种机制称为（ D ）。

A. 重叠
B. 重载
C. 继承
D. 多态

解析：考察软件设计-面向对象方法

- **多态**（Polymorphism） 是面向对象的三大基本特性之一（封装、继承、多态），指的是**同一个接口或方法名，在不同对象上调用时表现出不同的行为**。它通常通过继承 + 方法重写（Override） 实现，并在运行时根据对象的实际类型动态绑定方法。
- 选项 A“重叠”：不是标准面向对象术语，属于干扰项。
- 选项 B“重载（Overload）”：指在同一个类中定义多个同名但参数列表不同的方法，属于编译时多态（静态多态），但不涉及“不同对象对同一消息的不同响应”。
- 选项 C“继承”：是子类复用父类属性和方法的机制，是实现多态的基础，但本身不是多态。
- 选项 D“多态”：准确描述了题干中“不同对象对同一消息做出不同响应”的核心思想，故为正确答案。

12. 软件概要设计阶段的主要任务是确定（ B ）。

A. 模块的详细算法和数据结构
B. 软件系统的总体结构和模块间关系
C. 用户界面的具体布局和交互细节
D. 软件代码的编程规范和命名规则

解析：考察软件设计

软件概要设计阶段是软件开发过程中的关键环节，其核心任务是将需求分析阶段的成果转化为**软件系统的整体架构设计**，主要聚焦于“系统如何被划分”和“模块间如何交互”：

- 选项 A（模块的详细算法和数据结构）：属于**详细设计阶段**的任务，详细设计需针对每

淘宝店：PandaCode辉文档原创库 原创

个模块明确具体的实现逻辑（如算法步骤）和数据存储方式（如数据结构定义），而非概要设计的核心。

- 选项 B（软件系统的总体结构和模块间关系）：**符合概要设计**的核心定位。该阶段需通过模块划分(如按功能拆分模块)、定义模块间的调用关系(如主模块与子模块的交互)、确定模块的接口（如输入输出参数），形成软件的总体架构，为后续详细设计提供框架。故选 B。
- 选项 C（用户界面的具体布局和交互细节）：属于**界面设计阶段**的任务，聚焦于用户可见的界面元素排版、交互流程（如按钮点击后的响应），与概要设计的"总体结构"无关。
- 选项 D（软件代码的编程规范和命名规则）：属于**编码阶段**的规范要求，用于统一代码风格（如变量命名、注释格式），确保代码的可读性和可维护性，并非概要设计阶段的任务。

13．黑盒测试主要关注被测系统的（ B ）。

A. 内部程序结构和控制流程

B. 功能是否符合需求规格说明

C. 判定条件与分支覆盖情况

D. 源代码中的异常处理机制

解析：考察软件测试

黑盒测试（Black-box Testing）是一种基于规格说明的测试方法，它将软件视为一个"黑盒子"，**不关心内部实现细节**，只根据输入和预期输出来验证系统功能是否满足用户需求或需求规格说明书的要求。

- 选项 A 和 C 描述的是白盒测试的内容，涉及程序内部结构、路径、分支等，不符合黑盒测试的特点；
- 选项 D 虽然异常处理可能在黑盒测试中被间接验证（如通过非法输入），但其"源代码中的机制"属于内部实现，仍属白盒范畴；
- 选项 B 准确体现了黑盒测试的核心目标：**验证功能正确性**，即"做什么"，而非"怎么做"。故选 B。

14．在软件体系结构中，下列内聚类型中，具有最高内聚度的是（ D ）。

A. 逻辑内聚

B. 过程内聚

C. 通信内聚

D. 顺序内聚

解析：考察软件设计-结构化设计

模块内聚性由低到高可分为多种类型，其中功能内聚是最高等级，而顺序内聚是仅次于功能内聚的高内聚类型。在某些分类中，它与功能内聚一同被视为最理想的内聚形式之一。

- A. 逻辑内聚：错误。逻辑内聚是指模块内包含多个在逻辑上相似但功能不同的处理，由传入的控制标志决定执行哪一个。例如，一个"输入处理"模块可能根据标志分别处理键盘、鼠标或文件输入。这些处理之间并无紧密联系，内聚度很低。
- B. 过程内聚：错误。过程内聚是指模块内的处理元素按特定的过程顺序执行，但这种顺序可能源于外部控制流或业务流程，而非严格的数据依赖。其内部联系弱于顺序内聚。
- C. 通信内聚：错误。通信内聚是指模块中的所有处理元素都操作于同一个输入数据集或产生同一个输出数据集，但处理顺序并不重要。例如，一个模块同时计算某个数据的平均值和最大值。虽然操作同一数据，但功能间无必然顺序，内聚度中等。

淘宝店：PandaCode编程文档源码库，原创

- D. 顺序内聚：正确。顺序内聚是指模块中的各个处理元素必须顺序执行，且前一个元素的输出数据直接作为后一个元素的输入数据。这种紧密的数据流与处理顺序关联使得模块内部高度相关，内聚度非常高，是设计中应追求的目标之一。

在内聚度的标准排序中，从低到高通常为：**偶然内聚 < 逻辑内聚 < 时间内聚 < 过程内聚 < 通信内聚 < 顺序内聚 < 功能内聚**。因此，在本题给出的四个选项中，故选 D，**顺序内聚**是内聚度最高的。

15．在软件工程中，需求分析阶段的首要任务是（ D ）。

A. 设计软件的系统架构和模块划分

B. 确定模块的详细算法和数据结构

C. 编写用户手册和测试计划

D. 定义用户需求并建立系统逻辑模型

解析：考察软件需求

需求分析是软件生命周期中需求工程的核心阶段，其首要目标是准确、完整、无歧义地理解和描述用户对系统的功能与非功能需求，并在此基础上构建系统的逻辑模型，为后续设计与实现提供依据。

- 选项 A（系统架构和模块划分）属于概要设计阶段的任务；
- 选项 B（详细算法和数据结构）属于详细设计阶段的内容；
- 选项 C（用户手册、测试计划）通常在需求规格说明书完成后或开发后期编制，不是需求分析的首要任务；
- 选项 D 正确指出需求分析的核心工作：**定义用户需求**（包括功能需求、性能需求、约束等），并通过建模工具（如用例图、数据流图、实体关系图等）**建立系统逻辑模型**，反映“系统应该做什么”，而非“如何做”。故选 D。

16．软件开发的需求活动，其重要任务是（ D ）。

A．给出软件处理方案 B．给出系统模块构造

C．定义模块算法 D．定义需求并建立系统模型

解析：考察软件需求

答案是选项 D。

软件开发的需求活动处于项目早期阶段，核心目标是明确“软件要做什么”，而非“怎么做”：

- **A 选项“给出软件处理方案”：**属于设计阶段任务，侧重确定软件的实现思路（如数据处理流程），而非需求阶段。
- **B 选项“给出系统模块构造”：**属于概要设计阶段任务，关注系统的模块划分与模块间关系，与需求活动无关。
- **C 选项“定义模块算法”：**属于详细设计阶段任务，聚焦单个模块的具体逻辑实现（如排序算法、计算逻辑），不属于需求范畴。
- **D 选项“定义需求并建立系统模型”：**需求活动的核心是收集、分析用户需求（如功能需求、非功能需求），并通过需求模型（如用例图、数据流图）明确需求，符合需求阶段的任务定位。

17．软件总体设计结束后得到（ B ）。

A．初始化的软件构造图 B．优化的软件构造图

C．模块的算法 D．程序编码

解析：考察软件设计

淘宝店：ProductCode软件程序源码库 原创

答案是选项 B。

软件总体设计（概要设计）的核心任务是确定系统整体架构，明确模块划分、模块间关系及接口，最终输出经过优化的架构方案：

- **A 选项“初始化的软件构造图”：**仅为设计初期的初步架构草稿，未经过合理性评估（如耦合 / 内聚分析）与优化，不符合总体设计“确定最终架构”的目标。
- **B 选项“优化的软件构造图”：**总体设计需对初步架构进行优化（如调整模块划分以降低耦合、提升内聚），确保架构满足可扩展性、可维护性等质量要求，最终输出优化后的构造图（如模块结构图），符合该阶段输出物定义。
- **C 选项“模块的算法”：**属于详细设计阶段任务，聚焦单个模块的具体逻辑实现（如计算步骤、判断流程），与总体设计“关注整体架构”的范畴无关。
- **D 选项“程序编码”：**是编码阶段的产出，需依据详细设计文档编写代码，与总体设计阶段无直接关联。

18．面向对象分析措施与构造化分析措施的区别之一是（ B ）。

A．一种使用了“抽象”，而另一种没有使用“抽象”

B．一种以问题域的成分为基点来构造一种模块，而另一种不是

C．一种目的明确，而另一种目的不明确

D．一种需要考虑数据构造，而另一种不需要考虑数据构造

解析：考察软件设计

答案是选项 B。

面向对象分析侧重于问题域的实体（如对象），并以这些实体为中心来构建系统。

而结构化分析则侧重于功能和过程，通常不以问题域的成分为基点来构造模块。

面向对象分析（OOA）与结构化分析（SA）的核心区别在于分析视角与模块构造逻辑，具体对比如下：

- **A 选项：**错误。“抽象”是两种分析方法的共同核心思想 —— 结构化分析通过抽象数据流程、加工逻辑简化问题；面向对象分析通过抽象类、对象提炼问题域共性，二者均使用抽象。
- **B 选项：**正确。面向对象分析以“问题域中的成分”（如现实世界的实体、角色，对应类 / 对象）为基点构造模块，模块与问题域概念直接对应；结构化分析则以“数据流程”为核心，按“输入→加工→输出”的流程划分模块，不直接绑定问题域成分，这是二者的关键区别之一。
- **C 选项：**错误。两种分析方法的目的均明确 —— 均为清晰定义软件需求、建立需求模型，只是建模视角不同，不存在“目的不明确”的情况。
- **D 选项：**错误。数据结构是两种分析方法的重要考量对象 —— 结构化分析需定义数据字典、明确数据存储结构；面向对象分析需设计类的属性（数据结构）、关联关系，二者均需考虑数据构造。

19．确定测试的计划是在（ A ）阶段制定的。

A．需求分析 B．详细设计

C．编码 D．测试

解析：考察软件需求

答案是选项 A。

在需求分析阶段，开发团队会确定软件的功能和性能要求，并制定相应的测试计划以确保软件能够满足这些要求。在详细设计、编码和测试阶段，团队会根据这个计划进行具体的测试

淘宝店：ProjectCode源代码库，原创

工作。

软件测试计划需在项目早期制定，确保测试活动与需求、设计、编码等阶段同步衔接，具体分析如下：

- **A 选项“需求分析”：**正确。测试计划的核心是明确“测什么”（基于需求）、“怎么测”（测试策略）、“资源与进度”，需在需求分析阶段（明确软件功能 / 非功能需求后）制定，确保后续测试活动围绕需求展开，避免遗漏关键测试点。
- **B 选项“详细设计”：**错误。详细设计阶段聚焦模块内部逻辑（如算法、接口），此阶段制定的是“测试用例”（基于设计细节），而非整体测试计划。
- **C 选项“编码”：**错误。编码阶段是将设计转化为代码，此阶段主要进行单元测试（验证单个模块），依赖已制定的测试计划和用例，不制定测试计划。
- **D 选项“测试”：**错误。测试阶段是执行测试计划、记录结果的阶段，计划需在测试执行前完成，而非此阶段制定。

20．按不一样人员的工作内容来分类软件过程，基本过程包括（ C ）。

A．维护过程和改善过程等 B．开发过程和改善过程等

C．获取过程和供应过程等 D．改善过程和淘汰过程等

解析：考察软件过程

答案是选项 C。

按过程中活动的不同主体，将基本过程分为 5 个过程：获取过程、供应过程、开发过程、运行过程、维护过程。

按人员工作内容分类的软件过程中，“基本过程”是直接支撑软件从需求到交付的核心流程，聚焦与软件产品直接相关的关键活动，具体分析如下：

- **A 选项“维护过程和改善过程等”：**维护过程属于“支持过程”（保障软件交付后持续可用），改善过程属于“组织过程”（优化整体软件过程能力），均不属于基本过程。
- **B 选项“开发过程和改善过程等”：**开发过程虽属于基本过程，但改善过程属于组织过程，并非基本过程范畴，该选项包含非基本过程，表述错误。
- **C 选项“获取过程和供应过程等”：**获取过程（需求方获取软件的流程）、供应过程（开发方提供软件的流程），以及开发过程（核心的软件设计开发流程），共同构成软件的基本过程，直接覆盖软件从需求到交付的核心环节，表述正确。
- **D 选项“改善过程和淘汰过程等”：**改善过程属于组织过程，淘汰过程（软件生命周期末期的停用流程）属于支持过程，均与基本过程无关。

20．一个模块内部的各部分没什么联系，另一个模块通过传递基本参数向它传递一个控制，该模块属于什么（ B ）内聚。

A. 数据内聚

B. 逻辑内聚

C. 通信内聚

D. 顺序内聚

解析：考察软件设计-结构化设计

内聚是衡量模块内部各元素（语句、功能等）之间关联程度的指标，内聚程度从低到高可分为多种类型：

- **数据内聚：**模块内各部分通过数据传递关联，共同处理同一批数据，内聚程度较高。
- **逻辑内聚：**模块内各部分逻辑上相关（如多个相似功能的集合），但功能独立性差，内部联系松散。题目中“模块内部的各部分没什么联系”且接收控制参数（用于选择执行

淘宝搜：PandaCode源文档源码库，原创

不同逻辑），符合逻辑内聚的特征。

- **通信内聚：**模块内各部分使用相同的输入数据或产生相同的输出数据，关联基于数据通信，内聚程度中等。
- **顺序内聚：**模块内各部分按顺序执行，前一部分的输出是后一部分的输入，关联紧密，内聚程度高。
- 题干中模块内部元素联系弱，且通过控制参数决定执行逻辑，属于典型的逻辑内聚。

答案：B. 逻辑内聚。

21．DFD 顶层流图用于指出系统的（ C ）

A. 数据主体
B. 数据存储
C. 输入与输出
D. 处理过程

解析：考察软件设计

数据流图（DFD）的顶层流图，也称为上下文图，它将整个系统看作一个单一的处理过程，主要作用是展示系统与外部实体之间的交互，明确系统的输入来源和输出去向，也就是指出系统的输入与输出。

答案：C. 输入与输出。

22．黑盒测试主要考虑系统的（ B ）

A. 程序流程
B. 功能
C. 判定分支结构
D. 异常处理

解析：考察软件测试

黑盒测试是把测试对象看作一个黑盒子，测试人员完全不考虑程序内部的逻辑结构和内部特性，只依据程序的需求规格说明书，检查程序的功能是否符合它的功能说明。它主要关注的是系统的功能实现情况，而不是程序内部的流程、判定分支结构或者特定的异常处理细节（异常处理可作为功能的一部分进行测试，但不是黑盒测试的核心考量维度）。

答案：B. 功能。

23．RUP 中的‘谁做’，‘怎么做’，‘做什么’里的‘谁做’有哪个概念/术语（ B ）完成

A. 制品
B. 角色
C. 工作流
D. 数据

解析：考察软件过程

- **A. 制品：**在 RUP（统一软件开发过程）中，制品是软件开发过程中产生的各种工作成果，比如文档、模型、代码等，它主要体现“做什么”产生的结果，而不是“谁做”，所以 A 选项错误。
- **B. 角色：**角色定义了在软件开发过程中不同人员或团队的职责和分工，明确了某项任务由谁来执行，对应“谁做”这一概念， 所以 B 选项正确。
- **C. 工作流：**工作流描述了软件开发过程中活动的顺序和流程，侧重于“怎么做”，即规

淘宝店：PandaCode辉文档源码库，原创

定了完成任务的步骤和方式，并非“谁做”，所以 C 选项错误。

- D. **数据：**数据是软件开发过程中涉及的信息载体，如需求数据、设计数据等，与“谁做”没有直接关联，所以 D 选项错误。

答案：B. 角色。

24．面向对象设计模式，说法正确的是（ B ）

A. 使程序员能够更快的完成

B. 可以复用好的设计

C. 使程序以最快的速度

D. 仅适用于特定编程语言

解析：考察软件设计-面向对象方法

- **A 选项：**设计模式的核心是复用设计思想，而非直接加速编码速度，编码效率受多种因素影响，故 A 错误。
- **B 选项：**设计模式是对反复出现的问题的成熟解决方案，本质是复用经过验证的设计思路，减少重复设计，故 B 正确。
- **C 选项：**设计模式侧重代码的可维护性、扩展性，与程序运行速度无直接关联，甚至部分复杂模式可能略微影响性能，故 C 错误。
- **D 选项：**设计模式是通用设计层面的思想，不依赖特定编程语言(如单例模式可在 Java、Python 等多种语言中实现)，故 D 错误。

答案：B. 可以复用好的设计。

25. 采用一样的方法名，重新写方法体的方法是（ B ）。

A. 重叠

B. 重载

C. 继承

D. 多态

解析：考察软件设计-面向对象方法

- 重载（Overload）：是面向对象编程中，在同一个类中定义多个方法名相同、参数列表（参数个数 / 类型 / 顺序）不同的方法，方法体逻辑可以不同，用于实现“同名不同逻辑”的功能，符合题干描述，故选 B。
- 选项 A（重叠）：并非面向对象编程中的标准概念，属于干扰项。
- 选项 C（继承）：是子类获取父类属性和方法的机制，不涉及“同名重写方法体”。
- 选项 D（多态）：是父类引用指向子类对象、调用方法时呈现不同行为的特性，重载是实现多态的方式之一，但本身不是“同名重写方法体”的机制。

26．软件总体设计结束后得到（ B ）。

A．初始化的软件构造图 B．优化的软件构造图

C．模块的算法 D．程序编码

解析：考察软件设计

软件总体设计（概要设计）的核心任务是确定系统整体架构，明确模块划分、模块间关系及接口，最终输出经过优化的架构方案：

- **A 选项“初始化的软件构造图”：**仅为设计初期的初步架构草稿，未经过合理性评估（如耦合 / 内聚分析）与优化，不符合总体设计“确定最终架构”的目标。
- **B 选项“优化的软件构造图”：**总体设计需对初步架构进行优化（如调整模块划分以降

淘宝店：maocode辉文档源码库 原创

低耦合、提升内聚），确保架构满足可扩展性、可维护性等质量要求，最终输出优化后的构造图（如模块结构图），符合该阶段输出物定义。

- **C 选项“模块的算法”：**属于详细设计阶段任务，聚焦单个模块的具体逻辑实现（如计算步骤、判断流程），与总体设计“关注整体架构”的范畴无关。
- **D 选项“程序编码”：**是编码阶段的产出，需依据详细设计文档编写代码，与总体设计阶段无直接关联。

27．DFD 顶层流图用于指出系统的（ C ）

A. 数据主体

B. 数据存储

C. 输入与输出

D. 处理过程

解析：考察软件设计-结构化分析方法

数据流图（DFD）的顶层流图，也称为上下文图，它将整个系统看作一个单一的处理过程，主要作用是展示系统与外部实体之间的交互，明确系统的输入来源和输出去向，也就是指出系统的输入与输出。

答案：C. 输入与输出。

28．确定测试的计划是在（ A ）阶段制定的。

A．需求分析 B．详细设计

C．编码 D．测试

解析：考察软件需求

软件测试计划需在项目早期制定，确保测试活动与需求、设计、编码等阶段同步衔接，具体分析如下：

- **A 选项“需求分析”：**正确。测试计划的核心是明确“测什么”（基于需求）、“怎么测”（测试策略）、“资源与进度”，需在需求分析阶段（明确软件功能 / 非功能需求后）制定，确保后续测试活动围绕需求展开，避免遗漏关键测试点。
- **B 选项“详细设计”：**错误。详细设计阶段聚焦模块内部逻辑（如算法、接口），此阶段制定的是“测试用例”（基于设计细节），而非整体测试计划。
- **C 选项“编码”：**错误。编码阶段是将设计转化为代码，此阶段主要进行单元测试（验证单个模块），依赖已制定的测试计划和用例，不制定测试计划。
- **D 选项“测试”：**错误。测试阶段是执行测试计划、记录结果的阶段，计划需在测试执行前完成，而非此阶段制定。

29. 软件详细设计阶段的主要任务是（ B ）。

A. 确定模块的接口

B. 确定模块的算法和数据结构

C. 确定软件的总体结构

D. 确定软件的开发环境

解析：考察软件设计

软件设计分为总体设计（概要设计）和详细设计两个阶段：

- **总体设计阶段**：确定软件的总体结构（如模块划分、调用关系）和模块接口（选项 A、C），不涉及具体实现细节。
- **详细设计阶段**：针对每个模块，设计其具体的**算法**（如流程图、N-S 图描述的处理逻辑）

淘宝店：pocode源文档源码库 原创

和**数据结构**（如内部使用的数据组织方式），确保模块功能可被程序员直接实现。

- 选项 D（开发环境）属于项目规划阶段的辅助性工作，与设计阶段无直接关联。

30. E-R 模型中包括的基本成分是（ C ）。

A.数据、对象、实体

B.控制、联系、对象

C.实体、联系、属性

D.实体、属性、操作

解析：考察软件设计

E-R 模型（实体 - 联系模型）是数据库设计中用于描述现实世界的概念模型，其核心仅包含三个基本成分，具体定义如下：

- **实体：**现实世界中可独立存在的事物（如“学生”“课程”），是数据的基本载体；
- **属性：**实体的特征（如“学生”的“学号”“姓名”），用于描述实体的具体信息；
- **联系：**不同实体之间的关联关系（如“学生”与“课程”的“选课”联系），体现实体间的业务逻辑。

因此，选项 C 是正确的答案。

31. 正式技术评审的目标是（ A ）。

A.发现软件中的错误

B.评价程序员的工作效率

C.发现并改正程序中的错误

D.记录程序员的出错情况并与绩效挂钩

解析：考察软件质量

- 选项 A 正确：正式技术评审是软件质量保障的重要手段，其核心目标是**通过团队评审（如同行专家检查）主动发现软件产品（如需求文档、设计方案、代码）中的缺陷**，包括功能逻辑错误、不符合规范的实现、需求偏离等。
- 选项 B 错误：评审聚焦于软件产品质量，而非评价程序员的工作效率；
- 选项 C 错误：评审的核心是“发现错误”，改正错误由开发人员在评审后完成，并非评审过程的目标；
- 选项 D 错误：评审不涉及绩效挂钩，其目的是提升产品质量，而非记录个人出错情况。

32. 在 UML 中，（ B ）关系描述了两个对象类之间的一般化/特殊化关系，它可以使子对象类共享父对象类的属性和方法。

A、依赖 B.泛化 C.关联 D.实现

解析：考察软件设计-面向对象方法

UML（统一建模语言）中，不同关系对应类或对象间的不同交互逻辑，需结合题干“一般化 / 特殊化”“共享父类属性和方法”的核心特征判断：

- **选项 A（依赖）：**描述类间的“弱关联”，如一个类的方法参数引用另一个类（如“订单类”的“计算金额”方法引用“商品类”），仅体现“使用关系”，无“一般化 / 特殊化”及“共享属性方法”的特征，排除。
- **选项 B（泛化）：**是 UML 中专门描述“一般类（父类）与特殊类（子类）”的关系，本质即面向对象中的“继承关系”。子类通过泛化可直接共享父类的属性和方法（如“学生类”作为父类，“本科生类”作为子类，可共享“学号”“姓名”属性及“选课”方法），完全匹配题干描述，为正确选项。

淘宝店：Panda资源辉文档源码库 原创

- **选项 C（关联）：**描述类间的“结构性关联”，如“学生类”与“课程类”的“选课”关联，仅体现对象间的业务联系，不涉及“一般化 / 特殊化”，排除。
- **选项 D（实现）：**用于“接口与实现类”之间，描述类如何实现接口定义的抽象方法（如“支付接口”与“微信支付类”），无父类与子类的“一般化 / 特殊化”关系，排除。

33. 某模块内有两个处理 A 和 B，分别对数据区 X 写数据和读数据，则该模块的内聚类型属于（ C ）。

A. 逻辑内聚 B. 过程内聚 C. 通信内聚 D. 内容内聚

解析：考察软件设计

模块内聚是衡量模块内部各元素（如处理、数据）关联紧密程度的指标，需根据模块内处理与数据的关系判断：

- **选项 A（逻辑内聚）：**模块内的处理逻辑相关但功能不同（如一个模块包含“查询学生信息”“查询课程信息”，通过参数区分），处理并非围绕同一数据区，排除。
- **选项 B（过程内聚）：**模块内的处理按特定流程顺序执行（如“先输入数据，再验证格式，最后保存”），但处理未必共享同一数据区，重点是“步骤顺序”，排除。
- **选项 C（通信内聚）：**模块内的所有处理均操作同一数据区或通过同一数据接口交互。题干中处理 A（写数据）和处理 B（读数据）均针对“数据区 X”，即围绕同一数据区协同工作，符合**通信内聚的核心特征（共享数据）**，为正确选项。
- **选项 D（内容内聚）：**模块内的处理高度关联且不可分割（如一个模块包含“计算工资”及“计算工资所需的所有子步骤”），题干中“写数据”和“读数据”是相对独立的操作，不符合内容内聚的强耦合性，排除。

34. 关于增量开发模型的叙述，错误的是（ D ）。

A. 不必等到整个系统开发完成就可以使用

B. 可以使用较早开发的增量构件来构建稍后开发的增量构件

C. 优先级最高的服务先交付，这样最重要的服务能接受最多的测试

D. 有利于较好的模块划分

解析：考察软件过程

增量开发模型的核心是将系统拆分为多个“增量构件”，按优先级分批开发、交付，需结合其核心特征判断选项正误：

- **选项 A（正确）：**增量模型的关键优势是“分批交付”，无需等待整个系统开发完成，早期增量构件（如核心功能）开发完成后即可投入使用，能快速满足用户部分需求。
- **选项 B（正确）：**增量构件间可存在依赖关系，较早开发的基础构件（如“用户登录模块”）可作为后续构件（如“订单管理模块”）的基础，支持构件复用与迭代构建。
- **选项 C（正确）：**增量模型按需求优先级排序，优先级最高的服务（如核心业务功能）优先开发交付，能更早接受实际使用中的测试与反馈，降低核心功能的质量风险。
- **选项 D（错误）：**增量模型的重点是“按需求优先级拆分增量”，而非“模块划分”。模块划分是软件设计阶段（如结构化设计、面向对象设计）的核心任务，与增量开发的“交付策略”无直接关联，增量模型本身不直接提升模块划分的质量。

35.、瀑布模型是一种（ B ）。

A. 软件产品模型 B. 软件过程模型

C. 软件项目模型 D. 软件测试模型

解析：考察软件过程

淘宝店：PandaCode辉文档课程 原创

瀑布模型的本质是描述软件开发全过程的流程框架，属于**软件过程模型**。具体分析如下：

- **选项 A（软件产品模型）：**错误。软件产品模型聚焦软件本身的结构或功能形态（如层次模型、面向对象模型），用于刻画产品的内部构成，与开发流程无关，而瀑布模型不涉及产品结构描述。
- **选项 B（软件过程模型）：**正确。软件过程模型是对软件开发全过程（从需求到维护）的流程化描述，用于规范开发步骤与阶段衔接。瀑布模型将软件生命周期划分为“需求分析→设计→编码→测试→运行维护”等线性阶段，每个阶段依赖前一阶段的输出，是典型的过程模型。
- **选项 C（软件项目模型）：**错误。软件项目模型侧重项目管理维度（如资源调配、进度控制、成本核算），而瀑布模型核心是技术开发流程，不直接涉及项目管理要素。
- **选项 D（软件测试模型）：**错误。软件测试模型仅针对测试环节（如 V 模型、敏捷测试模型），而瀑布模型覆盖从需求到维护的全生命周期，远超测试范畴。

因此，答案是选项 B。

36、软件需求开发阶段包括多种活动，不属于需求开发活动的是（ D ）。

A.软件需求获取　B.软件需求分析建模

C.软件需求定义与评审　　D.软件需求管理

解析：考察软件需求

软件需求工程包含**需求开发和需求管理**两大核心部分，两者的边界与活动内容不同：

- **需求开发**是“生成需求文档”的阶段性活动，聚焦于从无到有地明确、分析和确认需求，具体包括：
  - **选项 A（需求获取）：**通过访谈、调研等方式收集用户需求；
  - **选项 B（需求分析建模）：**对获取的需求进行分析，用模型（如用例图、数据流图）梳理逻辑；
  - **选项 C（需求定义与评审）：**将需求整理为规范文档（如 SRS），并通过评审确认其准确性。
- **选项 D（需求管理）：**属于需求工程的后续持续性活动，核心是维护需求文档的一致性与可追踪性（如需求变更控制、版本管理、需求与设计的双向追踪），不直接参与“生成需求”的开发过程，因此不属于需求开发活动。

因此，答案是选项 D。

37、模块设计中，最差的内聚类型是（ D ）。

A.功能内聚　B.逻辑内聚　C.信息内聚　D.巧合内聚

解析：考察软件设计

内聚有如下的种类，它们之间的内聚度由弱到强排列如下：

（1）偶然内聚（巧合内聚）：一个模块内的各处理元素之间没有任何联系，只是偶然地被凑到一起。这种模块也称为巧合内聚，内聚程度最低。

（2）逻辑内聚：这种模块把几种相关的功能组合在一起， 每次被调用时，由传送给模块参数来确定该模块应完成哪一种功能 。

（3）时间内聚：把需要同时执行的动作组合在一起形成的模块称为时间内聚模块。

（4）过程内聚：构件或者操作的组合方式是，允许在调用前面的构件或操作之后，马上调用后面的构件或操作，即使两者之间没有数据进行传递。简单的说就是如果一个模块内的处理元素是相关的，而且必须以特定次序执行则称为过程内聚。

（5）通信内聚：指模块内所有处理元素都在同一个数据结构上操作或所有处理功能都通过

淘宝店：PandaCode源文件源码库，原创

公用数据而发生关联（有时称之为**信息内聚**）。即指模块内各个组成部分都使用相同的数据数据或产生相同的数据结构。

（6）顺序内聚：一个模块中各个处理元素和同一个功能密切相关，而且这些处理必须顺序执行，通常前一个处理元素的输出时后一个处理元素的输入。

（7）功能内聚：模块内所有元素的各个组成部分全部都为完成同一个功能而存在，共同完成一个单一的功能，模块已不可再分。即模块仅包括为完成某个功能所必须的所有成分，这些成分紧密联系、缺一不可。

因此，答案是选项 D。

38、软件集成测试主要依据的文档应该是（ C ）。

A. 软件需求规格说明书　　B. 软件工作分解结构

C. 软件概要设计说明书　　D. 软件详细设计说明书

解析：考察软件测试

软件集成测试的核心目标是验证**模块之间的接口是否正确**，以及模块组合后是否满足概要设计中的功能与性能要求。其依据的核心文档需明确模块间的接口定义、交互逻辑及整体结构，具体分析如下：

- **选项 A（软件需求规格说明书）：**错误。该文档描述软件的整体功能、性能及用户需求，是单元测试和系统测试的主要依据，而非集成测试（集成测试聚焦模块间接口，而非整体需求）。
- **选项 B（软件工作分解结构）：**错误。工作分解结构（WBS）是项目管理工具，用于将项目任务分解为子任务，与测试依据无关。
- **选项 C（软件概要设计说明书）：**正确。概要设计说明书明确模块划分、模块间的接口定义（如函数参数、数据传递方式）、模块组合逻辑及整体架构，这些是集成测试验证接口正确性和模块协作性的直接依据。
- **选项 D（软件详细设计说明书）：**错误。该文档描述单个模块的内部实现细节（如算法、数据结构），是单元测试（验证模块内部逻辑）的主要依据，而非集成测试。

因此，答案是选项 C。

39、软件项目管理中，用于实施进度控制的图是（ A ）。

A. Gantt 图　B. IPO 图　　C. 管理图　　D. 系统流程图

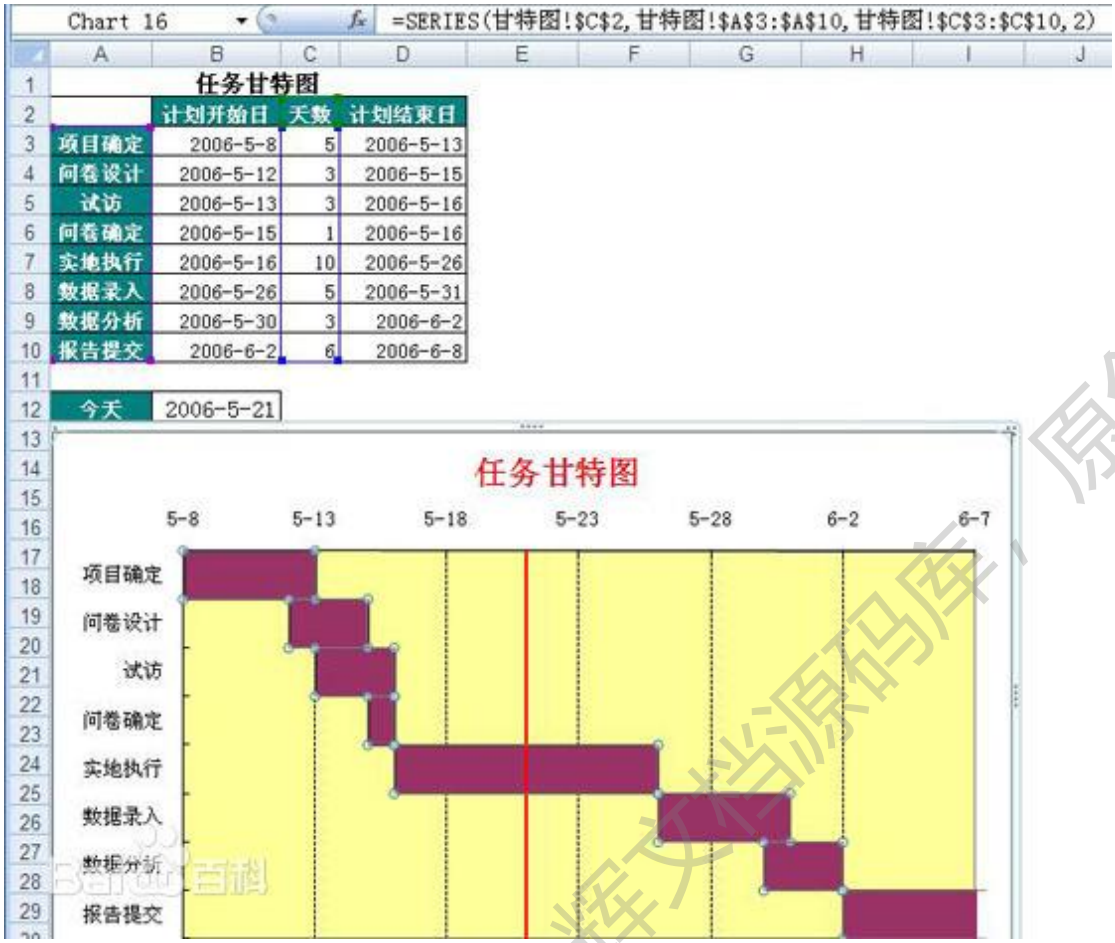
解析：考察软件管理

软件项目管理中，不同图表的功能与适用场景不同，需根据“进度控制”的核心需求判断：

- **选项 A（Gantt 图）：**正确。Gantt 图（甘特图）是项目进度管理的经典工具，通过横轴表示时间、纵轴表示任务，用横道线直观展示各任务的计划开始 / 结束时间、持续时长及任务间的依赖关系，可清晰跟踪进度是否滞后，是实施进度控制的核心图表。
- **选项 B（IPO 图）：**错误。IPO 图（输入 - 处理 - 输出图）用于描述单个模块的输入、处理逻辑和输出，是软件设计阶段的工具，与项目进度控制无关。
- **选项 C（管理图）：**错误。“管理图”是一个宽泛概念，通常指项目管理中的各类图表（如资源分布图、风险矩阵图等），并非专门用于进度控制的特定工具，不具备针对性。
- **选项 D（系统流程图）：**错误。系统流程图用于描述系统的物理流程（如数据在设备间的流动、处理步骤），属于系统分析阶段的工具，与项目进度管理无关。

因此，答案是选项 A。

淘宝店：PandaCode论文档源码库 原创



40. 一个重要的软件生存周期模型是增量模型。以下各选项中，不属于该模型开发活动的是（ D ）。

A. 增量分析　　B. 增量设计　　C. 增量实现　D. 增量发布

解析：考察软件过程

答案是选项 D。

增量模型（Incremental Model）是一种将软件产品分批次逐步交付的开发方法。每个增量都包含需求分析、设计、编码、测试等完整的开发活动，形成一个可运行的子系统或功能模块。

- 增量分析：对当前增量所需功能进行需求分析；
- 增量设计：针对该增量进行架构和详细设计；
- 增量实现：包括编码与单元测试；
- 这三项（A、B、C）都是开发过程中的核心工程活动，属于增量模型的标准组成部分。而“增量发布”虽然在实践中常伴随增量交付发生，但它属于项目管理或配置管理/部署活动，并非增量模型定义中的基本开发活动。

41. 在使用结构化分析方法时，首先需要定义系统的边界。通常用于表达系统边界的是（ B ）。

A. 数据存储　　B. 外部实体　　C. 数据接口　D. 运行环境

解析：考察软件设计-结构化分析

答案是选项 B。

在结构化分析方法（Structured Analysis, SA）中，**数据流图（DFD）**是核心建模工具。DFD 由以下四种基本元素组成：

淘宝店：PandaCode辉文档源码库 原创

- 外部实体（External Entity）：表示系统之外的人、组织或其他系统，与被建模系统有数据交互；
- 处理（Process）：对数据进行变换或处理的功能单元；
- 数据流（Data Flow）：数据在系统内或与外部之间的流动；
- 数据存储（Data Store）：系统内部存储数据的地方。

其中，外部实体位于系统边界之外，用于标识系统与外界的交互点，从而清晰界定系统的范围和边界。例如，“用户”“银行系统”“传感器”等都是典型的外部实体。

- 选项 A（数据存储）：属于系统内部，不表示边界；
- 选项 C（数据接口）：不是 DFD 的标准元素，属于实现层面概念；
- 选项 D（运行环境）：属于部署或非功能性需求范畴，不用于 DFD 中定义逻辑边界。

因此，外部实体是结构化分析中表达系统边界的唯一标准元素。

42. 内聚是模块内部各个成分之间相互关联的紧密程度的度量。如果一个模块内部各个处理必须按特定的次序执行，那么这种内聚类型是（ C ）。

A. 逻辑内聚　　B. 通信内聚　　C. 过程内聚　D. 功能内聚

解析：考察软件设计

答案是选项 C。

内聚性（Cohesion）是衡量模块内部元素彼此关联紧密程度的重要软件设计指标，内聚越高，模块独立性和可维护性越好。

题干中描述：“模块内部各个处理必须按特定的次序执行”，这正是过程内聚（Procedural Cohesion）的典型特征。

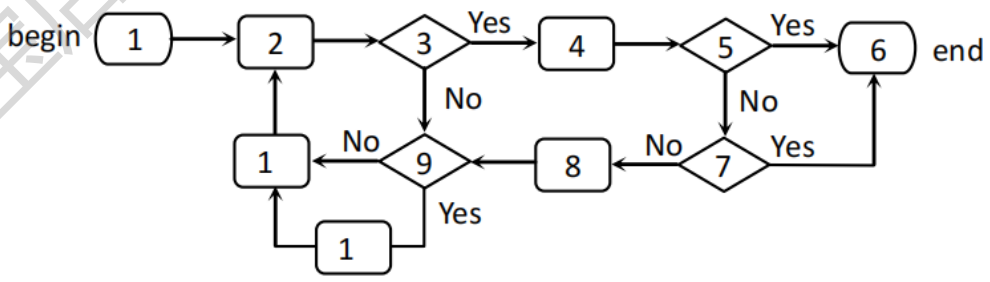
- **过程内聚：**模块中的处理元素按特定控制流程顺序执行，但它们之间不一定操作相同数据，也不一定共同完成单一功能，仅因执行顺序相关而被组织在一起。
- 例如：先打开文件 → 读取数据 → 关闭文件。这些操作顺序固定，但各自目的不同。

其他选项辨析：

- 逻辑内聚：模块执行一组逻辑上相似的操作（如“打印、保存、删除”由参数决定），内聚度低；
- 通信内聚：所有处理使用相同的输入数据或产生相同的输出数据，但无顺序要求；
- 功能内聚：模块所有成分共同完成一个且仅一个功能（如“计算工资总额”），是最高内聚，不强调顺序而是功能统一性。
- 因此，强调“执行次序”而非“数据共享”或“单一功能”时，应选“过程内聚”。

43. 对于下图所示的程序流程图，为达到 100%的语句覆盖率，至少需要的测试用例数为（ A ）。

A. 5　　B. 6　　C. 7　　D. 8



解析：考察软件测试

答案是选项 A。

淘宝店：PanCode源文档源码库，原创

语句覆盖（Statement Coverage）要求：程序中每一个可执行语句（即每个处理节点和判断节点的入口）至少被执行一次。
首先，明确流程图中所有必须被覆盖的语句节点：

- **处理节点（非判断）：**节点 1、节点 2、节点 4、节点 6、节点 8。
- **判定节点（条件判断）：**节点 3、节点 5、节点 7、节点 9。
- 注意：在语句覆盖中，判定节点本身（如 if 语句）被视为一条可执行语句，只要程序执行到该判断处，即视为“覆盖”，无论其分支是否执行。但本题中，为了确保所有后续处理节点被执行，往往需要触发特定分支。

正确方法：使用圈复杂度估算最小路径数。

- 圈复杂度 V(G) = 判定节点数 + 1 = 4（节点 3,5,7,9） + 1 = 5 。
- 圈复杂度给出了线性无关路径数，也是实现语句覆盖和判定覆盖所需的最小测试用例数下限。

因此，至少需要 5 个测试用例才能覆盖所有语句（尤其在存在循环和多分支时，确保每条路径上的语句都被执行）。
这与选项 A. 5 一致。

44. CMM 认为，支撑软件质量的要素有三个。以下各选项中，不属于软件质量三要素的是（ C ）。
A. 人员素质　　B. 过程　　C. 管理　　D. 技术
解析：考察软件过程
答案是选项 C。
CMM（Capability Maturity Model，软件能力成熟度模型）的核心思想是“过程决定质量”，其明确支撑软件质量的三大核心要素为**人员素质、过程、技术**：

- 选项 A（人员素质）：指开发团队的专业能力、经验及协作水平，是软件质量的核心人力保障；
- 选项 B（过程）：指标准化的软件开发流程（如需求分析、设计、测试等环节的规范），是 CMM 强调的核心要素，过程的成熟度直接决定软件质量；
- 选项 C（管理）：属于过程实施的保障手段（如项目管理、质量管理），并非 CMM 定义的软件质量三大核心要素本身；
- 选项 D（技术）：指开发、测试所采用的工具、方法及技术架构，是实现软件功能与质量的技术支撑。

因此，“管理”不属于 CMM 定义的软件质量三要素。

45. 软件的正确性是指（ B ）。
A. 软件能正常运行　　B. 软件达到预期功能的程度
C. 软件符合用户的要求　　D. 软件运行中不出现任何错误
解析：考察软件质量
答案是选项 B。
软件质量的核心特性中，正确性的定义是“软件按照预先规定的功能和性能需求，准确执行其预期任务的程度”，核心聚焦“是否实现了预期功能”：

- 选项 A（软件能正常运行）：仅描述软件“可运行”，未涉及功能是否符合预期（如软件能运行但计算结果错误，仍不具备正确性）；
- 选项 B（软件达到预期功能的程度）：精准匹配正确性的定义，正确性并非绝对“无错误”，而是相对“符合预期功能”的程度（允许存在不影响核心功能的微小缺陷）；故选

淘宝店：tanjiaodaima 文档源码库，原创

B。

- 选项 C（软件符合用户的要求）：混淆了“正确性”与“用户满意度”，用户要求可能包含非功能需求（如界面美观），而正确性仅聚焦功能是否达到预先规定的预期；
- 选项 D（软件运行中不出现任何错误）：过于绝对，软件正确性不要求“零错误”，只需核心功能符合预期（如轻微的界面显示瑕疵不影响正确性），且完全无错误在实际软件中难以实现。

46. 结构化分析方法中的主要概念有（ D ）。

A. 模块内聚和耦合　　B. 对象及类

C. 模块和信息隐蔽　　D. 加工、数据流和数据源等

解析：考察软件设计-结构化分析

答案是选项 D。

结构化分析方法（Structured Analysis, SA）是面向过程的软件需求分析方法，其核心建模工具是**数据流图（DFD）**，强调从数据流动的角度理解系统功能。

DFD 的基本构成要素包括：

- 加工（Process / Transformation）：对输入数据进行处理并产生输出；
- 数据流（Data Flow）：数据在系统内部或与外部之间的移动；
- 数据存储（Data Store）：系统中保存数据的地方；
- 外部实体（External Entity / 数据源或数据潭）：系统边界外的数据提供者或接收者，也称为数据源/数据潭（Source/Sink）。

因此，选项 D 中的“加工、数据流和数据源等”正是结构化分析方法的核心概念。

A. 模块内聚和耦合：属于结构化设计（而非分析）阶段的概念，用于模块划分；

B. 对象及类：属于面向对象方法，与结构化分析无关；

C. 模块和信息隐蔽：是结构化设计或模块化编程的思想，出现在设计阶段，非分析阶段内容。

47. 软件需求很明确时，适用的软件开发模型是（ C ）。

A. 喷泉模型　　B. 增量模型　　C. 瀑布模型　　D. 螺旋模型

解析：考察软件过程

答案是选项 C。

不同软件开发模型的适用场景核心取决于需求的明确程度、项目风险等因素：

- 选项 C（**瀑布模型**）：是线性顺序模型，核心特征是“阶段化、文档驱动、不可逆”，要求需求在项目初期完全明确、稳定，后续按“需求分析→设计→编码→测试→维护”的固定流程推进，无迭代或很少迭代，完美匹配“需求很明确”的场景；故选 C。
- 选项 A（**喷泉模型**）：是面向对象的迭代模型，核心是“迭代、无明显阶段划分”，适用于需求动态变化、需持续迭代优化的项目；
- 选项 B（**增量模型**）：将软件拆分为多个可独立交付的增量，按优先级逐步开发，适用于需求基本明确但需快速交付核心功能、后续逐步完善的项目（并非“完全明确”的场景）；
- 选项 D（**螺旋模型**）：融合了瀑布模型和快速原型模型，核心是“风险驱动”，通过多轮“规划→风险分析→开发→评审”的螺旋迭代降低风险，适用于需求不明确、风险较高的大型复杂项目。

48. 软件测试过程模型给出了（ B ）。

淘宝店：CondaCode资料档源码库，原创

A. 用例设计过程　　B. 软件测试的要素以及它们之间的关系
C. 白盒测试技术的要点　　D. 测试覆盖准则

解析：考察软件测试

答案是选项 B。

软件测试过程模型（如 V 模型、W 模型、H 模型等）的核心目的是描述软件测试活动在整个软件开发生命周期中的位置、流程及其与其他开发活动的关系。

这类模型通常包括以下要素：

- 测试级别（单元测试、集成测试、系统测试、验收测试）；
- 对应的开发阶段；
- 测试目标与任务；
- 各测试活动之间的依赖和交互关系。

因此，测试过程模型不是专注于某一种测试技术（如白盒测试），也不是仅讲用例设计或覆盖准则，而是从宏观角度刻画测试作为一个系统工程所包含的要素及其相互关系。

选项分析：

- A. 用例设计过程：属于测试设计技术细节，非过程模型关注重点；
- B. 软件测试的要素以及它们之间的关系：正确，符合测试过程模型的定义；
- C. 白盒测试技术的要点：属于测试方法，与过程模型无关；
- D. 测试覆盖准则：属于测试充分性度量标准，是测试设计层面内容。

49. 教师是学校的一个部分，学校和教师之间是一种（ A ）关系。

A. 聚合　　B. 分类　　C. 泛化　　D. 依赖

解析：考察软件设计-面向对象方法

答案是选项 A。

题干核心是辨析面向对象中“整体与部分”的关联关系，关键在于区分不同 UML 关系的定义：

- 选项 A（聚合）：表示“整体与部分”的松散关联，部分可脱离整体独立存在。学校作为“整体”，教师作为“部分”，教师可脱离某所学校（如离职后加入其他学校）仍具有独立存在意义，完全符合聚合关系的“可分离”特征；故选 A。
- 选项 B（分类）：并非 UML 标准关系类型，面向对象中无此正式关联定义，属于干扰项；
- 选项 C（泛化）：表示“一般与特殊”的继承关系（如“教师”与“大学教师”），核心是子类继承父类属性和行为，与“整体 - 部分”无关；
- 选项 D（依赖）：表示对象间的临时关联（如“教师使用教案”），依赖关系中不存在“整体与部分”的逻辑，仅体现一方对另一方的使用依赖。

三、判断题

1. 结构化分析中，数据流图用于描述系统的控制流程。（ × ）

解析：考察软件设计-结构化设计

结构化分析中，数据流图（DFD）用于描述系统的数据流动和处理过程，而系统的控制流程由控制流图或流程图描述。二者分别关注数据流向和逻辑控制，不可混淆。所以该题错误。

淘宝店：hanc00d辉文档源码库，原创

2. 编程风格与程序的运行效率相关。（ √ ）

解析：考察软件工程管理

编程风格包括代码格式、命名规范、注释方式等，良好的编程风格可减少冗余代码、优化逻辑结构，间接提升程序运行效率（如简洁的条件判断减少执行步骤）。所以该题正确。

3. 结构化开发方法的核心指导原则是自顶向下、逐步求精。（ √ ）

解析：考察软件设计-结构化设计

结构化开发方法的核心原则是**自顶向下、逐步求精**，即从系统整体出发，逐层分解为更小的子系统或模块，逐步细化功能和实现细节，符合结构化分析与设计的思想。所以该题正确。

4. 测试没有发现问题，证明软件没有错误。（ × ）

解析：考察软件测试

测试的目的是发现错误，但 “未发现问题” 仅说明在当前测试范围内未找到错误，不能证明软件完全无错（可能存在未覆盖的测试场景或潜在缺陷）。软件测试无法完全证明软件无错，这是测试的局限性。所以该题错误。

5. 模块耦合中，数据耦合是最差的耦合类型。（ × ）

解析：考察软件设计-结构化设计

模块耦合中，**数据耦合是耦合程度最低、最理想的类型**（模块间仅传递基本数据）；而内容耦合（一个模块直接访问另一个模块的内部数据或代码）是最差的耦合类型，应尽量避免。

6. 结构化开发方法中，数据流图用于描述系统的控制流程。（ × ）

解析：

数据流图 (DFD) 描述系统的数据流动，而非控制流程。控制流程通常用流程图 (Flowchart) 或控制流图 (CFG) 表示。DFD 关注数据的来源、处理和去向，是结构化分析的核心工具。

考点关联：结构化方法 (DFD)

7. 软件设计中，模块的内聚性越高越好，耦合性越低越好。（ √ ）

解析：

内聚性衡量模块内部功能的相关性，耦合性衡量模块间的依赖程度。高内聚（功能单一）、

淘宝店：NandaCode原文档源码库，原创

低耦合（独立性强）是软件设计的核心原则，可提高可维护性和可复用性。
考点关联：软件设计原则（内聚 / 耦合）

8. 单元测试的主要目的是发现模块接口之间的错误。（ × ）

解析：
单元测试主要针对模块内部的代码逻辑，验证单个模块是否符合设计要求。发现模块接口错误是集成测试的主要目标。
考点关联：测试阶段划分（单元 / 集成）

9. 软件维护就是对软件进行修改和完善，不包括纠错。（ × ）

解析：
软件维护包括四类活动：纠错性维护（修复缺陷）、适应性维护（适应环境变化）、完善性维护（增强功能）和预防性维护（优化代码）。纠错是维护的重要组成部分。
考点关联：软件维护类型

10. 在面向对象编程中，类是对象的实例。（ × ）

解析：
类是对象的抽象模板，对象是类的实例。例如，“汽车”类的实例可以是“我的红色轿车”。关系应为：对象∈类。
考点关联：维护类型及面向对象基本概念（类与对象）

11. CMMI 是过程改进模型，它既支持连续性过程改进，也支持阶段性过程改进（ √ ）
解析：考察软件工程管理
CMMI(能力成熟度模型集成)是一种用于过程改进的模型,它同时支持阶段性过程改进,也支持连续性过程改进:

- **阶段性改进：**通过成熟度等级（从 1 级到 5 级）描述组织过程的进化路径，强调按阶段组织整体过程能力的提升。
- **连续性改进：**通过过程域的能力等级（从 0 级到 3 级）评估单个过程域的改进，允许组织根据自身需求选择特定过程域进行优化。

因此，题干说法正确。

12. 在面向对象程序中，程序运行时所有已创建的对象都始终处于活动状态。（×）
解析：考察软件设计-面向对象方法
在面向对象系统中，程序由对象及其交互构成，但对象的生命周期是动态的：

- 对象只有在被**实例化**（new 或类似操作）后才存在；
- 对象可能因作用域结束、引用丢失或显式销毁而被回收（如通过垃圾回收机制）；
- 即使对象存在于内存中，也可能处于**非活跃状态**（如未被调用、处于等待状态等），并非“始终活动”。

“活动状态”通常指对象正在执行方法、处理消息或参与计算。大多数对象在大部分时间处于**静默或休眠状态**，仅在接收到消息时才被激活。
因此，题干说法错误。

淘宝店：BaoDaCode源文档源码库 原创

13. 单元测试的主要目的是发现模块接口之间的错误。（ × ）

解析：考察软件测试

单元测试主要针对模块内部的代码逻辑，验证单个模块是否符合设计要求。

发现模块与模块之间接口错误，是集成测试阶段的主要任务之一。

因此，题干说法错误。

14. 软件设计中，模块的内聚性越高越好，耦合性越高越好。（ × ）

解析：考察软件设计

- 概念：内聚性衡量一个模块内部各成分之间相互关联的紧密程度。
- 设计目标：内聚性越高越好。高内聚意味着模块功能单一、职责明确，便于理解、复用和维护。这是正确的。
- 概念：耦合性衡量不同模块之间相互依赖的紧密程度。
- 设计目标：耦合性越低越好。低耦合意味着模块间接口简单、依赖关系弱，一个模块的修改不会轻易“牵一发而动全身”地影响其他模块，从而提高系统的可维护性和可扩展性。

优秀的软件设计追求“**高内聚、低耦合**”。题目中“耦合性越高越好”的说法完全违背了这一核心原则。因此，该判断题表述错误。

16．在软件开发措施中，构造化措施总的指导原则是自顶向下、逐渐求精。（√）

解析：考察软件设计-结构化方法

结构化方法（包括结构化分析、结构化设计、结构化编程）的核心指导原则就是“自顶向下、逐渐求精”：

- **自顶向下：**从系统整体功能出发，将复杂系统逐层分解为更简单的子系统或模块，先明确顶层（整体）目标，再逐步细化底层（局部）功能。
- **逐渐求精：**对每个分解后的模块，逐步补充细节（如数据流程、处理逻辑），从抽象的功能描述逐步转化为具体的实现方案，避免一次性处理所有复杂细节。

这一原则贯穿结构化方法的全流程，是其区别于其他早期开发方法的关键特征，因此题干说法正确。

17．当一种模块直接修改另一种模块的数据时，模块间的这种耦合称为数据耦合。（×）

解析：考察软件设计

模块间直接修改对方数据的是内容耦合而非数据耦合。

- **内容耦合：**是各种耦合中最严重的一种。当一个模块不通过正常入口直接访问并修改另一个模块的内部数据或控制流时，即存在内容耦合。这破坏了被修改模块的封装性，因此通常被视为设计不良，应尽量避免。
- **数据耦合：**在数据耦合中，模块之间通过参数传递数据，而不是共享数据。这是一种较为理想的耦合方式，因为它保持了模块间的独立性和松散联系。在这种耦合方式下，模块通过定义良好的接口来交换数据，内部实现细节则互相隐藏。
- **其他类型的耦合：**还有标记耦合、控制耦合、外部耦合等，它们的耦合程度介于数据耦合与内容耦合之间。标记耦合涉及到模块间通过数据结构传递信息，而控制耦合则是通过控制变量影响被调模块的行为。

因此，题干中对“数据耦合”的描述错误，实际应为内容耦合，说法错误。

淘宝店：PandaCode程序文档资源库，原创

18．测试用例只需选用合理的输入数据。（×）

解析：考察软件测试

测试用例的核心目标是全面验证软件功能的正确性与鲁棒性，不能仅选用合理的输入数据，需覆盖多种场景：

- **合理输入数据：**用于验证软件在正常场景下是否按预期执行（如查询成绩时输入正确的学号和课程名），这是测试用例的基础部分，但非全部。
- **不合理输入数据：**需包含异常、边界或非法输入（如输入不存在的学号、超出范围的数值、非预期格式的字符），用于验证软件的容错能力（如是否能提示“输入错误”而非崩溃），这是保障软件稳定性的关键测试场景。

因此，题干中“只需选用合理的输入数据”的说法错误。

19．边界值分析法是一种白盒测试法。（×）

解析：考察软件测试

边界值分析法属于黑盒测试法，而非白盒测试法，核心区别在于测试视角与依据不同：

- **黑盒测试法：**不关注软件内部逻辑结构和代码实现，仅依据软件的需求规格说明书，通过输入输出验证功能是否符合预期。边界值分析法正是基于需求中的“边界条件”（如输入的最大值、最小值、临界值）设计测试用例，无需了解代码细节，符合黑盒测试的特征。
- **白盒测试法：**需深入软件内部，依据代码逻辑、分支结构（如 if-else、循环）设计测试用例，目的是覆盖代码的所有执行路径（如语句覆盖、判定覆盖），与边界值分析法的测试逻辑完全不同。

因此，题干中“边界值分析法是一种白盒测试法”的说法错误。

20．关联是建立类之间关系的一种手段。（√）

答：考察软件设计-面向对象方法

在面向对象设计中，关联（Association）是 UML 中定义类之间关系的核心手段之一，主要用于描述类的实例之间的连接或交互关系：

- 关联可以明确类与类之间的业务关联（如“学生”类与“课程”类之间的“选课”关联），体现实例间的静态联系。
- 除关联外，类之间的关系还包括继承、依赖、聚合、组合等，但关联是最基础、最常用的关系类型，直接用于建立类与类之间的核心业务连接，是构建面向对象模型的重要手段。

因此，题干中“关联是建立类之间关系的一种手段”的说法正确。

21．对象把数据和处理数据的操作结合为一体。（√）

解析：考察软件设计-面向对象方法

面向对象思想的核心特征之一是“封装”，即对象将其内部的数据（属性）和对数据的操作（方法）封装为一个整体，外部只能通过对象提供的接口（方法）访问或修改数据，而无法直接操作内部数据。这种“数据 + 操作”的结合形式，是面向对象方法区别于结构化方法中“数据与操作分离”的关键特征。因此，题干说法正确。

22．程序是一些对象的集合，在程序运行时对象都处于活动状态。（×）

解析：考察软件设计-面向对象方法

淘宝店：Pro&Code原文档题库，原创

题干存在两处关键偏差：

- **程序的构成表述不准确：**面向对象程序的核心是“对象的交互”，但程序不仅包含对象，还需通过对象间的消息传递（如调用方法）实现功能，并非单纯“对象的集合”；且非面向对象程序（如结构化程序）根本不基于对象构建，更不存在“对象集合”的说法。
- **对象活动状态表述错误：**程序运行时，并非所有对象都处于活动状态。部分对象可能仅在特定场景下被创建（如用户触发操作时），或创建后因完成任务被销毁，存在“未创建”“已销毁”等非活动状态，并非全程活动。

因此，题干说法错误。

23. 用计数方法而不是用文件结束符或输入序列结束符来判断程序输入的结束。（×）

解析：考察软件设计

程序判断输入结束的方式需根据场景选择，不存在“必须用计数方法”的固定规则，核心问题在于题干表述的“绝对性”：

- **计数方法的局限性：**仅适用于“输入数据量已知”的场景（如明确需输入 10 个学生成绩），若输入数据量未知（如用户随机输入文本直到主动停止），计数方法完全无法适用。
- **其他合法结束方式的必要性：**实际开发中，文件结束符（如 C 语言的 EOF）、输入序列结束符（如用户输入特定字符“#”）是更常用的结束判断方式，尤其适用于数据量不确定的场景，这些方式与计数方法同等合法，无优劣之分。

因此，题干中“用计数方法而不是用其他方式”的绝对化表述错误，输入结束判断需结合具体需求选择合适方式。

24. 在面向对象系统中，类与对象的关系是抽象与具体的关系。（√）

解析：考察软件设计-面向对象方法

在面向对象系统中：

- **类（Class）：**是对一类具有相同属性（数据）和方法（操作）的对象的抽象描述，是一个概念性的模板，不直接对应具体实体（如“学生”类是对所有学生的抽象）。
- **对象（Object）：**是类的实例化结果，是具体存在的实体，具备类所定义的属性和方法（如“张三”是“学生”类的一个具体对象）。

因此，类是对对象的抽象概括，对象是类的具体体现，二者是抽象与具体的关系，题干说法正确。

25. 接受消息的对象必须回答消息发送者。（×）

解析：考察软件设计-面向对象方法

面向对象系统中，消息传递是对象交互的核心方式，但“接受消息的对象必须回答发送者”的表述不符合实际逻辑，关键原因如下：

- **消息类型决定是否需“回答”：**消息分为“请求型消息”（如调用对象的查询方法，需返回结果，如“获取用户年龄”）和“通知型消息”（仅告知对象执行操作，无需返回结果，如“弹出提示框”“记录日志”）。对于通知型消息，接收对象执行操作后无需向发送者反馈，即无需“回答”。
- **“回答”并非强制规则：**消息传递的核心是“触发操作”，而非“强制反馈”。是否需要返回结果，取决于业务逻辑设计（如仅需执行动作时，无需返回），不存在“必须回答”的强制要求。

因此，题干中“必须回答”的绝对化表述错误。

- **总结面向对象核心概念（类、对象、消息）关系：**

淘宝店：Panda Code辉文档原创

- **类**是“抽象模板”，**对象**是“具体实例”，二者是“抽象与具体”的关系；
- **消息**是对象之间的“通信方式”，依赖类定义的方法实现交互，是面向对象系统中功能协作的核心纽带。

26. CMMI 是过程改进模型，它支持阶段性过程改进不支持连续性过程改进（×）

解析：考察软件过程

CMMI(能力成熟度模型集成)是一种用于过程改进的模型,它同时支持阶段性过程改进,也支持连续性过程改进:

- **阶段性改进：**通过成熟度等级（从 1 级到 5 级）描述组织过程的进化路径，强调按阶段组织整体过程能力的提升。
- **连续性改进：**通过过程域的能力等级（从 0 级到 3 级）评估单个过程域的改进，允许组织根据自身需求选择特定过程域进行优化。

因此，题干中“不支持连续性过程改进”的说法错误。

27. 某项目组织打算开发一个大型系统，且具备了相关领域知识以及开发经验，此时采用瀑布模型是最好的选择（×）

解析：考察软件过程

瀑布模型的核心是按“需求分析→设计→编码→测试→维护”的线性阶段推进，要求需求在初期完全明确且稳定。虽然题干中提到“具备相关领域知识和开发经验”，但大型系统通常存在需求复杂、易变的特点:

- 即使有经验，大型系统的需求也可能在开发过程中因业务变化、用户反馈等发生调整，瀑布模型难以适应这种变化，修改成本高。
- 对于大型系统，更适合采用迭代模型（如 RUP）、敏捷模型等，通过分阶段交付和持续反馈应对需求变化，降低风险。

因此，“采用瀑布模型是最好的选择”说法错误。

28. 采用 UML 进行软件设计时，可用依赖关系表示两类事物的“整体/部分”关系（×）

解析：考察软件设计-面向对象方法

在 UML（统一建模语言）中，不同关系有明确的语义和用途，不可混淆:

- **依赖关系：**表示一个事物（客户端）依赖另一个事物（供应端）的功能或服务，客户端的实现需要供应端支持（如“类 A 调用类 B 的方法”），是一种弱关联，不体现“整体 / 部分”的包含或组成关系。
- **“整体 / 部分”关系：**需通过 UML 的**聚合关系**（整体与部分可独立存在，如“班级与学生”）或**组合关系**（整体与部分不可分割，如“汽车与发动机”）表示，二者均属于关联关系的特殊形式，而非依赖关系。

因此，题干中“用依赖关系表示‘整体 / 部分’关系”的说法错误。

29. 质量控制不是软件配置管理活动（√）

解析：考察软件工程管理

软件配置管理 (SCM) 和软件质量控制 (QC) 是软件工程中两个独立且目标不同的活动:

- **软件配置管理活动：**核心是对软件生命周期中的配置项（如代码、文档、数据）进行标识、控制、审计和报告，确保配置项的完整性、一致性和可追溯性，具体包括配置标识、版本控制、变更控制、配置审计等。
- **软件质量控制活动：**核心是通过测试、评审等手段验证软件是否符合质量要求（如功能

淘宝店：Good Code原文档源码库，原创

正确性、性能、可靠性），识别并修正质量缺陷，具体包括测试（黑盒 / 白盒测试）、代码评审、质量度量分析等。

二者目标和范畴不同，质量控制不属于软件配置管理的活动范畴，因此题干说法正确。

30. 风险预测从两个方面，即风险发生的可能性跟风险消除方法（×）

解析：考察软件工程管理

风险预测（也称为风险评估）的核心是识别和分析潜在风险对项目的影响，其关键两个维度是风险发生的可能性（概率） 和风险发生后的影响程度（后果严重性），而非“风险消除方法”：

- **“风险发生的可能性”：**评估风险事件发生的概率高低（如“需求变更的概率为 30%”）；
- **“风险影响程度”：**评估风险若发生，对项目范围、进度、成本、质量等目标的破坏程度（如“严重影响开发进度，导致延期 2 周”）；
- **“风险消除方法”：**属于风险应对阶段的内容，是在风险预测后，针对已识别的风险制定的处理策略（如规避、转移、缓解风险），与风险预测的核心维度无关。

因此，题干中“风险预测从风险发生的可能性跟风险消除方法两个方面”的说法错误。

31. 面向对象分析方法的常用工具是用例图（×）

解析：考察软件设计-面向对象方法

面向对象分析方法的常用工具**不仅包括用例图，还有类图、对象图、状态图、活动图等**。用例图主要用于描述系统的功能需求（用户与系统的交互），是面向对象分析中“需求建模”的工具之一，但不能说“常用工具是用例图”（表述过于单一、片面）。因此该表述错误。

32. 若对象 a 是对象 b 的一个组成部分，则对象 a 是整体对象，对象 b 是部分对象。（×）

解析：考察软件设计-面向对象方法

对象间的“组成部分”关系（即聚合或组合关系）中，“整体”与“部分”的定义与题干表述相反：

- 若对象 a 是对象 b 的组成部分，则对象 b 是整体对象（包含 a），对象 a 是部分对象（被 b 包含）。例如，“汽车（b）包含发动机（a）”，汽车是整体，发动机是部分。
- 题干颠倒了“整体”与“部分”的关系，因此表述错误。

33．测试用例只需选用合理的输入数据。（×）

解析：考察软件测试

测试用例的核心目标是全面验证软件功能的正确性与鲁棒性，不能仅选用合理的输入数据，需覆盖多种场景：

- **合理输入数据：**用于验证软件在正常场景下是否按预期执行（如查询成绩时输入正确的学号和课程名），这是测试用例的基础部分，但非全部。
- **不合理输入数据：**需包含异常、边界或非法输入（如输入不存在的学号、超出范围的数值、非预期格式的字符），用于验证软件的容错能力（如是否能提示“输入错误”而非崩溃），这是保障软件稳定性的关键测试场景。

因此，题干中“只需选用合理的输入数据”的说法错误。

34. 程序是一些对象的集合，在程序运行时对象都处于活动状态。（×）

解析：考察软件设计-面向对象方法

题干存在两处关键偏差：

淘宝搜索 MonkaCode 源码库，原创

- **程序的构成表述不准确：**面向对象程序的核心是“对象的交互”，但程序不仅包含对象，还需通过对象间的消息传递（如调用方法）实现功能，并非单纯“对象的集合”；且非面向对象程序（如结构化程序）根本不基于对象构建，更不存在“对象集合”的说法。
- **对象活动状态表述错误：**程序运行时，并非所有对象都处于活动状态。部分对象可能仅在特定场景下被创建（如用户触发操作时），或创建后因完成任务被销毁，存在“未创建”“已销毁”等非活动状态，并非全程活动。

因此，题干说法错误。

35. 软件设计中，模块的内聚性越高越好，耦合性越低越好。（√）

解析：考察软件设计

内聚性衡量模块内部功能的相关性，耦合性衡量模块间的依赖程度。高内聚（功能单一）、低耦合（独立性强）是软件设计的核心原则，可提高可维护性和可复用性。因此，题干说法正确。

36. 软件的开发成本不但需要考虑开发的人力消耗，还要考虑期间的其他经常性消耗。（√）

解析：考察软件工程管理

以下是软件开发成本估算时需要关注的几个方面：

- **直接人力成本**：这是软件开发成本中最主要的部分，涉及到需求分析、设计、编码、测试等环节的人力资源消耗。不同职级的人员具有不同的标准成本，计算时需根据实际工作人天和每人每月的工作天数来核算。
- **间接人力成本**：除了直接参与项目的人员成本外，还应考虑项目管理、质量保证、配置管理等活动所产生的人力成本。
- **直接非人力成本**：这包括软件研发过程中使用的材料、设备租赁、外包服务费用等。
- **间接非人力成本**：例如，为开发团队提供的办公环境、电力、水费等基础设施成本也应被计入软件开发成本之中。
- **规模和工作量估算**：在计算成本之前，首先需要对软件的规模和所需的工作量进行估算，这将影响最终的成本估算结果。
- **成本估算方法**：可以采用方程法、类比法或类推法等多种方法进行成本估算，同时应充分利用基准数据来提高估算的准确性。
- **费率选择**：在计算人力成本时，可以根据行业基准费率或企业自身的人力成本费率来确定每项工作的造价。

因此，题干表述正确。

37. 模块的独立程度可以使用两个标准来衡量，这两个标准分别是模块的内聚度和模块之间的耦合度。它们属于定性的标准（√）

解析：考察软件设计

模块独立性的衡量与定性属性判断均符合题干描述：

- **衡量标准的正确性：**模块独立程度的核心衡量指标确实是**内聚度（模块内部元素的关联紧密程度）和耦合度（模块之间的依赖紧密程度）**。内聚度越高、耦合度越低，模块独立性越强。
- **定性标准的合理性：**内聚度（如通信内聚、过程内聚）和耦合度（如数据耦合、控制耦合）的划分，是基于“关联紧密程度”的主观判断，而非可量化的数值（如“内聚度 = 80 分”）。例如，判断一个模块是“通信内聚”还是“过程内聚”，需依据模块内部逻辑

淘宝店：PanYuCode辉文资源码库，原创

的关联特征，属于定性评估，而非定量计算。
因此，题干表述正确。

38. 描述一个模块内的处理流程时，一种改进的方法是使用 N-S 图 （盒图），与最常用的程序流程图相比，它的优点是完全避免了 GOTO 转移，彻底遵循了结构化程序设计的思想。（ √ ）
解析：考察软件设计-结构化方法
N-S 图（盒图）与程序流程图的核心差异及优势符合题干描述：

- **N-S 图的核心设计特点：**采用“盒状结构”描述处理流程，仅包含顺序、选择、循环三种基本结构化控制结构，且所有结构均通过嵌套或并列的盒体表示，不存在程序流程图中的箭头（GOTO 转移） 。
- **结构化程序设计思想的契合度：**结构化程序设计要求避免无规律的 GOTO 跳转，仅用基本控制结构构建程序逻辑。N-S 图通过自身结构强制限制了非结构化跳转，从形式上彻底遵循该思想；而程序流程图可通过箭头随意跳转，易出现非结构化逻辑。

因此，题干对 N-S 图优点的描述正确。

39. 描述一个模块内的处理流程时，一种改进的方法是使用 PAD 图，与最常用的程序流程图相比，它的优点是完全避免了 GOTO 转移，彻底遵循了结构化程序设计的思想。（ √ ）
解析：考察软件设计-结构化方法
答案是：正确。
PAD 图（问题分析图）的设计特性与结构化程序设计思想高度契合，其优势符合题干描述：

- **PAD 图的核心结构特点：**采用树形或网状的图形符号，仅支持顺序、选择、循环三种基本结构化控制结构，且控制流程只能从顶部向下、从左向右单向流动，不存在程序流程图中可任意跳转的箭头（GOTO 转移），从形式上杜绝了非结构化逻辑。
- **与结构化思想的一致性：**结构化程序设计的核心是“无 GOTO、仅用基本控制结构构建流程”，PAD 图通过固定的图形逻辑强制实现这一要求；而程序流程图可通过箭头随意改变执行顺序，易产生难以维护的非结构化代码。

因此，题干对 PAD 图优点的描述正确。

40. 在描述系统功能时常使用用例图建模，但也需要辅之以规格说明，即用例实现的场景。场景从用户角度描述每一个功能处理的事件序列。（ √ ）
解析：考察软件设计
用例图与场景规格说明的配合关系及场景的定义，均符合题干描述：

- **用例图的局限性与补充需求：**用例图仅能通过“参与者 - 用例 - 系统”的关系，宏观展示系统功能范围（如“学员”参与“报名课程”用例），但无法描述用例内部的具体执行步骤（如报名需“提交信息→审核课程→确认缴费”），因此必须辅之以规格说明。
- **场景的核心属性：**场景是用例实现的具体事件序列，且始终从用户（参与者）角度出发（如以“学员”视角描述“报名课程”的每一步操作与系统响应），清晰呈现功能处理的完整流程，弥补用例图在细节描述上的不足。

因此，题干表述正确。

41. 软件配置管理是一种保护伞活动，它贯穿于整个软件生存周期过程，用于控制变更。（ √ ）
解析：考察软件过程
软件配置管理（SCM）的核心属性与题干描述一致：

- **保护伞活动的定位：**软件工程中的“保护伞活动”指贯穿软件生存周期全过程的支持性

淘宝店铺：xiangdao001 资料源题库，原创

活动（如配置管理、质量保证、项目管理）。软件配置管理从项目启动到维护阶段持续运行，覆盖需求文档、设计方案、代码、测试用例等所有“配置项”，符合保护伞活动的特征。

- **控制变更的核心目标：**软件配置管理的核心任务包括“标识配置项、版本控制、变更控制、配置审计”等，其中变更控制通过规范变更申请、评估、审批、实施流程，确保变更有序进行（如避免未经授权的代码修改），有效控制变更带来的风险。

因此，题干表述正确。

42. 若对象 a 是对象 b 的一个组成部分，则对象 a 是整体对象，对象 b 是部分对象。（×）

解析：考察软件设计-面向对象方法

对象间的“组成部分”关系（即聚合或组合关系）中，“整体”与“部分”的定义与题干表述相反：

- 若对象 a 是对象 b 的组成部分，则对象 b 是整体对象（包含 a），对象 a 是部分对象（被 b 包含）。例如，“汽车（b）包含发动机（a）”，汽车是整体，发动机是部分。
- 题干颠倒了“整体”与“部分”的关系，因此表述错误。

43. 边界值分析法是一种常用的白盒测试的测试用例设计方法。（×）

解析：考察软件测试

边界值分析法的测试用例设计逻辑与白盒测试的核心特征不匹配：

- **白盒测试：**基于对程序内部逻辑结构（如代码、控制流、数据流）的了解设计测试用例，关注“如何通过代码路径覆盖验证逻辑正确性”（如语句覆盖、分支覆盖）。
- **边界值分析法：**属于黑盒测试方法，它不依赖程序内部结构，仅根据软件的输入输出规格（如功能需求），选取输入域边界或边界附近的值作为测试用例（如输入范围为 1~100 时，测试 0、1、100、101 等），核心是验证边界条件下的功能正确性。

因此，边界值分析法是黑盒测试方法，题干表述错误。

表4-12 软件测试的基本概念

软件测试	基本概念
定义	测试是为了发现程序中的错误而执行程序的过程，其不能证明程序中没有错误
目标	①好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案； ②成功的测试是发现了至今为止尚未发现的错误的测试； ③根本目标是尽可能多地发现并排除软件中潜藏的错误，最终把一个高质量的软件系统交给用户使用； ④测试的目标是发现错误，调试的任务是在测试的基础上去改正错误
测试方法	①白盒测试（结构测试）：把程序看成装在一个透明的白盒子里，测试者完全知道程序的结构和处理算法，按照程序内部的逻辑测试程序，检测程序中的主要执行通路是否都能按预定要求正确工作； ②黑盒测试（功能测试）：把程序看作一个黑盒子，完全不考虑程序的内部结构和处理过程，是在程序接口进行的测试，只检查程序功能是否能按照规格说明书的规定正常使用

淘宝店：CSDNcoco源码库 原创

表4-15　黑盒测试技术的基本概念

黑盒测试技术	基本概念
等价划分	把程序的输入域划分成若干个数据类，选取每个等价类中的一个典型值或任意值作为测试数据
边界值分析	确定边界情况，选取刚好等于、稍小于和稍大于等价类边界值的数据作为测试数据
错误推测	利用判定表或判定树为工具，列出输入数据各种组合与程序所发生的动作以及相应的输出数据之间的相互对应关系，然后为判定表的每一列至少设计一个测试用例

44. 在使用 MSD 的变换分析方法建立系统模块结构时，模块结构的主模块应是由 DFD 的变换中心部分映射而来。（√）

解析：考察软件设计-结构化方法

MSD（模块结构图）的变换分析方法是从 DFD（数据流图）映射系统模块结构的核心技术，其主模块的来源符合题干描述：

- **变换分析的核心逻辑：**DFD 可分为“输入流”（数据进入系统）、“变换中心”（对数据进行核心处理）、“输出流”（处理后的数据离开系统）三部分。变换分析方法通过将这三部分映射为模块，构建模块结构。
- **主模块的映射来源：**变换分析中，模块结构的主模块（顶层模块）对应 DFD 的“变换中心”，负责协调输入模块（处理输入流）和输出模块（处理输出流）的工作，控制整个系统的核心处理流程。例如，若 DFD 的变换中心是“订单处理”，则主模块即为“订单处理控制模块”，统筹输入（接收订单）和输出（生成发货单）。

因此，题干表述正确。

45. MVC 架构是一种典型的软件体系结构，它主要包括三部分，即表示层、控制逻辑层和数据层。（√）

解析：考察软件设计

MVC（Model-View-Controller）架构的核心组成与题干描述一致：

- **MVC 的三部分构成：**
  - **Model（模型）：**对应“数据层”，负责管理应用程序的数据和业务逻辑（如数据库交互、数据验证），独立于用户界面；
  - **View（视图）：**对应“表示层”，负责数据的展示（如网页、APP 界面），用户通过视图与系统交互；
  - **Controller（控制器）：**对应“控制逻辑层”，接收用户输入（如点击、表单提交），协调模型与视图的交互（如调用模型处理数据，通知视图更新展示）。
- **与题干的一致性：**题干中“表示层、控制逻辑层、数据层”分别对应 MVC 的 View、Controller、Model，准确概括了 MVC 架构的核心组成。

因此，题干表述正确。

46. 系统的性能刻画了功能执行达到什么样的效果，因此，在考虑需求时功能和性能必须一起考虑。（√）

解析：考察软件需求

该表述是正确的。在软件需求工程中，功能需求和非功能需求（性能是其中最重要的一类）共同构成了完整的软件需求规格。

- **功能需求：**定义了系统“做什么”，即系统必须执行的功能或服务。
- **性能需求：**定义了系统“做到什么程度”，即功能执行的效果，例如响应时间、吞吐量、

淘宝店：ProdCode软文代码库 原创

资源利用率等。

在软件需求工程中，**功能需求与非功能需求（包括性能）必须同步分析和定义**。若仅定义功能而忽略性能，可能导致系统虽能运行但无法满足实际使用场景（如高延迟、低并发等）。

47. 对于规模较大的模块，应通过分解以提高其内聚性，从而降低其模块独立性。（×）

解析：考察软件设计

该表述错误，关键问题在于后半句“从而降低其模块独立性”。

模块独立性是软件设计的重要原则，由两个核心指标衡量：

- **高内聚（High Cohesion）：**模块内部元素紧密相关；
- **低耦合（Low Coupling）：**模块之间依赖尽可能少。

当一个模块规模过大、功能混杂时，通常内聚性较低。此时，将其分解为多个小模块，可以使每个新模块聚焦单一职责，提高内聚性，同时减少与其他模块的不必要关联，增强模块独立性（而非降低）。

48. 如果一个剧组包含许多演员，每个演员又可以是其他剧组的成员，那么剧组和演员之间的关系应属于一种特殊的聚合关系，即组合关系。（×）

解析：考察软件设计

该表述是错误的，原因在于它混淆了聚合与组合这两种关联关系。

- **聚合关系：**表示一种“部分-整体”的关系，但部分可以独立于整体而存在。题目中的描述“每个演员又可以是其他剧组的成员”正是聚合关系的典型特征。演员（部分）的生命周期并不依赖于某个特定的剧组（整体），他们可以自由地在不同剧组间流动。
- **组合关系：**是一种更强形式的“部分-整体”关系，具有严格的从属关系。在组合中，部分不能独立于整体而存在，整体的生命周期决定了部分的生命周期。例如，“公司”与“部门”之间的关系就更接近组合，因为部门不能脱离公司而独立存在。如果公司解散，其下属的部门也随之不复存在。
- 因此，剧组和演员之间的关系是标准的聚合关系，而非题目中所述的组合关系。

49. 等价类划分和边界值方法都没有考虑输入条件之间的关系。（√）

解析：考察软件测试

该表述是正确的。

- **等价类划分：**该方法将程序的输入域划分为若干个子集（有效等价类和无效等价类），使得在同一子集中的数据对于发现错误是等价的。它主要关注单个输入条件的取值情况，并从每个等价类中选取少数代表值进行测试。
- **边界值分析：**该方法是对等价类划分的补充，它关注输入等价类或输出等价类的边界，因为边界附近区域是错误的高发区。它同样主要针对单个输入条件的边界值进行测试。
- **这两种经典的黑盒测试技术**，其核心思想都是基于对单个输入变量的分析。它们没有系统地考虑多个输入条件之间可能存在的组合关系或依赖关系。若要测试输入条件之间的组合情况，需要使用其他测试技术，如判定表、因果图或正交实验法等。

50. 最常用的软件规模度量方法是代码行度量和功能点度量。（√）

解析：考察软件项目管理

该表述是正确的。

软件规模度量是软件工程中估算工作量、成本和进度的基础，代码行度量（LOC，Lines of Code） 和功能点度量（FP，Function Point） 是行业内最常用的两种核心方法：

淘宝搜：PandaCode软考资源码库，原创

- **代码行度量：**通过统计实际编写的源代码行数（如有效代码行、非注释行）衡量软件规模，特点是简单直观、易于统计，适用于已有代码或明确技术实现的项目，但依赖编程语言（不同语言行数差异大）、无法在需求阶段估算；
- **功能点度量：**基于软件的功能需求（如外部输入、外部输出、数据存储等）进行量化估算，不依赖具体技术实现，可在需求分析阶段早期开展，适用于跨语言、跨平台的项目规模对比，是需求阶段常用的规模度量方法。

51. 在对象的整个生命周期中，不同的对象可以具有相同的标识。（×）

解析：考察软件设计-面向对象方法

面向对象方法学中，**对象标识（Object Identifier, OID）是对象的唯一标识**，核心特征是“全局唯一性”：

- 对象标识用于区分不同对象，无论对象的属性、状态如何变化，其标识在整个生命周期中保持不变；
- 不同对象（即使是同一类的不同实例）必须具有不同的标识，否则无法在系统中区分对象的独立存在（例如：两个“教师”对象，即使姓名、年龄相同，也需通过唯一标识区分）；
- 题干中“不同的对象可以具有相同的标识”违背了对象标识的唯一性原则，因此判断为错误。

52. 模块间通过参数传递基本类型的数据，模块间的这种耦合称为数据耦合。（√）

解析：考察软件设计

耦合性是模块间依赖关系的度量，**数据耦合**是软件工程中低耦合的典型类型，其核心定义与题干描述完全一致：

- 数据耦合的定义：模块之间仅通过传递**基本类型的数据参数**（如整数、字符串、布尔值等）进行交互，模块间不共享复杂数据结构（如数组、对象）、不直接访问对方的内部状态或功能，仅依赖输入输出数据的逻辑关联；
- 题干中“模块间通过参数传递基本类型的数据”精准匹配数据耦合的核心特征，这种耦合方式独立性强、维护成本低，是软件设计中推荐的低耦合形式；
- 需区分数据耦合与其他耦合类型（如控制耦合：传递控制信号；公共耦合：共享公共数据区），题干描述未涉及其他耦合的特征，因此判定为正确。

53. 以结构化分析方法建立的系统模型就是数据流图。（×）

解析：考察软件设计-结构化分析

题干的核心错误在于将结构化分析方法的系统模型等同于单一的数据流图，忽略了结构化分析模型的完整性：

结构化分析的系统模型是一个“模型体系”，核心由三大要素构成，数据流图（DFD）仅为其中之一：

- 数据流图（DFD）：描述系统的功能逻辑和数据流转（“做什么”），是动态模型；
- 数据字典：定义数据流图中所有数据元素、数据结构、加工逻辑、数据存储等的详细说明，是静态数据描述；
- 加工说明（如结构化语言、判定表、判定树）：详细描述数据流图中每个“加工”的具体处理规则，补充 DFD 无法体现的细节逻辑。

数据流图仅能直观展示数据的来源、处理、存储和去向，但无法说明数据的具体定义、加工的内部逻辑，必须与数据字典、加工说明配合，才能构成完整的结构化分析系统模型。

淘宝店：Panda666解析资料源码库 原创

54. CMM(能力成熟度模型)的第三级是已定义级。(√)

解析: 考察软件过程

该表述是正确的。CMM（能力成熟度模型）将软件过程的成熟度分为五个等级，从低到高依次为:

- **初始级：**过程不可预测且缺乏控制。
- **可重复级：**建立了基本的项目管理过程，能重复以往的成功实践。
- **已定义级：**软件过程均已文档化、标准化，并集成为组织的标准软件过程。
- **已管理级：**对软件过程和产品质量有详细的度量，并能进行定量的控制和管理。
- **优化级：**能够基于过程的量化反馈，以及新思想和新技术，持续改进过程。

因此，第三级确实是"已定义级"。该级别的核心特征是：过程已经文档化、标准化，并能够被组织内部的所有项目一致地遵循。

55. 面向对象分析方法与结构化分析方法的区别之一是一个使用“抽象”，而另一个没有使用“抽象”。(×)

解析: 考察软件设计

该判断题表述错误。

“抽象”是软件工程中所有分析与设计方法共有的基本思想，并非面向对象方法所独有。

（1）**结构化分析方法**（如使用数据流图 DFD）同样运用了抽象:

- 将系统抽象为"加工"（处理）、"数据流"、"数据存储"和"外部实体";
- 通过逐层分解（如顶层图 → 0 层图 → 1 层图）实现**功能抽象与逐步求精**;
- 忽略具体实现细节，聚焦于"做什么"而非"怎么做"。

（2）**面向对象分析方法**则通过**类、对象、继承、封装**等机制进行抽象，强调**数据与行为的统一抽象**。

因此，两种方法都使用"抽象"，区别在于抽象的方式和视角不同:

- **结构化方法：功能/过程抽象（面向过程）;**
- **面向对象方法：对象/实体抽象（面向数据与行为）。**

题干错误地认为"结构化方法没有使用抽象"，这是对软件工程基本概念的误解。

四、简单题

1. 简述软件生命周期的几个阶段及其主要任务。

解析:

软件生命周期通常分为以下阶段，各阶段的主要任务如下:

- **(1) 可行性研究与计划阶段**
    - **任务**:
        - 确定项目的可行性（技术、经济、操作等）;
        - 制定初步的项目开发计划（如进度、资源、成本估算）;
        - 编写可行性研究报告和项目开发计划。
- **(2) 需求分析阶段**

淘宝店：Panda_Code 微关注源码库 原创

- **任务**:
    - 深入调研用户需求，明确软件的功能、性能、接口及约束条件;
    - 建立系统的逻辑模型（如数据流图、数据字典）;
    - 编写需求规格说明书，作为后续开发的基准。

- **(3) 软件设计阶段**

    - **任务**:
        - **概要设计（总体设计）**：划分模块结构，定义模块间的调用关系和接口，设计数据结构;
        - **详细设计**：为每个模块设计具体的算法和内部数据结构（如流程图、N-S图），确保可实现性。
        - 输出设计文档（如软件设计说明书）。

- **(4) 编码阶段（实现阶段）**

    - **任务**:
        - 根据详细设计文档，选择编程语言实现各个模块;
        - 遵循编码规范，确保代码的可读性和可维护性;
        - 生成可运行的程序代码。

- **(5) 软件测试阶段**

    - **任务**:
        - 设计测试用例，对软件进行单元测试、集成测试、确认测试和系统测试;
        - 发现并修复缺陷，验证软件是否满足需求规格说明;
        - 编写测试报告，评估软件质量。

- **(6) 软件维护阶段**

    - **任务**:
        - **纠错性维护**：修复运行中发现的缺陷;
        - **适应性维护**：调整软件以适应环境变化（如硬件升级、操作系统更新）;
        - **完善性维护**：增加新功能或优化现有功能;
        - **预防性维护**：优化代码结构，提高可维护性，预防未来问题。

- **(7) 退役阶段**

    - **任务**:
        - 评估软件是否已无法满足业务需求，决定是否停止使用;
        - 迁移数据或逐步替换为新系统，完成软件生命周期。

考点关联：对应考试大纲中 “软件过程” 部分，要求掌握软件生命周期的阶段划分及各阶段核心任务。

淘宝店：RendaCode辉文档源码库，原创

2. 什么是软件设计模式？请列举两种常见的设计模式并简要说明其用途。

解析：

软件设计模式是在软件开发过程中，针对反复出现的问题总结归纳出的通用解决方案。它是经过实践检验、被广泛认可的设计经验，可提高软件的可维护性、可扩展性和可复用性。

- **单例模式**：确保一个类只有一个实例，并提供全局访问点。用途在于当系统中某个资源只需要一个实例，如数据库连接池、日志记录器等场景时使用。避免创建多个实例造成资源浪费或不一致问题，保证资源的统一管理和使用。
- **工厂模式**：定义一个创建对象的接口（工厂类），由工厂类负责创建对象，而不是在客户端直接实例化对象。用途是将对象的创建和使用分离，当创建对象的逻辑较为复杂，或者需要根据不同条件创建不同类型对象时，使用工厂模式可以使代码更具可维护性和可扩展性 ，降低模块间的耦合度。

考点关联：对应考试大纲中“软件设计”部分，要求掌握软件设计模式的基本概念，以及常见设计模式的原理和应用场景。

3. 简述软件测试的原则。

解析：

软件测试需遵循以下原则：

- **尽早测试**：测试应尽早介入软件生命周期，从需求分析阶段就开始规划测试工作。尽早发现问题可降低修复成本，避免后期大量返工。
- **全面测试**：要对软件的功能、性能、可靠性、兼容性等多方面进行测试。覆盖各种输入情况、边界条件、异常场景等，确保软件质量。
- **重点测试**：在全面测试基础上，针对软件的关键功能、复杂模块、高风险区域重点测试。因资源有限，集中精力保障核心部分质量。
- **独立测试**：测试工作最好由独立的测试团队或人员承担，避免开发人员自己测试自己的代码，保证测试的客观性和公正性。
- **及时反馈**：测试过程中发现的问题要及时记录并反馈给开发团队，以便开发人员尽快定位和修复。同时，测试结果也应及时与相关人员沟通。
- **回归测试**：修复缺陷后，需进行回归测试，确保修改没有引入新的问题，原有功能依然正常工作。
- **测试计划**：制定详细的测试计划，包括测试目标、范围、方法、进度、资源等，使测试工作有序进行。
- **测试记录**：完整记录测试过程和结果，便于追溯问题、评估软件质量和积累测试经验。

考点关联：对应考试大纲中“软件测试”部分，要求掌握软件测试的基本原则，理解其对保证软件质量的重要性。

4．给出模块耦合的定义，并举例阐明 3 种模块耦合。

答：考察软件设计

模块耦合是对不同模块之间互相依赖程度的度量。(定义 1 分)

一般地，模块耦合可分为：

(1) 内容耦合：当一种模块直接修改或操作另一种模块的数据时，模块间的这种耦合称为内容耦合。

(2) 公共耦合：当两个以上的模块共同引用一种全局数据项时，模块间的这种耦合称为公共

淘宝店：Pan.code 软文档源码库，原创

耦合。
(3)控制耦合：一种模块在界面上传递一种信号控制另一种模块，接受信号的模块的动作根据信号值进行调整，模块间的这种耦合称为控制耦合。
(4)标识耦合：当两个模块至少有一种通过界面传递的公共参数时，模块间的这种耦合称为标识耦合。
(5)数据耦合:当模块间通过参数传递基本类型的数据时,模块间的这种耦合称为数据耦合。
（给出任意 3 种模块耦合，得 3 分，其中每种耦合 1 分）

一、模块耦合的定义

模块耦合是指软件系统中不同模块之间相互依赖、相互关联的程度，它反映了模块的独立性——耦合程度越低，模块独立性越强，软件的可维护性、可扩展性和可重用性越好；反之则模块间依赖紧密，修改一个模块易影响其他模块。

二、3 种模块耦合及举例

(1)数据耦合（Data Coupling）

- **定义：**两个模块通过参数传递简单数据（如基本数据类型、结构体、对象属性等）进行交互，模块间仅依赖传递的数据，不访问彼此的内部数据或逻辑。
- **举例：**在学生成绩管理系统中，“计算总成绩模块”接收“成绩录入模块”传递的“学号”和“各科目成绩”（均为简单数据），仅根据这些数据计算总成绩并返回结果，不访问“成绩录入模块”的内部变量（如临时存储的未提交成绩列表）。

(2)控制耦合（Control Coupling）

- **定义：**两个模块通过参数传递控制信息（如标志位、状态码、开关值等），一个模块通过控制信息直接影响另一个模块的执行逻辑（如决定另一个模块执行哪段分支代码）。
- **举例：**在文件处理系统中，“文件操作模块”接收“用户交互模块”传递的控制参数（如“1”表示“读取文件”、“2”表示“写入文件”），并根据该控制参数选择执行对应的读取或写入逻辑。此时“文件操作模块”的执行流程依赖“用户交互模块”传递的控制信息，属于控制耦合。

(3)内容耦合（Content Coupling）

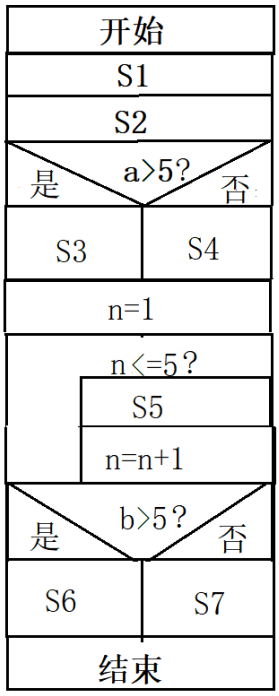
- **定义：**两个模块间存在最紧密的依赖关系，一个模块直接访问、修改或依赖另一个模块的内部数据、代码或实现细节，严重破坏模块独立性，是最差的耦合类型。
- **举例：**在库存管理系统中，“库存修改模块”直接访问并修改“库存存储模块”的内部数组（如直接修改`stockArray[0]`的值以更新某商品库存），而非通过“库存存储模块”提供的公开接口（如`updateStock(id, count)`方法）进行操作。此时两个模块存在内容耦合，若“库存存储模块”后续修改内部数组结构，“库存修改模块”会直接失效。

5．将下面的伪码转换成 N-S 图和 PAD 图。

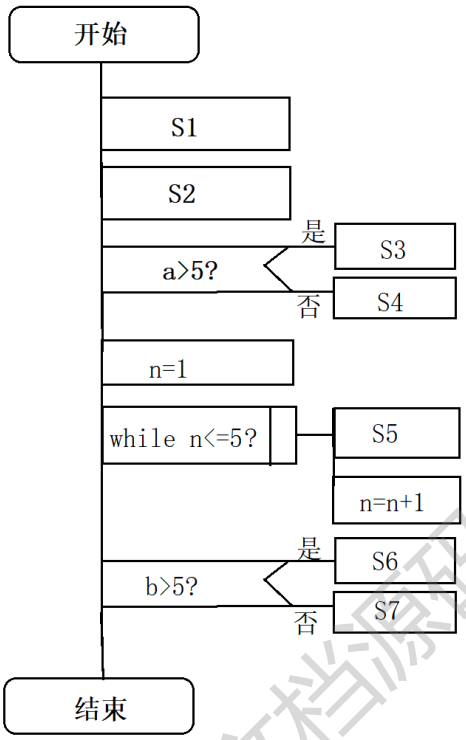
```
BEGIN
S1;
S2;
IF  a>5  THEN  S3  ELSE  S4;
n=1;
WHILE  n<=5  DO
{   S5;
n=n+1;  }
IF  b>5  THEN  S6  ELSE  S7;
END.
```

淘宝店：Pan-Good资料源码库 原创

答：考察软件设计



N-S图



PAD图

6．给出 CMM 的 5 级模型，并以其中一种 CMM 等级为例，列出该等级包括的关键过程域。

答：考察软件过程

CMM（能力成熟度等级）的 5 级模型如下所示：(2 分)

初始级(1 级)

可重复级(2 级)

已定义级(3 级)

已管理级(4 级)

持续优化级(5 级)

例如：

（1）初始级(1 级)，关键过程域包括：无。

（2）可重复级(2 级)，关键过程域包括：需求管理、软件项目规划、软件项目跟踪和监督、软件转包管理、软件质量控制、软件配置管理。

（3）已定义级(3 级)，关键过程域包括：组织过程焦点、组织过程定义、培训方案、集成软件管理、软件生产工程、组间协调、互相评审。

（4）已管理级(4 级)，关键过程域包括：软件质量管理、定量过程管理。

（5）持续优化级(5 级)，关键过程域包括：过程变更管理、技术变更管理、故障预防。

（至少答出 2 个关键过程域，得 2 分）。

CMM（能力成熟度等级）表

能力成熟度等级	关键过程域（KPA）	核心特征

淘宝店：PandaCode课文档源码库，原创

初始级(1 级)	无明确的关键过程域	过程无序，依赖个人能力，项目成功依赖个人经验，无稳定的过程规范，质量和进度不可预测。
可重复级(2 级)	1. 需求管理 2. 软件项目计划 3. 软件项目跟踪与监控 4. 软件子合同管理 5. 软件质量保证 6. 软件配置管理	建立基本项目管理过程，能重复过往成功项目的经验（如跟踪成本、进度、质量），但过程未标准化，仅能保证项目级的稳定。
已定义级(3 级)	1. 组织过程焦点 2. 组织过程定义 3. 培训大纲 4. 集成软件管理 5. 软件产品工程 6. 组间协调 7. 同行评审	软件过程被标准化、文档化，形成组织级标准过程，所有项目基于标准过程裁剪执行，过程可预测，强调组织级过程统一。
已管理级(4 级)	1. 定量过程管理 2. 软件质量管理	对软件过程和产品质量进行量化管理（如用统计方法监控过程稳定性、产品缺陷率），过程可精确控制，质量和过程性能有量化指标。
持续优化级(5 级)	1. 缺陷预防 2. 技术变更管理 3. 过程变更管理	持续改进软件过程，通过分析过程数据、引入新技术等主动优化过程，提升组织整体能

淘宝店：PandaCode辉文档源码库原创

		力，强调创新和过程的持续提升。

表4-8　能力成熟度等级的基本概念

能力成熟度等级	关键过程域	基本特征
初始级（1级）	—	以个人主观认为合适的方式来开发
可重复级（2级）	需求管理 软件项目规划 软件项目跟踪和监督 软件转包管理 软件质量控制 软件配置管理	不存在组织范围的过程，利用已存在的项目管理实践和经验来开发
已定义级（3级）	组织过程焦点 组织过程定义 培训方案 集成软件管理 软件生产工程 组间协调 互相评审	有定义好的属于组织范围的过程，按照过程有效的进行开发
已管理级（4级）	软件质量管理 定量过程管理	可以对过程能力进行量化，根据量化来预测和控制一个项目的过程绩效
优化级（5级）	过程变更管理 技术变更管理 故障预防	过程能力可控、可改进，并且可以以量化方式来评估改进

7. 以 G.J.Myers 的观点，简述软件测试的目的。

解析：考察软件测试

一、G.J.Myers 关于软件测试目的的核心观点

G.J.Myers 在《软件测试艺术》中提出了与传统“找错”认知不同的测试目的，核心可概括为：**软件测试的目的是“以最少的代价，在最短的时间内，找出软件中尽可能多的错误”，同时验证软件是否满足用户需求**，而非单纯证明软件无错。其具体观点可拆解为 4 点：

二、软件测试目的的具体阐述

- **(1)首要目的：发现错误**

  Myers 认为，测试的核心价值在于主动暴露软件中的缺陷（如功能异常、逻辑漏洞、边界错误），而非验证软件“能正常工作”。只有发现错误，才能通过修复提升软件质量，避免错误流入用户手中导致更大损失。

- **(2)次要目的：验证需求满足度**

  测试需对照需求规格说明书，验证软件功能、性能、安全性等是否符合用户预期。例如，通过测试确认“用户登录模块”是否满足“支持手机号 / 邮箱两种登录方式”的需求，确保软件方向与用户需求一致。

- **(3)辅助目的：评估软件质量**

  测试过程中收集的错误数据（如错误类型、出现频率、修复成本）可用于评估软件质量

淘宝店铺：PandaCode资料文档源码库，原创

水平。例如，若测试中发现“支付模块”存在高频率转账计算错误，可判断该模块质量风险较高，需重点优化。

- **(4)最终目的：降低风险与代价**

  测试需在软件发布前发现并修复错误，避免错误在生产环境中爆发（如电商平台支付漏洞导致用户财产损失）。通过“早期测试”（如单元测试、集成测试）减少后期修复成本，实现“以最少代价保障质量”的目标。

表4-12　软件测试的基本概念

软件测试	基本概念
定义	测试是为了发现程序中的错误而执行程序的过程，其不能证明程序中没有错误
目标	①好的测试方案是极可能发现迄今为止尚未发现的错误的测试方案； ②成功的测试是发现了至今为止尚未发现的错误的测试； ③根本目标是尽可能多地发现并排除软件中潜藏的错误，最终把一个高质量的软件系统交给用户使用； ④测试的目标是发现错误，调试的任务是在测试的基础上去改正错误
测试方法	①白盒测试（结构测试）：把程序看成装在一个透明的白盒子里，测试者完全知道程序的结构和处理算法，按照程序内部的逻辑测试程序，检测程序中的主要执行通路是否都能按预定要求正确工作； ②黑盒测试（功能测试）：把程序看作一个黑盒子，完全不考虑程序的内部结构和处理过程，是在程序接口进行的测试，只检查程序功能是否能按照规格说明书的规定正常使用

8. 消息传递是面向对象方法中对象之间进行交互的机制，消息传递机制与传统程序设计模式中的过程调用有什么本质区别?

解析：考察软件设计-面向对象方法

消息传递机制与传统过程调用的本质区别

对比维度	面向对象的消息传递机制	传统程序设计的过程调用
调用主体与接收主体	主体是**对象**：消息由一个对象发送给另一个对象（接收对象），请求其执行特定方法（如“对象 A 向对象 B 发送‘计算’消息”）。	主体是**过程 / 函数**：调用者直接指定过程名（如 sum(a,b)），接收者是独立的过程，与调用者无“对象”身份关联。
依赖关系	发送者仅需知道接收对象的**接口（可接收的消息类型）**，无需了解其内部实现（如方法如何编码），体现“封装性”。	调用者需知道过程的**具体实现细节**（如参数类型、返回值、内部逻辑），否则无法正确调用，依赖紧密。
灵活性与动态性	接收对象可动态决定如何响应消息：同一消息被不同对象接收时，可执行不同实现（多态性），如“‘绘图’消息对‘圆形’和‘矩形’对象有不同处理”。	过程调用是静态绑定：调用哪个过程由调用语句直接指定，执行逻辑固定，无动态适配能力。
参数传递的侧重点	消息包含**操作名称和参数**，核心是“请求对象执行操作”，参数是操作的输入，不直接暴露接收对象的内部数据。	过程调用传递的是**数据参数**，核心是“对数据进行处理”，过程可能直接操作传入的全局数据或共享变量。
本质目标	实现对象间的**协作**：通过消息驱	实现**功能复用**：将重复逻辑封

淘宝店：Panda Code辉文档原创

	动多个对象分工完成复杂功能，体现“面向对象的交互逻辑”。	装为过程，通过调用减少代码冗余，体现“功能的模块化”。

两者的核心区别在于：

- 消息传递是**对象级的交互**，依赖接口而非实现，支持动态多态，体现面向对象的封装性和灵活性；
- 过程调用是**功能级的复用**，依赖具体实现，静态绑定，更侧重代码的模块化而非实体间的协作。

这种差异使得消息传递更适合描述复杂系统中实体（对象）的动态行为和关系，而过程调用更适合简单功能的抽象与复用。

9. 说明面向对象分析阶段所建立的对象模型、动态模型和功能模型之间的关系。

解析：考察软件设计-面向对象方法

一、三大模型的核心定位（基础区别）

首先明确各模型的核心职责，这是理解其关系的前提：

模型类型	核心定位	描述视角	关键产出物
对象模型	描述系统的**静态结构**	“是什么”（实体及关系）	类图（类、属性、方法、关联）
动态模型	描述系统的**动态行为**	“怎么做”(行为时序 / 状态)	状态图、序列图
功能模型	描述系统的**功能目标**	“做什么”（功能及数据流向）	用例图、数据流图

二、三大模型的核心关联逻辑

面向对象分析的对象模型、动态模型和功能模型，从静态、动态、功能三个维度共同描述系统，三者相互依赖、互补，构成完整分析模型，具体关系如下：

- **对象模型是基础**
  对象模型定义系统的静态结构（类、属性、关联），是另外两个模型的载体。动态模型描述的“行为主体”(如对象状态变化、消息传递)，以及功能模型描述的“数据载体”（如功能输入输出的属性），均需基于对象模型中的类或对象，无对象模型则动态与功能模型无实体依托。
- **动态模型是行为桥梁**
  动态模型通过状态图、序列图描述对象的动态交互（如“用户支付订单”的消息传递时序），将对象模型的“静态结构”转化为可执行的“动态行为”，同时落地功能模型的“抽象功能”——例如功能模型的“支付”功能，需通过动态模型明确“验证余额→扣减金额→更新订单状态”的行为步骤，否则功能仅为抽象目标。
- **功能模型是需求导向**
  功能模型定义系统需实现的功能目标（如用例图中的“订单管理”“支付”），决定对象模型的设计范围（如需“支付”功能则设计“订单”“支付账户”类），同时引导动态模型的行为重点（优先细化核心功能的交互逻辑），确保对象与动态模型围绕用户需求构建，避免偏离目标。

10. 请阐述 Web 应用软件的基本概念，并列举至少两个不同类型的经典示例。

解析：考察软件工程概述

(1)基本概念：

Web 应用软件(Web Application)是指基于浏览器/服务器(B/S)架构、通过 HTTP/HTTPS

淘宝店：Pandaoo软件文档资料库，原创

协议在互联网或内网中运行的软件系统。用户无需安装客户端程序，只需使用 Web 浏览器即可访问和使用其功能。Web 应用通常由前端（HTML/CSS/JavaScript 等）与后端（如 Java、Python、Node.js 等）组成，数据存储于服务器端数据库中，具有跨平台、易维护、集中部署等特点。

(2)经典示例：

- **电子商务类**：这类平台集成了商品展示、用户账户管理、购物车、在线支付、订单处理等一系列复杂的业务流程；
- **信息资讯类：**这类网站核心功能是向用户展示新闻、政策解读、热点资讯等内容。

11. RAD 有哪 3 种建模，每个建模的主要活动是？

解析：考察软件过程

RAD（快速应用程序开发）是一个线性顺序软件开发过程模型，该模型使用基于元素的构造方法来强调简洁的开发周期。如果对需求有很好的理解和描述，并且项目范围是一个约束，则 RAD 流程可使开发团队在简明的时间内创建功能全面的系统。

三种建模及主要活动分别是：

- **(1)业务建模：**通过回答以下问题来定义业务职能之间的信息流：诸如哪些数据驱动了业务流程，生成了哪些数据，信息去向何处，由谁处理等等。
- **(2)数据建模：**将从业务建模中收集的数据完善为支持业务所需的一组数据对象(实体)。标识属性(每个实体的字符)，并定义这些数据对象(实体)之间的关系。
- **(3)流程建模：**转换数据建模阶段中定义的信息对象，以实现业务功能所需的数据流。创建处理说明以添加，修改，删除或检索数据对象。

12. 假设有一个求三个整数 a、b、c 最小值的程序，其程序流程图逻辑如下：

```
S1: 开始
S2: 输入 a, b, c
S3: min = a
S4: if (b < min)
S5:     min = b   // 此语句可能不被执行
S6: if (c < min)
S7:     min = c   // 此语句可能不被执行
S8: 输出 min
S9: 结束
```

请设计最少的测试用例，确保实现该程序的语句覆盖（要求列出每个测试用例的输入值、预期输出，并说明覆盖的关键语句）。

解析：考察软件测试

只需 1 个测试用例：

输入：a = 3，b = 2，c = 1 ；

预期输出：1 ；

覆盖情况：

min = a ，覆盖；

if (b < min) min = b 的判断与赋值，覆盖 ；

if (c < min) min = c 的判断与赋值，覆盖 ；

输出语句，覆盖 ；

满足语句覆盖准则，且用例数量最少。

淘宝店：PandaCode原文档源码库，原创

13. 什么是开源软件？试举一简单说明。

解析：考察软件工程概述

(1)开源软件：指源代码可以被公众自由获取、查看、修改、分发的软件，其核心特征是遵循开源许可协议(如 GPL、MIT 协议等)，允许用户在协议规定范围内对软件进行二次开发、传播，且通常不收取核心功能的使用费用，同时强调社区协作开发和透明化的代码管理。

(2)举例：Linux 操作系统，其源代码完全开放，全球开发者可参与代码修改与优化，用户可自由下载、安装并根据需求定制（如用于服务器、嵌入式设备等场景），且遵循 GPL 开源协议，确保修改后的衍生版本也需保持开源特性。

14. RUP 的每一个迭代周期中设计 6 个核心工程工作流，3 个核心支持工作流。试简述每个核心工程工作流的名称和主要目的。

解析：考察软件过程

RUP(统一软件开发过程)的每个迭代周期中，6 个核心工程工作流及其主要目的如下：

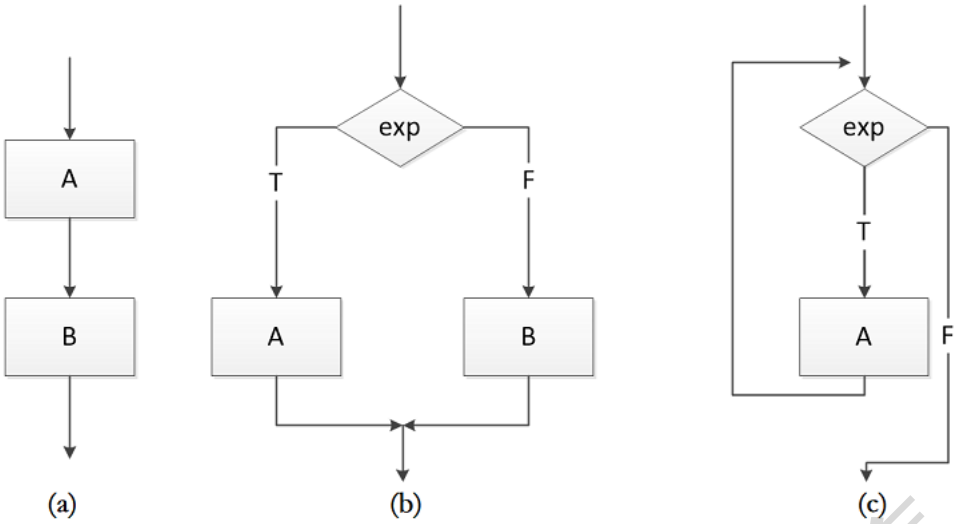
- **1. 业务建模：**通过建立业务用例模型，理解组织的业务流程、目标和组织结构，明确系统需要解决的业务问题，为后续开发提供业务上下文。
- **2. 需求：**收集、分析和文档化用户需求，建立用例模型和补充需求规格，明确系统必须实现的功能和非功能需求，作为设计和测试的依据。
- **3. 分析与设计：**将需求转化为系统的设计模型，包括架构设计、类设计、接口设计等，优化系统结构以满足需求，同时平衡性能、可扩展性等质量属性。
- **4. 实现：**根据设计模型编写代码，将设计转化为可执行的软件组件，进行单元测试和集成测试，确保代码符合设计规范。
- **5. 测试：**验证软件是否满足需求，包括单元测试、集成测试、系统测试和验收测试，识别并修复缺陷，确保软件质量。
- **6. 部署：**将经过测试的软件系统交付给用户，包括安装、配置、用户培训和数据迁移等，确保系统在实际环境中正常运行。

15. 结构化程序设计方法要求再复杂的程序结构也应使用基本控制结构组合嵌套来实现。试问这些基本控制结构有哪些？并试用类 C 语言或程序流程图简单描述。

解析：考察软件设计-结构化设计

答：结构化程序设计的基础控制结构有顺序结构、选择结构、循环结构，每个只有一个入口和出口。如下程序流程图描述。

淘宝店：Pass Code软考资料源码库 原创



(a)顺序结构，先执行A再执行B；(b)IF-THEN-ELSE 型选择(分支)结构；
(c)DO-WHILE 型循环结构

16. 软件需求是什么？共分为几个层次？

解析：考察软件需求

软件需求是“要予构造”的陈述，描述了待开发软件产品在功能、性能、属性等方面的要求，是软件开发的核心依据。

软件需求分为 2 个核心层次：功能需求和非功能需求；其中非功能需求进一步细分为性能需求、外部接口需求、质量属性、设计约束。

17．软件质量保证的是什么？它的四个活动是什么？

解析：考察软件质量

质量保证是为项目生存周期内的软件过程和软件产品提供适当保障的过程，目的是使它们符合所规定的需求，并遵循已建立计划。

包括四个活动：过程实现，产品保证，过程保证，质量体系保证四个活动。

18．说明客户端/服务器，对等模式采用的三层结构是什么？

解析：考察软件设计

客户端/服务器（C/S）模式和对等（P2P）模式中采用的三层结构均为：**表示层、业务逻辑层、数据层**。

三层结构是软件体系架构中经典的分层模式，通过职责分离提高系统的可维护性、可扩展性，在 C/S 和 P2P 模式中均有应用：

- **表示层：**
  负责用户交互与数据展示，是用户与系统的接口（如客户端界面、APP 页面）。其核心职责是接收用户输入（如点击、表单提交）并展示处理结果，不涉及业务逻辑或数据处理。
- **业务逻辑层：**
  位于表示层与数据层之间，是系统的核心“大脑”，负责实现业务规则、处理业务流程（如用户登录验证、订单状态更新）。它接收表示层的请求，调用数据层获取/处理数据，再将结果返回给表示层，实现了业务逻辑与界面、数据的解耦。

淘宝店 PanCode软文档源码库，原创

- **数据层：**
  负责数据的存储、管理与访问（如数据库操作、文件读写），提供数据查询、插入、更新、删除等服务。其职责是屏蔽底层数据存储细节（如使用 MySQL 还是 Oracle），为业务逻辑层提供统一的数据接口。

19. 在承包软件项目之前为什么要做可行性研究？软件项目的可行性研究主要研究哪几个方面的可行性?

解析：考察软件需求

(1) 可行性研究是项目启动前的核心评估环节，核心目的是**避免盲目投入资源（人力、时间、成本），提前识别项目风险，判断项目是否具备实施价值与成功可能性**。具体原因包括两点：

- **降低决策风险：**通过评估可提前发现项目的致命问题(如技术无法实现、成本远超预算、无市场需求)，避免项目中途失败导致的资源浪费；
- **明确实施基础：**可行性研究可梳理项目的核心约束(如技术瓶颈、资金上限、时间节点)，为后续项目计划制定（如资源分配、进度安排）提供依据，确保项目从启动阶段就具备清晰的目标与路径。

(2) 软件项目可行性研究主要围绕技术、经济、操作、社会四个关键层面：

- **技术可行性：**评估现有技术能否实现项目目标，包括 “是否有成熟技术方案”“团队是否具备所需技术能力”“硬件 / 软件环境是否满足需求”（如开发 AI 推荐系统需评估团队是否掌握机器学习技术）；
- **经济可行性：**分析项目的成本与收益，判断是否具备经济价值，包括 “计算开发总成本（人力、设备、维护）”“预估项目收益（直接盈利、成本节省）”“对比投入与产出，判断投资回报率（ROI）”；
- **操作可行性：**评估项目交付后能否被用户顺利使用，包括 “用户是否具备操作系统的技能”“系统是否符合用户现有工作流程”“使用环境（如网络、设备）是否易于部署”；
- **社会可行性：**判断项目是否符合法律法规、行业规范及社会伦理，包括 “是否遵守数据隐私法规（如个人信息保护法）”“是否符合行业标准”“是否存在伦理风险（如算法歧视）”。

20. 事件驱动风格的体系结构在软件体系结构分类中属于控制模型，它是通过外部生成的事件来驱动系统。典型的事件驱动风格的体系结构有哪两种类型？简述它们的控制机制。

解析：考察软件设计

典型的事件驱动风格的体系结构主要有以下两种类型：

- **事件发布/订阅模型（Publish/Subscribe Model）：**
  - **控制机制：**系统中的组件（发布者）在发生特定事件时，将事件发布到一个事件通道（如事件总线或消息中间件）；其他组件（订阅者）事先声明自己感兴趣的事件类型（即订阅）。当事件被发布后，事件分发机制会自动将事件传递给所有相关订阅者。控制流由事件的产生和分发隐式驱动，发布者与订阅者之间解耦，彼此无需知道对方的存在。
- **显式事件处理模型（Explicit Event Handling Model，也称回调模型）：**
  - **控制机制：**组件通过注册回调函数（或事件处理器）来响应特定事件。当事件发生时（如用户点击按钮、系统状态变化等），事件调度器（如事件循环）会显式调用已注册的对应处理函数。控制流由事件队列和调度器管理，事件按顺序被取出并分发给相应的处理器执行。

淘宝店：ChinaCode软考资源题库 原创

21. 软件生存周期中可能执行的活动可分为 5 个基本过程，这 5 个基本过程是什么？每一个基本过程与软件项目的哪一方相关?

解析：考察软件过程

5 个基本过程:

(1) 获取过程：为需求方而定义的活动，启动，招标，合同，对供方监督，验收等。相关方：**获取方（需求方，如客户、用户组织）。**

(2) 供应过程：为供应方而定义的活动，启动，准备投标，签订合同，编制计划，执行，交付和完成。相关方：**供应方（提供方，如软件开发公司）。**

(3) 开发过程：为开发方而定义的活动：需求、设计、编码、测试、安装、验收。相关方：**供应方（内部开发团队）。**

(4) 运作过程：为操作方而定义的活动：运行测试，系统运行，用户支持。相关方：**运行方（获取方内部运维 / 使用团队）。**

(5) 维护过程：为维护方而定义的活动：问题和修改分析，修改实现，维护评审/验收，迁移，软件退役。相关方：**维护方（供应方、获取方内部团队或第三方机构）。**

22. 什么是面向对象系统中的消息？一个消息应包括哪几部分？

解析：考察软件设计-面向对象方法

(1) 面向对象系统中的消息是对象之间的通信机制。

在面向对象的系统中，各个对象实例通过消息来进行交互和协作。一个对象通过发送消息来请求另一个对象执行特定的操作或方法。这种通信方式允许对象保持独立性，同时能够灵活地相互配合完成任务。消息的概念不仅包括了调用的方法，还涵盖了多态性，即同一个消息可以根据接收对象的类型产生不同的行为。

(2) 一个消息通常包括三个部分：接收消息的对象、接收对象要采取的方法以及方法所需的参数。

具体说明如下:

- 接收消息的对象：这是消息的接收者，它负责处理消息并作出响应。
- 接收对象要采取的方法：消息会指定一个方法名，告诉接收对象需要执行的具体操作。
- 方法所需的参数：这些是传递给方法的值，用于方法的操作过程中。

总的来说，在设计面向对象的系统时，理解消息的概念和结构对于实现对象间高效的交互至关重要。通过定义清晰的接口和传递恰当的参数，对象能够在保持封装性的同时互相合作，共同完成复杂的任务。

23. 什么是基本路径覆盖测试？它应满足哪一种测试覆盖准则？

解析：考察软件测试

(1) 基本路径覆盖测试是一种白盒测试方法，它依据程序的控制流图来设计测试用例。

基本路径覆盖测试的目的是通过分析程序控制结构的环路复杂性，导出一个基本可执行路径集合。这个集合包含了程序中所有可执行的路径，且在设计测试用例时，要确保程序中的每一个可执行语句至少被执行一次。

这种方法通常涉及以下步骤：首先构建程序的控制流图，然后确定该图的环路复杂度，接着导出独立路径的基本集，并据此设计测试用例，最后准备测试用例以确保基本路径集中的每一条路径都能被执行。

(2) 这种测试方法应满足的测试覆盖准则主要是基本路径覆盖。

测试覆盖准则要求在测试过程中程序的每个可执行语句至少要被执行一次，以及每个判定条件的每个可能结果都至少被覆盖一次。此外，基本路径覆盖法还与圈复杂度相关，圈复

淘宝搜：PassCode源文件源码库，原创

杂度衡量了程序模块判定结构的复杂程度，并且可以作为确定测试用例数量的一个参考标准。

总的来说，基本路径覆盖测试旨在通过对程序控制结构进行详尽的分析，确保设计的测试用例能够覆盖程序中的所有基本路径，从而达到较高的测试覆盖率，帮助发现程序中可能存在的错误。

24.什么是 CMMI？在 CMMI 连续式表示中把能力等级划分为哪几个等级？

解析：考察软件过程

(1) CMMI(Capability Maturity Model Integration)，即能力成熟度模型集成。它是一个用于软件过程改进的模型，旨在帮助软件企业提高开发和改进能力，降低成本和质量。

CMMI 不是一个标准或流程，而是提供了一套框架，集成了多学科的最佳实践，以促进组织的过程改进。它覆盖了项目管理、工程、组织流程管理和支持等方面，每个方面都包含若干个“过程域”，这些过程域提供了具体实施的指南和实践。

(2) 在连续式表示法中，CMMI 的能力等级被细分为六个级别，从 0 到 5 级，分别是：不完整级（0 级）、已执行级（1 级）、已管理级（2 级）、已定义级（3 级）、量化管理级（4 级）和最优化级（5 级）。这种连续式的评估方法允许对每个过程域进行单独评定，从而更细致地识别组织的强项和弱点。连续式表示法强调的是组织在实施和控制其过程以及改善过程绩效方面所具备的能力，而不是像阶段式表示法那样将成熟度分为几个固定的阶段。

总之，了解 CMMI 及其能力等级对于希望提升软件过程能力和质量管理水平是至关重要的。这不仅有助于企业在国际竞争中提高自身形象，还能够通过高效的流程管理和质量保障，为客户提供更加优质的产品和服务。

CMM　　表4-8　能力成熟度等级的基本概念

能力成熟度等级	关键过程域	基本特征
初始级（1 级）	—	以个人主观认为合适的方式来开发
可重复级（2 级）	需求管理 软件项目规划 软件项目跟踪和监督 软件转包管理 软件质量控制 软件配置管理	不存在组织范围的过程，利用已存在的项目管理实践和经验来开发
已定义级（3 级）	组织过程焦点 组织过程定义 培训方案 集成软件管理 软件生产工程 组间协调 互相评审	有定义好的属于组织范围的过程，按照过程有效的进行开发
已管理级（4 级）	软件质量管理 定量过程管理	可以对过程能力进行量化，根据量化来预测和控制一个项目的过程绩效
优化级（5 级）	过程变更管理 技术变更管理 故障预防	过程能力可控、可改进，并且可以以量化方式来评估改进

淘宝店：Pandacode程序员资源店　原创

25. 验证和确认是软件质量审查的两个重要手段，简述它们的区别和联系。

解析：考察软件质量

(1) 核心区别（定义与核心目标）

维度	验证（Verification）	确认（Validation）
核心定义	检查软件“是否正确地做了这件事”（Did we build the product right?）	检查软件“是否做了正确的事”（Did we build the right product?）
核心目标	确保软件开发过程符合规范（如需求文档、设计文档），产品符合设计要求	确保软件产品满足用户实际需求和预期用途
关注对象	开发过程的规范性、产品与文档的一致性	产品的实用性、用户需求的满足度
实施阶段	贯穿软件开发全生命周期（需求分析、设计、编码、测试等阶段）	主要在测试阶段后期（如系统测试、验收测试）及产品交付前
常用方法	评审（需求评审、设计评审）、代码走查、单元测试、集成测试	系统测试、验收测试、用户试用、原型验证

(2) 联系

- 二者均是软件质量保证（SQA）的核心手段，共同目标是确保软件产品质量，减少缺陷；
- 验证是确认的基础：只有先通过验证确保开发过程规范、产品符合设计，才能通过确认进一步验证产品是否满足用户需求；若验证不通过（如产品不符合设计），确认将失去意义；
- 二者相辅相成，缺一不可：仅验证不确认，可能导致“产品符合设计但不符合用户需求”；仅确认不验证，可能因开发过程不规范导致产品质量不稳定，难以持续满足用户需求。

26. 说明什么是 UML 状态图？它的作用是什么？

解析：考察软件设计

(1) UML 状态图的定义：

UML（统一建模语言）状态图是面向对象建模中用于描述 单个对象（或类实例）在其生命周期内的状态变化过程 的动态建模图。它以“状态”为核心，通过状态转移、事件触发、动作执行等元素，直观展现对象从创建到消亡（或进入终止状态）的行为模式，聚焦对象在不同条件下的状态切换逻辑。

(2) UML 状态图的作用：

- 刻画对象的动态行为：清晰展示对象在不同条件下如何响应事件并改变状态；
- 识别异常或遗漏状态：帮助发现设计中未考虑的状态或非法转换；
- 支持系统测试设计：为状态覆盖、转移覆盖等测试策略提供依据；
- 促进开发与业务沟通：用直观图形表达业务规则（如审批流程、工作流）；
- 指导代码实现：可直接映射到状态模式（State Pattern）等编程结构。

27. 说明什么是条件组合覆盖准则，说明对于一个判断中的复合条件(A≥9) and (B<0)，为达到 100%的条件组合覆盖率，至少需要多少测试用例。

解析：考察软件测试

条件组合覆盖准则（Condition Combination Coverage，简称 CC）是一种测试覆盖标准，要求设计足够的测试用例，使得每个判断中的所有可能的条件组合至少出现一次。换句话说，对于一个具有多个条件的布尔表达式，必须确保每个条件的所有取值（真或假）的所有组合都被测试到。

淘宝店 PanaCode编程文档源码库 原创

测试用例设计：

对于给定的复合条件：(A≥9) and (B<0)。

（1）识别简单条件：该判定包含两个简单条件：

- 条件 C1: (A≥9)。
- 条件 C2: (B<0)。

（2）列出所有条件组合：每个条件有真(T)、假(F)两种取值，因此共有 $2^2 = 4$ 种可能的组合。

组合编号	条件 C1 (A≥9)	条件 C2 (B<0)	整个判定 (C1 and C2) 的结果
1	T	T	T
2	T	F	F
3	F	T	F
4	F	F	F

（3）确定最少测试用例数：

- 为了达到 100%的条件组合覆盖率，必须使上述 4 种条件组合每种都至少出现一次。
- 一个测试用例在程序执行到该判定点时，会为 C1 和 C2 产生一组特定的取值，即覆盖上述表格中的一行。
- 由于这 4 种组合互不相同，且一个用例只能覆盖其中一种组合，因此，至少需要 4 个测试用例才能覆盖全部 4 种组合。

结论：为达到 100%的条件组合覆盖率，至少需要 4 个测试用例。

28. 给出模块耦合的定义，并举例说明 3 种模块耦合。

解析：考察软件设计

（1）模块耦合的定义

模块耦合（Coupling）是指软件系统中不同模块之间相互依赖或关联的程度。耦合度越低，模块的独立性越强，系统的可理解性、可维护性和可复用性越高。因此，**低耦合是软件设计的重要原则之一**。

（2）三种模块耦合类型及示例（按耦合度从低到高排列）

①数据耦合

- 定义：模块之间通过参数表来传递基本类型的数据项。这是一种最理想、耦合度最低的形式。
- 举例：模块 A 调用模块 B，并传递一个整数参数 score。模块 B 根据 score 计算等级并返回。
  - `result = calculateGrade(score);`
- 特点：接口简单、清晰，依赖关系最小。

②标记耦合（特征耦合）

- 定义：模块之间通过参数传递一个完整的数据结构（如记录、对象），但接收模块只使用该结构中的一部分数据。
- 举例：模块 A 将一个 Student 对象（包含学号、姓名、各科成绩等）传递给模块 B，但模块 B 只使用其中的 studentId 来生成学籍报告。
  - `report = generateReport(student); // 函数内部只用了 student.id`
- 特点：耦合度高于数据耦合。如果数据结构发生变化，即使模块 B 不使用的部分发生改变，也可能导致调用代码（模块 A）需要修改。

③控制耦合

- 定义：一个模块向另一个模块传递了用于控制其内部逻辑的信号或标志。

淘宝店：PandaCode辉文档源码库 原创

- 举例：模块 A 调用模块 B 时，传递一个控制标志 mode。模块 B 根据 mode 是 "SAVE" 还是 "LOAD" 来决定是执行保存数据还是加载数据的操作。
  - fileOperation("data.txt", "SAVE");
- 特点：耦合度较高。调用模块必须了解被调用模块的内部逻辑，这使得模块 B 的功能不单一，且修改模块 B 的逻辑可能会影响到模块 A。

模块耦合的核心是 **“模块间依赖的紧密程度”，从低到高的典型耦合等级为：数据耦合 < 控制耦合 < 标记耦合 < 公共耦合 < 内容耦合**。软件设计的核心原则是 **“低耦合、高内聚”**，因此应优先采用数据耦合，避免使用公共耦合、内容耦合等强耦合类型，以提升系统的灵活性和可维护性。

29. 给出对象的关联关系定义，并举例说明。

解析：考察软件设计

（1）对象的关联关系定义

在面向对象方法学中，**关联关系（Association）**是指两个或多个类（或对象）之间存在的长期、稳定的语义联系，用于描述对象之间的“相互联系”或“参与关系”。关联关系是 UML 中最基础的关系类型，核心特征包括：

- **双向性：**默认情况下关联是双向的（如“学生”与“课程”相互关联），也可通过箭头指定单向关联；
- **多重性：**描述关联双方的对象实例数量对应关系（如 1 对 1、1 对多、多对多）；
- **无依赖约束：**关联的对象之间是独立存在的（区别于组合、聚合的“整体 - 部分”关系），仅通过语义关联产生交互。

关联关系的本质是“对象间的协作关系”，不涉及对象的创建或消亡依赖，仅明确对象在业务场景中的互动逻辑。

（2）对象的关联关系示例

- 一对一关联（1:1）：“学生”与“学生证”的关联。
- 一对多关联（1:N）：“班级”与“学生”的关联。
- 多对多关联（M:N）：“学生”与“课程”的关联。

30. 给出模块的高内聚、低耦合原则的具体含义。

解析：考察软件设计

“高内聚、低耦合”是软件模块化设计的核心原则，旨在提高软件的可理解性、可维护性、可复用性和可测试性。

1. 高内聚（High Cohesion）：

- 含义：指一个模块内部各个元素（如函数、语句、数据）彼此密切相关，共同完成单一、明确的功能。
- 目标：模块职责单一，功能集中。
- 优点：
  - 易于理解与修改；
  - 减少模块内部逻辑混乱；
  - 提高复用可能性。
- 示例：一个名为 UserAuthentication 的模块，只负责用户登录、密码验证和会话创建，不涉及日志记录或数据库备份，即为高内聚。
- **因此，内聚性越高，模块越“专注”。**

2. 低耦合（Low Coupling）：

淘宝店：PandaCode辉艾资源码库 原创

- 含义:指模块之间相互依赖的程度尽可能低,即一个模块对其他模块的了解和影响最小。
- 目标：模块接口简洁，交互仅通过明确定义的接口进行。
- 优点:
  - 修改一个模块不影响其他模块;
  - 便于独立开发与测试;
  - 增强系统稳定性。
- 实现方式：使用数据耦合（传递基本参数），避免全局变量、控制标志或直接访问内部数据。
- **因此，耦合度越低，模块越“独立”。**

五、建模题

1. 问题陈述：某销售管理系统具备如下核心功能：

- **用户：**用户可提交商品购买请求。请求提交后，等待系统对其请求进行校验，并接收最终的订单处理结果，结果包含订单信息或者缺货通知。
- **系统：**系统针对用户的购买请求执行校验流程。在校验时，需从“用户信息表”中获取用户类型（新用户或老用户）以及信誉状态等信息，同时依据“商品表”判断对应商品的库存情况。若校验通过，系统将订单信息写入“订单文件”。
- **供应商：**当系统判断商品缺货时，会向供应商发送缺货信息。供应商在补充商品库存后，对“商品表”中的库存数据进行更新。
- **销售经理：**销售经理能够查询“订单文件”，统计商品的销售情况，并据此生成销售报表。

(1). (3 分) 用结构化分析方法绘制该系统的顶层 DFD (系统环境图)。

(2). (3 分) 绘制该系统的 0 层 DFD (将顶层 DFD 中的系统分解为 “请求校验”“订单处理”2 个核心加工)。

(3). (4 分) 针对 “请求校验” 加工，绘制其 1 层 DFD (分解为 “用户信誉校验”“商品库存校验” 2 个细分加工)。

答: 考察软件设计-结构化方法

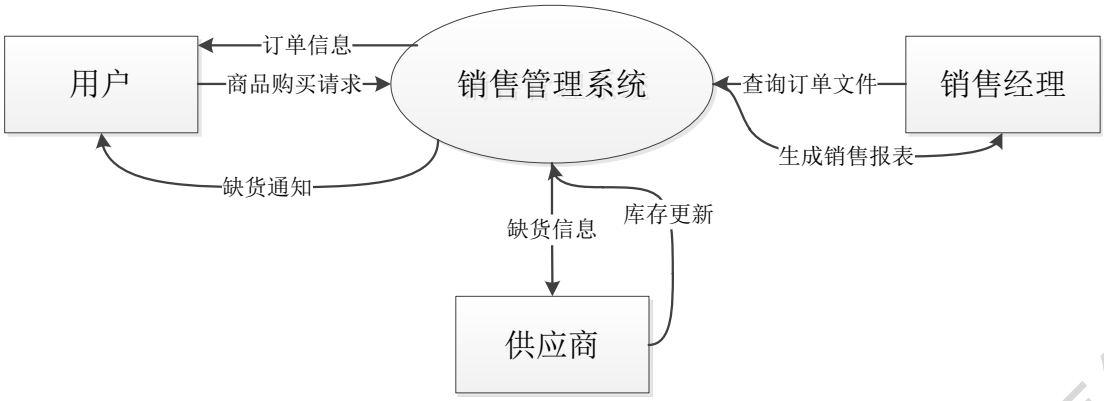
淘宝店：PandaCode程序档源码库，原创

(1). 该系统的顶层 DFD（系统环境图）。

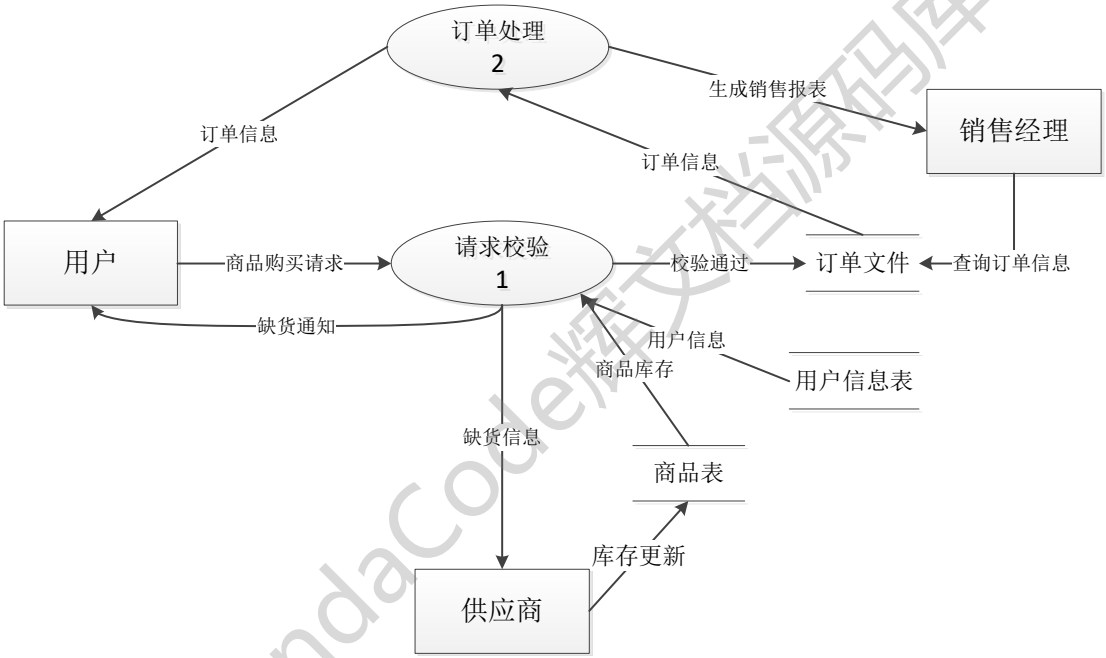
- **顶层 DFD（Data Flow Diagram，数据流图）** 是系统环境图，它展示了系统的外部实体、系统本身以及系统与外部实体之间的数据流。
- **外部实体**：与系统交互的外部参与者，包含三个主要外部实体：
  - **用户：**购买商品。
  - **销售经理：**查询订单。
  - **供应商：**更新库存。
- **数据存储**：系统存储的数据，用于增删改查：
  - **用户信息表：**记录所有用户详细信息。
  - **商品库存表：**记录所有商品详细信息。
  - **订单文件：**记录所有商品订单详细信息。
- **系统**：销售管理系统。
- **数据流**：外部实体与系统之间的数据交换。数据流说明：
  - **用户 → 系统**：商品购买请求。
  - **系统 → 用户**：订单确认信息、商品缺货通知。
  - **销售经理 → 系统**：查询订单文件。
  - **系统 → 销售经理**：生成销售报表。
  - **供应商 → 系统**：库存更新。
  - **系统 → 供应商**：缺货商品信息。

该系统的顶层 DFD（系统环境图）如下：

淘宝店：PandaCode辉文档源码库，原创

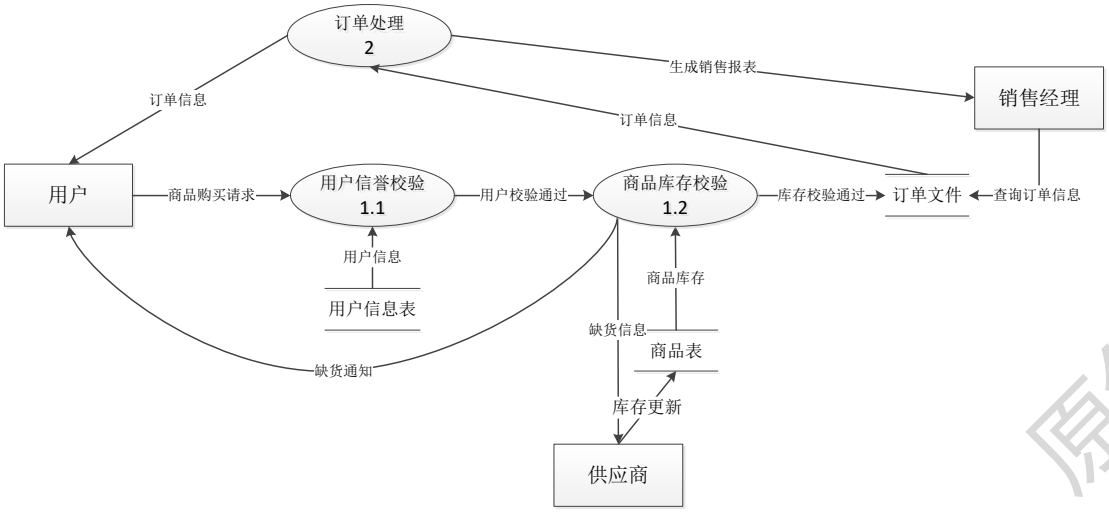


(2). 该系统的 0 层 DFD 图（将顶层 DFD 中的系统分解为 “请求校验”“订单处理”2 个核心加工）如下:



(3). 针对 “请求校验” 加工（分解为 “用户信誉校验”“商品库存校验” 2 个细分加工), 该系统的 1 层 DFD 图如下:

淘宝店：PandaCode源文档源码库，原创



2. 问题陈述：在一个简单的在线购物系统中，用户可以浏览商品、将商品添加到购物车、结算购物车中的商品。管理员可以管理商品信息，包括添加、修改和删除商品。

(1) (3 分) 用结构化分析方法给出该系统的顶层 DFD（系统环境图)。

(2) (2 分) 给出顶层 DFD 中的两个数据存储的定义。

(3) (3 分) 选择该在线购物系统中的“添加商品到购物车”交互，并用顺序图来描述。

解析:

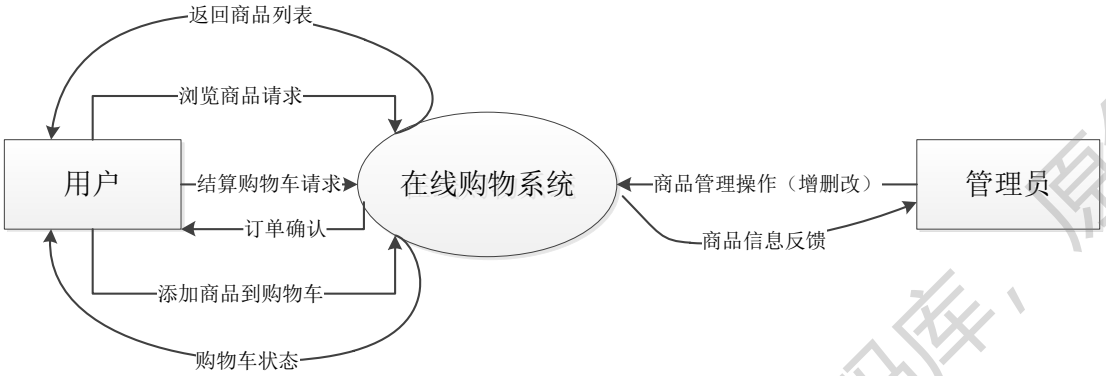
(1). 该系统的顶层 DFD（系统环境图)。

- **顶层 DFD（Data Flow Diagram，数据流图）** 是系统环境图，它展示了系统的外部实体、系统本身以及系统与外部实体之间的数据流。
- **外部实体**：与系统交互的外部参与者，包含两个主要外部实体:
  - **用户**：浏览商品、添加商品到购物车、结算购物车。
  - **管理员**：管理商品信息（添加、修改、删除)。
- **系统**：在线购物系统。
- **数据流**：外部实体与系统之间的数据交换。数据流说明:
  - 用户 → 系统：浏览商品请求、添加商品到购物车、结算购物车请求。
  - 系统 → 用户：商品列表、购物车状态、订单确认。

淘宝店：PandaCode程序文档源码库，原创

- 管理员 → 系统：商品管理操作（增删改）。
- 系统 → 管理员：商品信息反馈。

该系统的顶层 DFD（系统环境图）如下：



(2). 顶层 DFD 中的两个数据存储的定义。

商品数据库：

定义：存储所有可供销售的商品信息，包括商品 ID、名称、描述、价格、库存数量等。

目的：支持用户浏览商品及管理员对商品信息的管理操作（添加、修改、删除）。

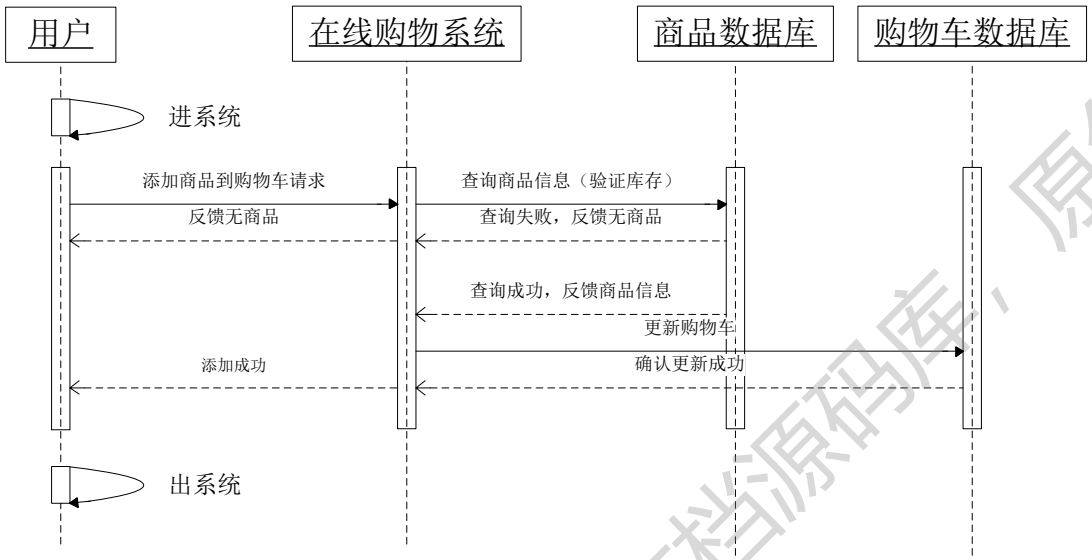
购物车数据库：

定义：保存每个用户的购物车内容，记录用户选择的商品及其数量。

目的：支持用户将商品添加到购物车、查看购物车内容以及完成结算流程。

(3). 选择在线购物系统中的“添加商品到购物车”交互，用顺序图如下：

淘宝店：PandaCode课文档源码库，原创



3. 问题陈述：在一简化的某学校图书管理系统中，

库房管理员职责：

根据各系的规定负责购置图书，并为新书编制图书卡片（分类目录号、流水号、书名、作者、价格、购书日期），写入图书目录文献。

图书管理员职责：

根据借还书的规定负责借还图书处理：

①当没有要借的图书时，向借阅人发无书告知；

②当借阅人将图书丢失时，以图书丢失单的形式汇报图书管理员，由图书管理员下罚款单给借阅人；同步图书管理员负责修改图书目录文献，并告知库房管理员。

图书馆主任职责：

每月末根据分类目录号查看某种图书库存量，当库存量低于库存下限时，告知库房管理员购书。

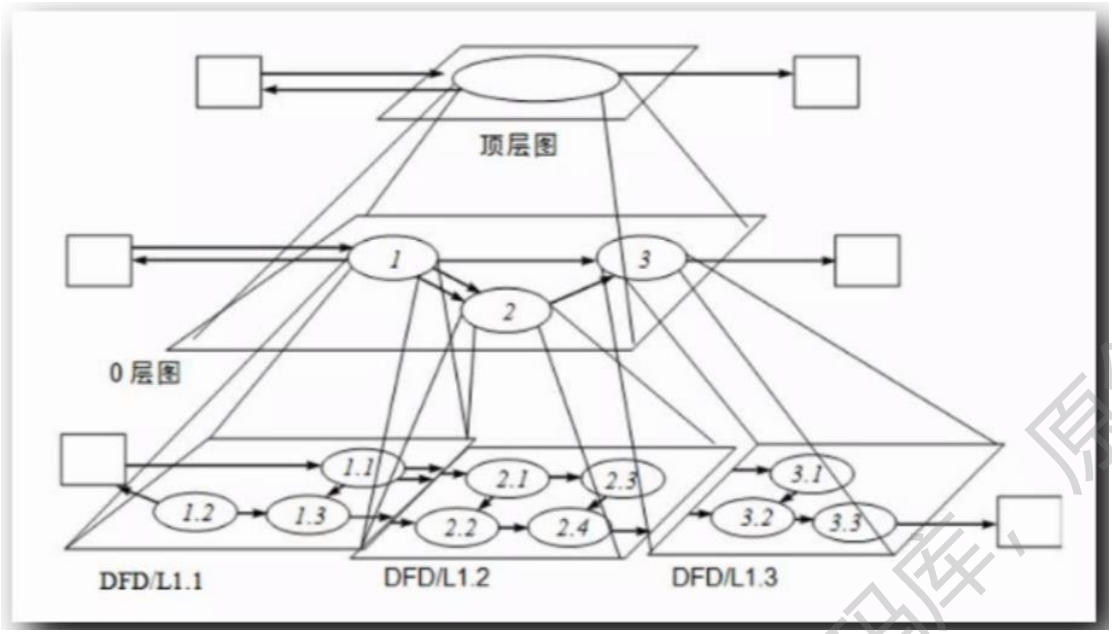
(1)(3 分)用构造化分析措施给出该系统的顶层 DFD（系统环境图）；

(2)(2 分)给出顶层 DFD 的数据字典；

(3)(2 分)选择该图书管理系统中的一种交互，并用次序图（顺序图）来描述。

答：考察软件设计-结构化方法

淘宝店：PanCode浑文档源码库，原创



0、数据流图分层说明

数据流图分层，最上层是顶层数据流图，第二层是 0 层数据流图，最底层是底层数据流图，“顶层数据流图”与“底层数据流图”之间是若干中层数据流图，中层数据流图需要进行编号，从 0 开始编号。

结构化分析方法，简单说，从顶层简单到底层，逐层分解精细化。

1、顶层数据流图

顶层数据流图：中间的椭圆是需要开发的系统，周边的矩形表示的是外部实体人或组织，外部实体与系统之间，有数据传输关系；顶层数据流图能够表达的信息是非常有限的，其将整个系统，使用一个节点表示，其可以体现出系统与外界实体之间的交互，但是系统内部的情况，系统内部模块之间的数据交换是没有体现的。

2、中层数据流图(0 层)

将“顶层数据流图”进行细化，细化后的即 0 层数据流图；

与顶层数据流图比较，没有变化的部分：外部实体，外部实体与系统之间的数据流，是没有变化的；

变化部分：有变化的部分是系统内部，系统内部进行了细化，原来系统是一个节点，在中层数据流图(0 层)中，将一个节点拆分成多个节点，这些节点就是系统中的数据处理部件，即加工。这些数据处理部件（加工）之间会有数据流的交互。

3、底层数据流图(1 层)

针对每个加工节点，将其拆分，绘制其中的更详细的数据流转情况；

数据流图（DFD）分层，是从 顶层 -> 中层 -> 底层，逐层进行分解，这种分解思路，与结构化的开发方法，是完全匹配的。

注：该题的答案在形式上也许是多种多样的，但在回答时，重要注意三点：

（1）顶层 DFD 与否符合系统环境图的构造特点；

（2）数据字典的定义与否符合其书写原则；

淘宝店：Panda辉文档源码库，原创

（3）所画的次序图与否符合 UML 中次序图的定义。

（1）该系统的顶层 DFD（系统环境图）：

在结构化分析中，顶层 DFD（数据流图）也被称为系统环境图。它用于展示系统与外部实体之间的数据流动。

在这个简化的图书管理系统中，我们可以识别出以下实体和数据流：

外部实体：

① 借阅人 – 借书，还书，图书丢失单。

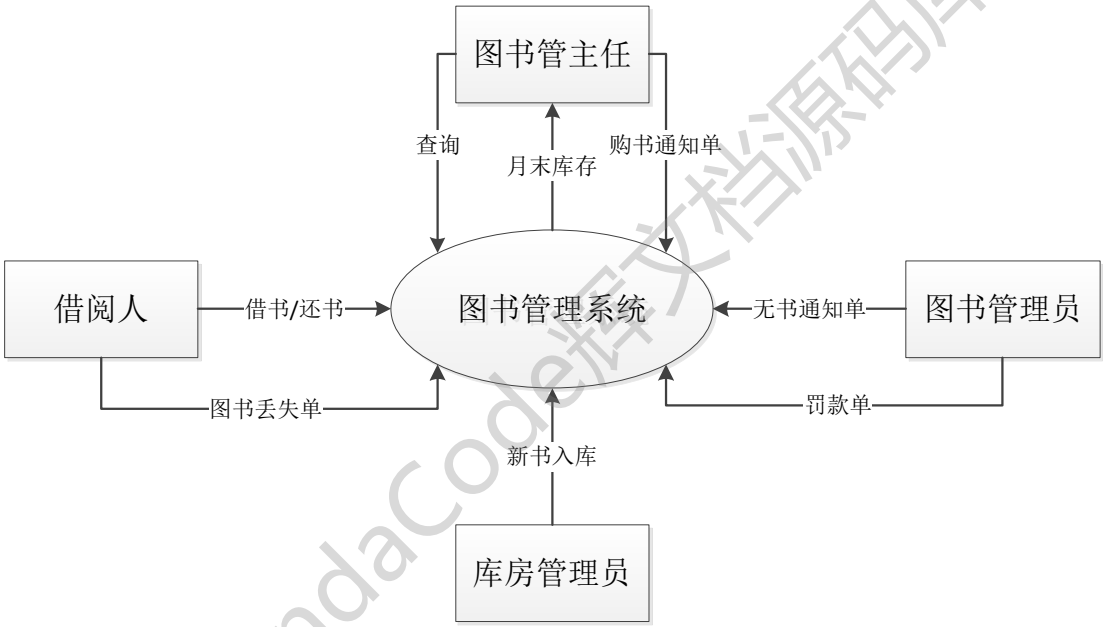
② 库房管理员 – 购置图书、编制图书卡片、新增图书目录。

③ 图书管理员 – 发出无书告知、处理丢失书籍和罚款、更新图书目录。

④ 图书馆主任 – 查看库存，指示购书。

数据存储：

① 图书目录文献 – 记录所有图书的详细信息。



（2）给出顶层 DFD 的数据字典：

新书入库单 = 分类目录号 + 流水号 + 书名 + 作者 + 价格 + 购书日期

借还书的规定 = [借书信息|还书信息]

借书信息 = [书名|作者] + 借书日期

还书信息 = 书名 + 还书日期 + 借阅人图书证号

图书丢失单 = 图书流水号 + 借阅人图书证号

无书告知单 = 借阅人图书证号 + 无书信息

罚款单 = 借阅人图书证号 + 罚款金额

查看规定 = 分类目录号

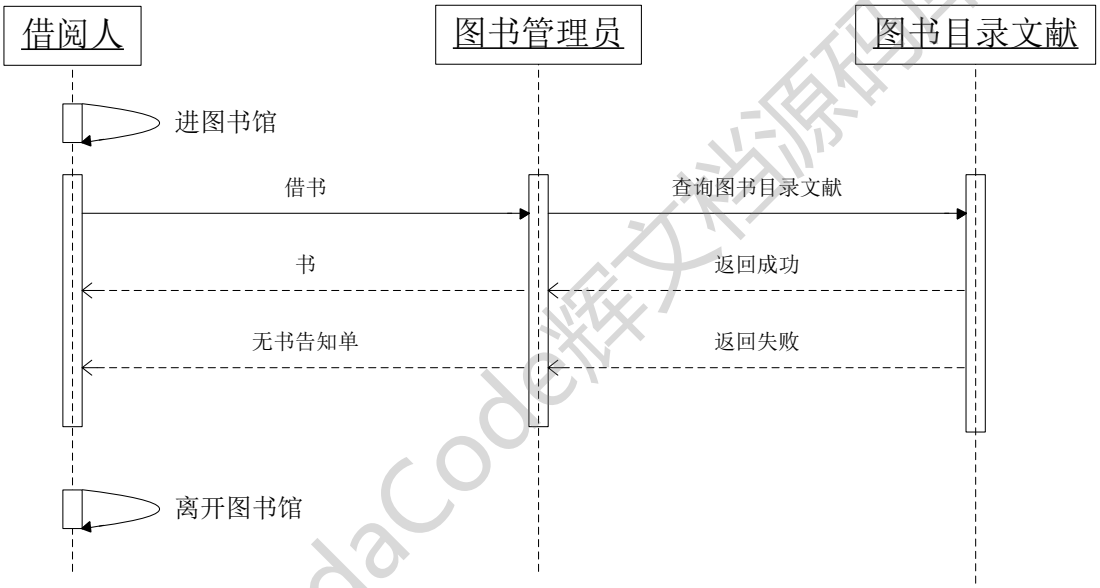
月末库存量 = {分类目录号 + 书名 + 库存量}

购书告知 = 分类目录号 + 书名 + 购置量

（3）选择该图书管理系统中的“借阅图书”交互，并用次序图（顺序图）来描述：

淘宝店：PandaCode程序档源码库，原创

系统行为（交互）的建模工具-顺序图
定义：顺序图是一种交互图，即由一组对象以及这些对象之间的关系组成，其中还包含这些对象之间被发送的消息。
顺序如下：
①借阅人向图书管理员发起借书请求，提供要借的图书信息。
②图书管理员查询图书目录文献，判断是否有库存。
③若有库存，图书管理员登记借阅信息，包括借阅人信息、借阅图书信息、借阅时间等，并将借阅信息存储至借阅记录中。
④若没有库存，图书管理员向借阅人发无书告知单。
⑤借阅人拿到图书或收到无书告知后，结束本次交互。



4. 某培训中心要研制一个计算机管理系统，它的业务是将学员发来的信息收集分类后，按以下不同情况进行处理：

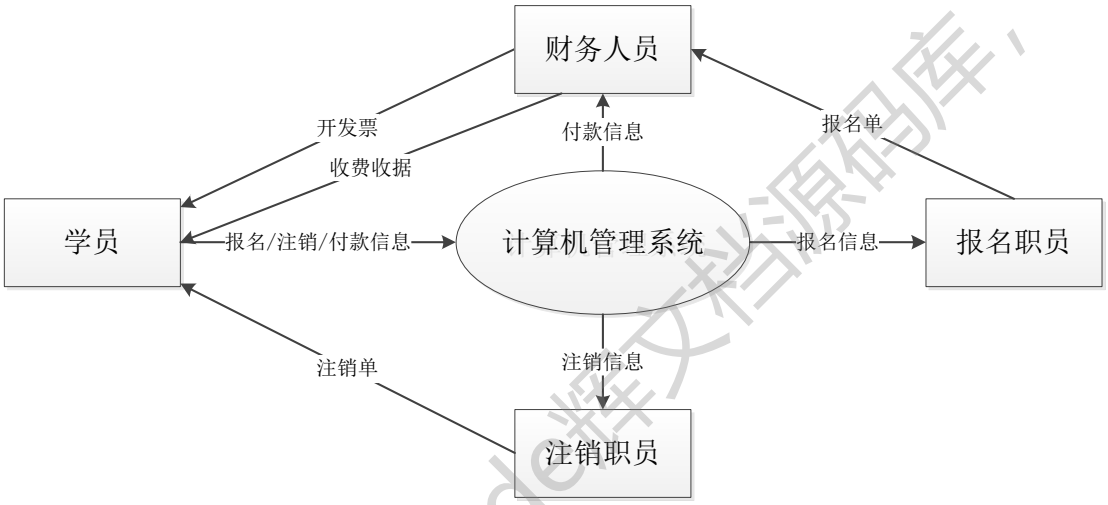
1，如果是报名，则将报名信息送给负责报名事务的职员，他们将查阅课程文件， 检查该课程是否满额，然后在学生文件、课程文件、账目文件上登记，并开出报名单交财务部门，财务人员开出发票给学生；
2，如果是想注销原来已选修的课程，则由注销人员在课程文件、学生文件和账目文件上做相应的修改，并给学生注销单；
3，如果是付款，则由财务人员在账目文件上登记，也给学生一张收费收据。请根据该计算机管理系统的功能描述，采用“自顶向下”的原则，画出其分层数据流图（顶层 DFD 和 0 层 DFD）。

解析：考察软件设计-结构化方法
（1）顶层 DFD（系统环境图）如下：

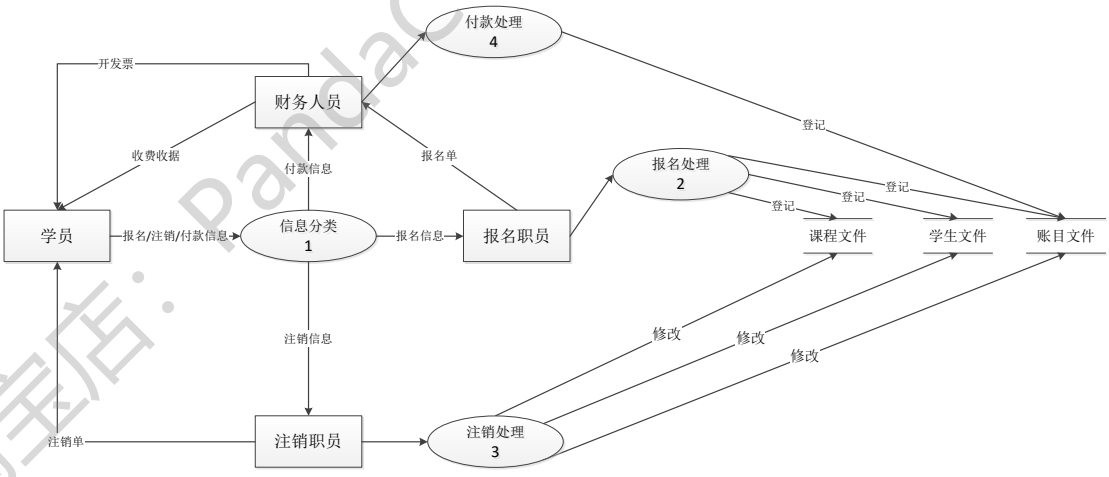
淘宝店：PandaCode编程文档源码库，原创

在结构化分析中，顶层 DFD（数据流图）也被称为系统环境图。它用于展示系统与外部实体之间的数据流动。
在这个计算机管理系统中，我们可以识别出以下实体和数据流：

- **外部实体：**
  - 学员 – 报名/注销/付款信息。
  - 报名职员 – 报名单。
  - 注销职员 – 注销单。
  - 财务人员 – 开发票，收费收据。
- **数据存储：**
  - 学生文件 - 记录学生信息。
  - 课程文件 - 记录课程信息。
  - 账目文件 - 记录账目信息。



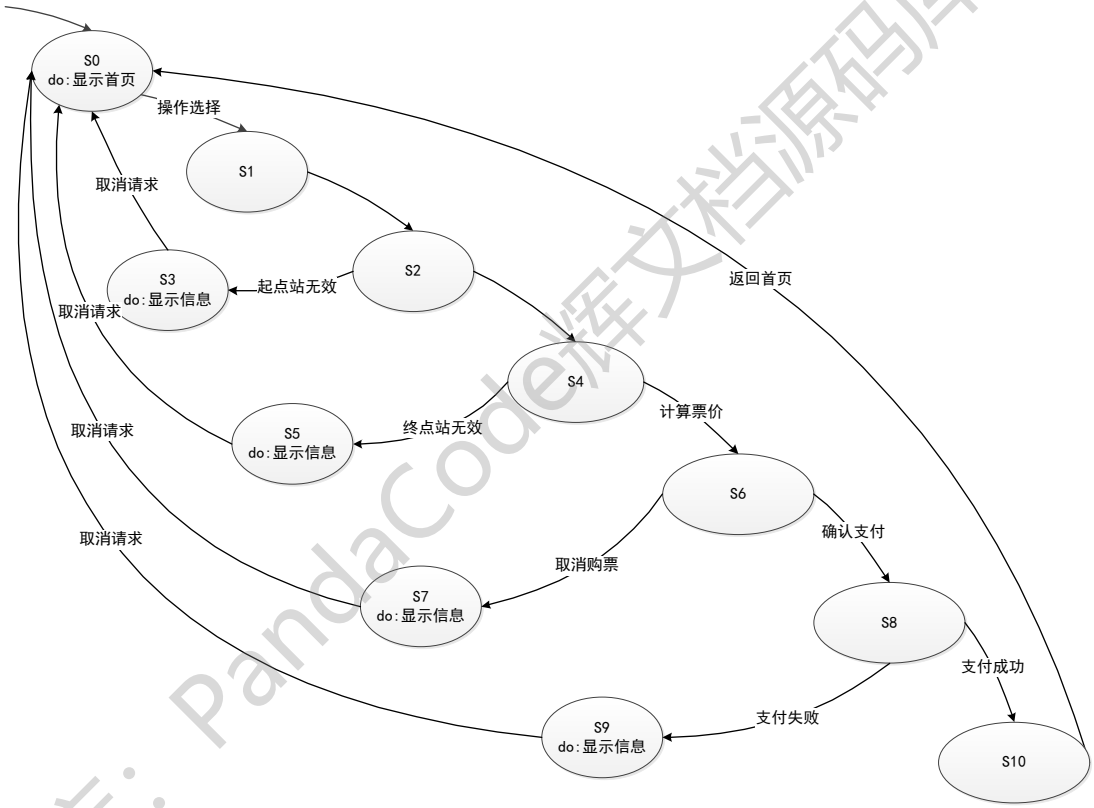
（2）0 层 DFD（数据流图）如下：（实际画图时数据流线尽量不能交叉!!!）



5. 在城市轨道交通自动售票系统中，“购买地铁票”是用户与系统之间的一次典型交互行为。系统需引导用户完成站点选择、票价计算、支付确认等步骤，并对输入有效性及支付结果进行判断和反馈。
问题陈述如下：
系统启动后，终端屏幕显示“欢迎使用地铁自动售票机”（首页）。

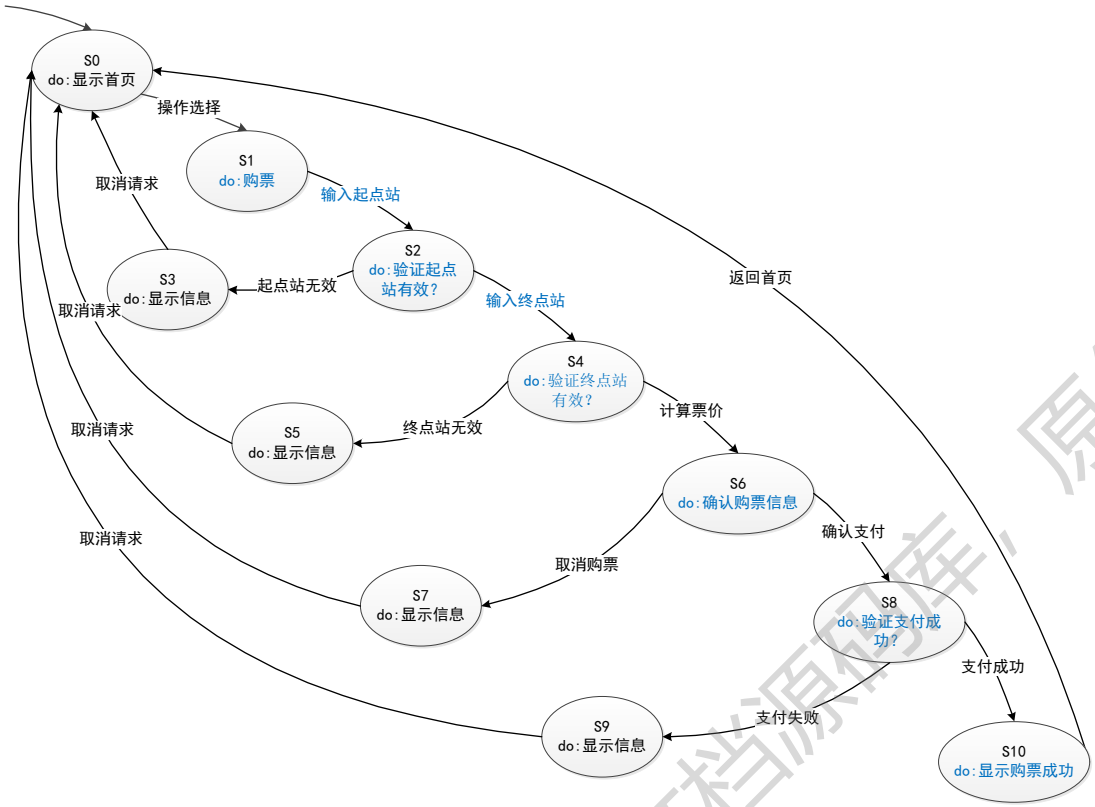
淘宝店：PandaCode辉文档源码库，原创

用户点击“购票”按钮后，系统进入“请输入起点站”状态；
用户输入起点站后，系统验证该站点是否存在：
若起点站无效，则显示“起点站不存在”，本次购票取消，返回首页；
若有效，则进入“请输入终点站”状态；
用户输入终点站后，系统验证终点站是否有效且与起点不同：
若无效，则显示“终点站无效或与起点相同”，返回首页；
若有效，则计算票价并进入“请确认购票信息”状态；
在此状态下，用户可选择“确认支付”或“取消购票”：
若选择“取消”，直接返回首页；
若选择“确认支付”，系统尝试扣款：
若支付成功，则出票并显示“购票成功，请取票”，随后返回首页；
若支付失败（如余额不足），则显示“支付失败”，返回首页。
(1)．（8 分）下图是该地铁票销售系统的状态图，图中共有 8 个状态节点缺失名称。请根据上述业务流程，补全以下状态名称。



解析：考察软件设计-面向对象方法与 UML 状态图
根据问题描述和状态图的逻辑，**补齐的系统状态图如下：**

淘宝店：PandaCode编文档源码库，原创



6. 在学校的教学管理系统中，学生查询成绩就是系统中的一次交互行为。问题陈述如下：开始时终端上显示首页，用户选择“查询”请求后，显示“请输入学号”。

在用户输入学号后，系统查询学生文件，和对学生学号：若输入的学号不正确，则显示“输入学号不正确”，此次查询取消，回到首页；若输入正确，出现“请输入课程名”。

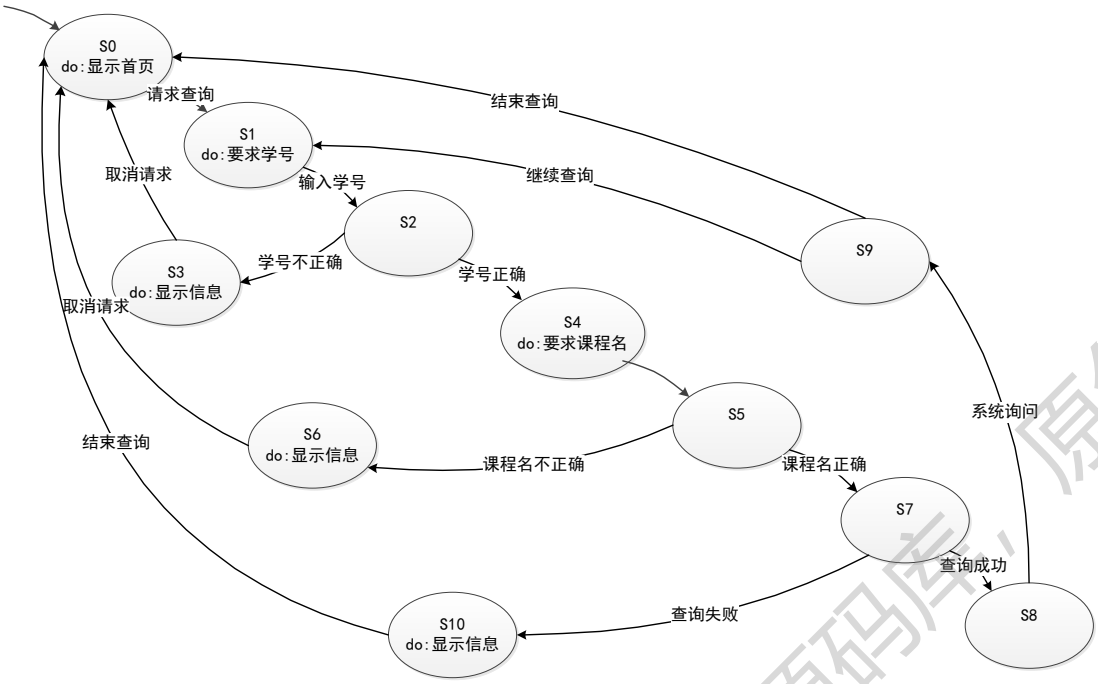
一旦输入课程名，则开始查找课程文件：若输入的课程名不正确，则显示“输入的课程名不正确”，此次查询取消，回到首页；若输入正确，则根据“学号”和“课程名”查询选课文件。

若在选课文件中查询成功，显示查询到的成绩，当用户选择“继续查询”后回到“请输入学号”，当用户选择“结束查询”后回到首页。若查询失败，则显示“查询失败”后回到首页。

(1)． (5 分) 下图是该系统的状态图，其中有 5 个状态的名字缺失，请补充完整。

(2)． (3 分) 计算途中的 McCabe 环路复杂度，并依据基本路径覆盖原则确定可能的测试路径。

淘宝店：Pass 100 文档源码库 原创



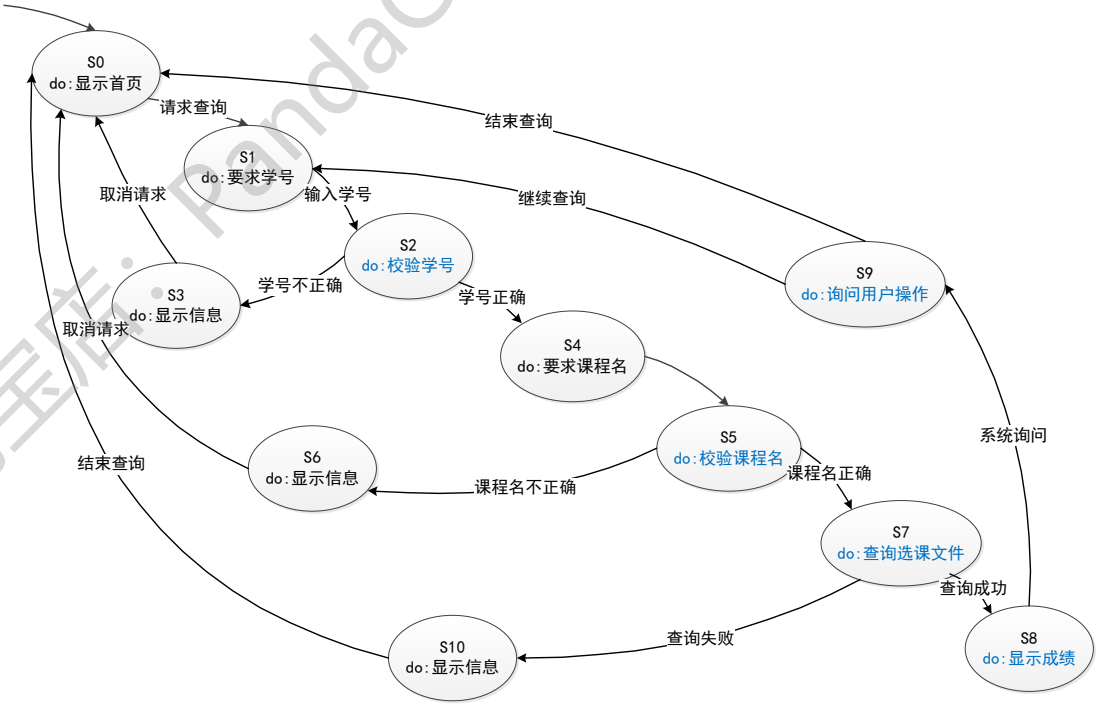
解析：考察软件设计-面向对象方法与 UML 状态图

(1)问题 1：补充状态图中缺失的状态名：

根据问题描述和状态图的逻辑，缺失状态的补充如下：

- S2：**校验学号**（系统对输入的学号进行校验的状态）。
- S5：**校验课程名**（系统对输入的课程名进行校验的状态）。
- S7：**查询选课文件**（系统根据学号和课程名查询选课文件的状态）。
- S8：**显示成绩**（查询成功后显示成绩的状态）。
- S9：**询问用户操作**（显示成绩后，询问用户是继续查询还是结束查询的状态）。

补齐的系统状态图如下：



淘宝店：PandaCode原创文档源码库

(2)**问题 2：计算 McCabe 环路复杂度并确定测试路径：**

①计算 McCabe 环路复杂度

McCabe 环路复杂度有多种计算方法，这里采用"**判定节点数 + 1**"的方法（判定节点是指有分支判断的节点，如判断学号是否正确、课程名是否正确、查询是否成功等）。

从状态图中可以看出，判定节点有：学号是否正确、课程名是否正确、查询是否成功、用户选择（继续查询/结束查询），共 4 个判定节点。

因此，环路复杂度 = 判定节点数 + 1 =4 + 1 = 5 。

②确定可能的测试路径（基于基本路径覆盖原则）

基本路径覆盖要求设计足够的测试用例，使得程序中每一条独立的路径都至少被执行一次。

结合状态图的逻辑，可能的测试路径如下：

- **路径 1：** S0 → S1 → S2 → S3 → S0 （输入学号不正确，查询取消返回首页）。
- **路径 2：** S0 → S1 → S2 → S4 → S5 → S6 → S0 （输入课程名不正确，查询取消返回首页）。
- **路径 3：** S0 → S1 → S2 → S4 → S5 → S7 → S10 → S0 （查询选课文件失败，返回首页）。
- **路径 4：** S0 → S1 → S2 → S4 → S5 → S7 → S8 → S9 → S0 （查询成功，用户选择结束查询返回首页）。
- **路径 5：** S0 → S1 → S2 → S4 → S5 → S7 → S8 → S9 → S1 （查询成功，用户选择继续查询，回到要求学号的状态）。

7. 一个图书管理系统中，用户借阅图书的流程如下：

开始时系统显示主界面，用户选择“借阅图书”后，系统提示“请输入图书 ID”。

用户输入图书 ID 后，系统查询图书库存文件：若图书 ID 不存在，显示“图书 ID 无效”，回到主界面；若图书 ID 有效，检查图书是否可借（未被借出）：若不可借，显示“图书已借出”，回到主界面；若可借，系统提示“请输入读者 ID”。

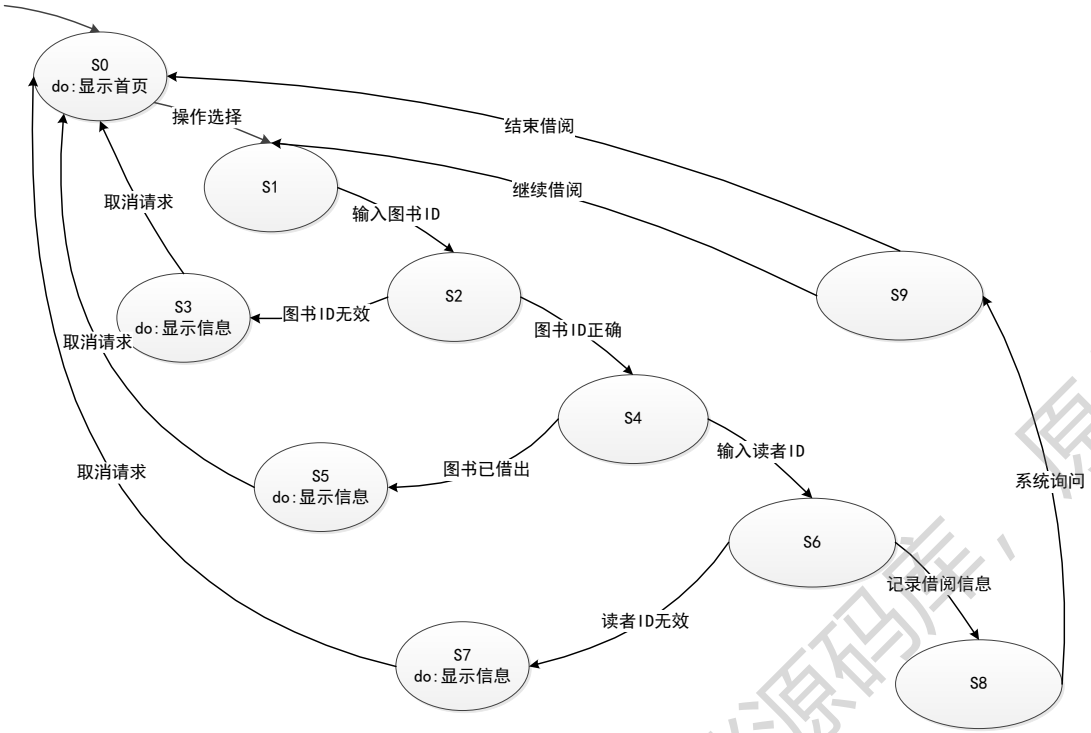
用户输入读者 ID 后，系统查询读者信息文件：若读者 ID 无效，显示“读者 ID 无效”，回到主界面；若读者 ID 有效，记录借阅信息（更新库存为“已借出”、生成借阅记录），显示“借阅成功”；随后询问用户操作，是否继续借阅？选择继续借阅，重新输入图书 ID，结束借阅则回到主界面。

根据上述流程，回答以下问题：

(1). （5 分）下图是该系统借阅流程的状态图，其中有 6 个空白状态的名字缺失，请补充完整。

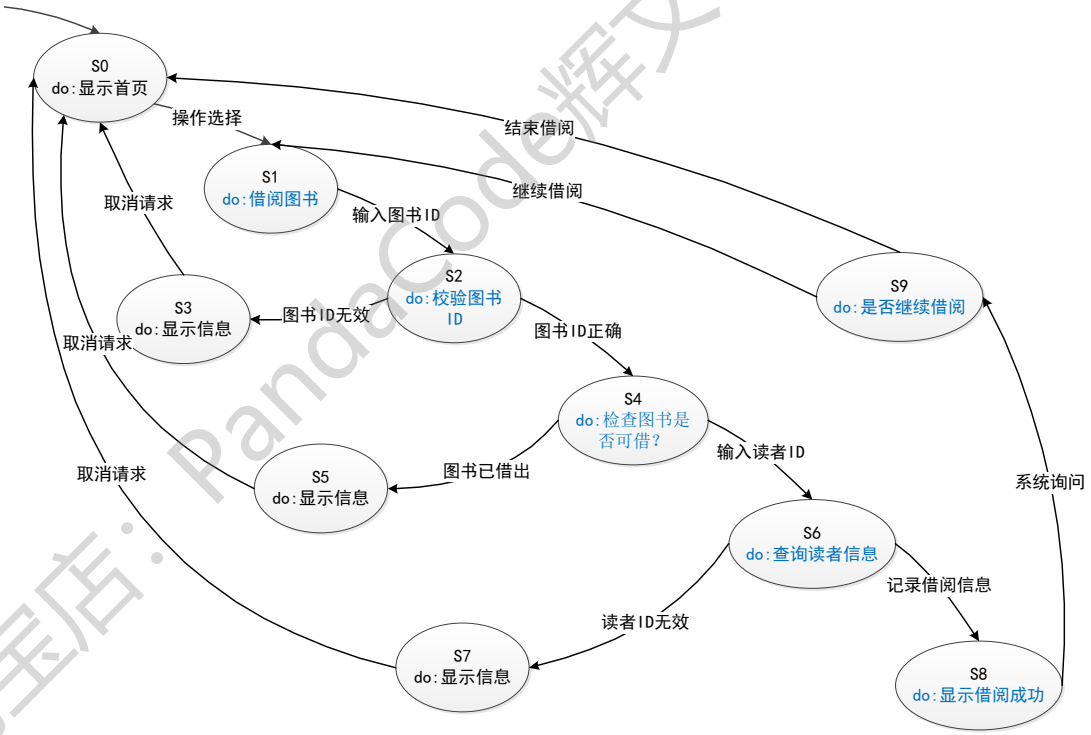
(2). （3 分）计算该流程图的 McCabe 环路复杂度是多少？

淘宝店：Pandaccc原文档源码库 原创



解析：考察软件设计-UML 状态图

(1)．补齐该系统借阅流程的状态图。



(2)．计算该流程图的 McCabe 环路复杂度。

方法 1（**判定节点数 + 1**）：判定节点数，如分支条件：校验图书 ID、图书是否可借等，共 4 个判定节点。

因此，环路复杂度 = 判定节点数 + 1 =4 + 1 = 5 。

方法 2（独立路径数）：流程中独立路径包括：

淘宝店：PandaCode辉文档源码库，原创

- ①主界面→输入图书 ID→ID 无效→主界面;
- ②主界面→输入图书 ID→ID 有效→图书已借出→主界面;
- ③主界面→输入图书 ID→ID 有效→图书可借→输入读者 ID→ID 无效→主界面;
- ④主界面→输入图书 ID→ID 有效→图书可借→输入读者 ID→ID 有效→借阅成功→主界面;

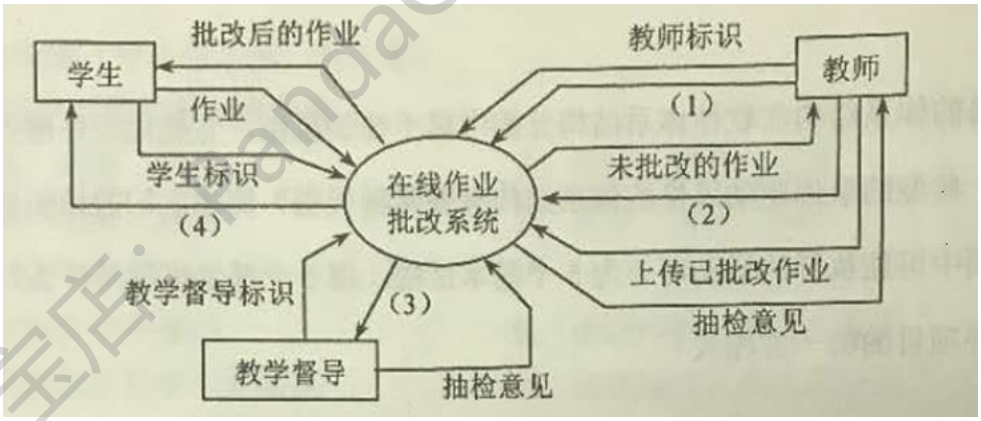
共 4 条独立路径，加 1 得复杂度为 5。

8. 问题陈述：下面是在某慕课教育平台上一个在线作业批改系统的简化陈述：

- 学生
  - ◇ 系统登陆，提交学生标识(学号、密码)；
  - ◇ 在线提交作业(作业题量、完成题目、答案、提交时间)；
  - ◇ 获取作业已批改的通知(学号、批改时间、评分、评价(可选))；
  - ◇ 查看已批改的作业。
- 教师
  - ◇ 教师登陆，提交教师标识(教师号、密码)；
  - ◇ 上传作业信息(作业题量、布置题目、标准答案、评分标准、最后期限)；
  - ◇ 获取有作业提交的通知(学号，提交时间)；
  - ◇ 下载学生提交的作业(在屏幕上显示)；
  - ◇ 批改后上传已批改作业(学号、批改时间、评分、评价(可选))；
  - ◇ 获取作业抽检意见(学号、教师号、建议)。
- 教学督导
  - ◇ 教师登陆，提交督导标识(教师号、密码)；
  - ◇ 抽取并下载作业样本(学号、教师号、批改时间、评分、评价(可选))；
  - ◇ 抽检并提交抽检意见(学号、教师号、建议)。

(1). (4 分)下图是根据结构化分析方法建立的系统顶层数据流图,其中有 4 个数据流缺失，请补充完整。

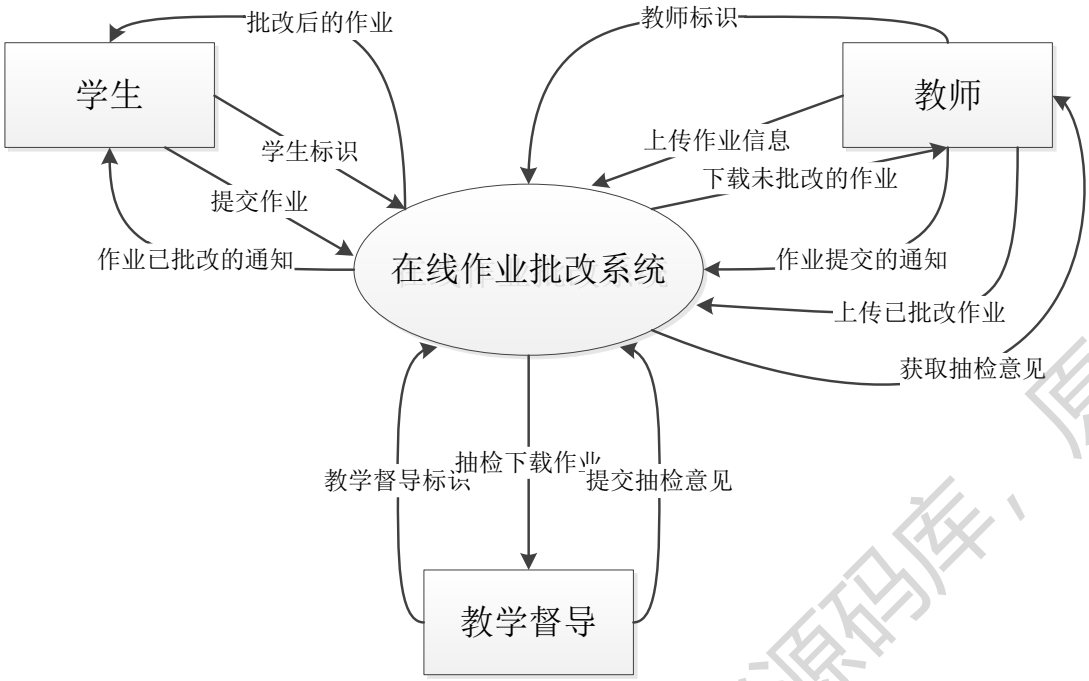
(2). (4 分)针对数据流图上所涉及作业的数据流，用数据字典描述它们。



解析：考察软件设计-结构化方法

(1). 完善后的顶层 DFD（系统环境图）:

淘宝店：PandaCode程文档源码库，原创



(2). 以下是涉及作业的数据流的数据字典格式:

- 教师，上传作业信息 = 作业题量 + 布置题目 + 标准答案 + 评分标准 + 最后期限
- 学生，提交的作业 = 作业题量 + 完成题目 + 答案 + 提交时间
- 教师，获取作业提交通知 = 学号 + 提交时间
- 教师，下载未批改的作业 = 学号 + 提交时间+ 下载标识
- 教师，上传批改后的作业 = 学号 + 批改时间 + 评分 + 评价
- 学生，获取作业已批改通知 = 学号 + 批改时间 + 评分 + 评价
- 教学督导，抽取下载作业 = 学号 + 教师号 + 批改时间 + 评分 + 评价
- 教学督导，提交抽检意见 = 学号 + 教师号 + 建议
- 教师，获取抽检意见= 学号 + 教师号 + 建议

9. 一个简化的教学管理系统的描述如下：

●与学生相关的活动：

◇学生向系统提出注册申请

◇系统向学生反馈学籍通知

◇系统向部分优秀学生发出奖励通知

●与教务处相关的活动：

◇教务处将学籍审批结果输入系统

◇系统向教务处显示统计报告

◇教务处向系统提供奖励凭证

●与系教学办相关的活动：

◇系统显示学籍报告管理情形

◇系教学办将学籍处理意见和奖励建议输入系统

●与教师相关的活动：

◇系统将教学安排通知教师

◇教师将学生成绩输入系统

淘宝店:xiandaCode 毕业设计源码库，原创

试回答：

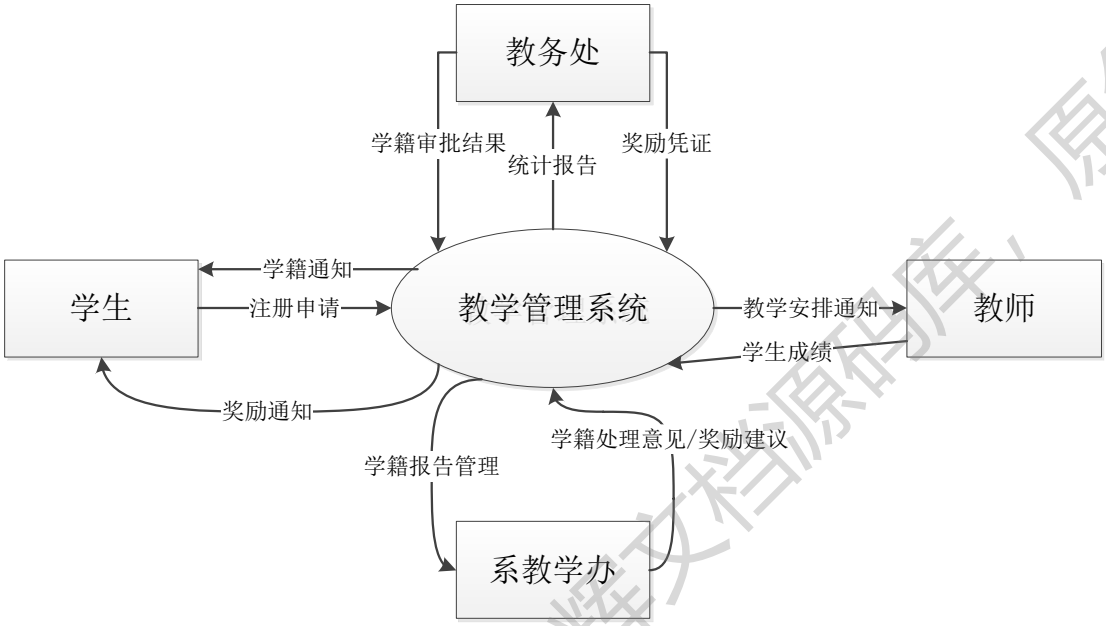
(1)．(3 分) 根据以上描述，画出该系统的顶层数据流图；

(2)．(2 分) 写出至少两个相关的数据文件；

(3)．(3 分) 如果系统的主要功能是注册管理、学籍管理、成绩管理、奖励管理，请画出系统的第一层数据流图。

解析：考察软件设计-结构化方法

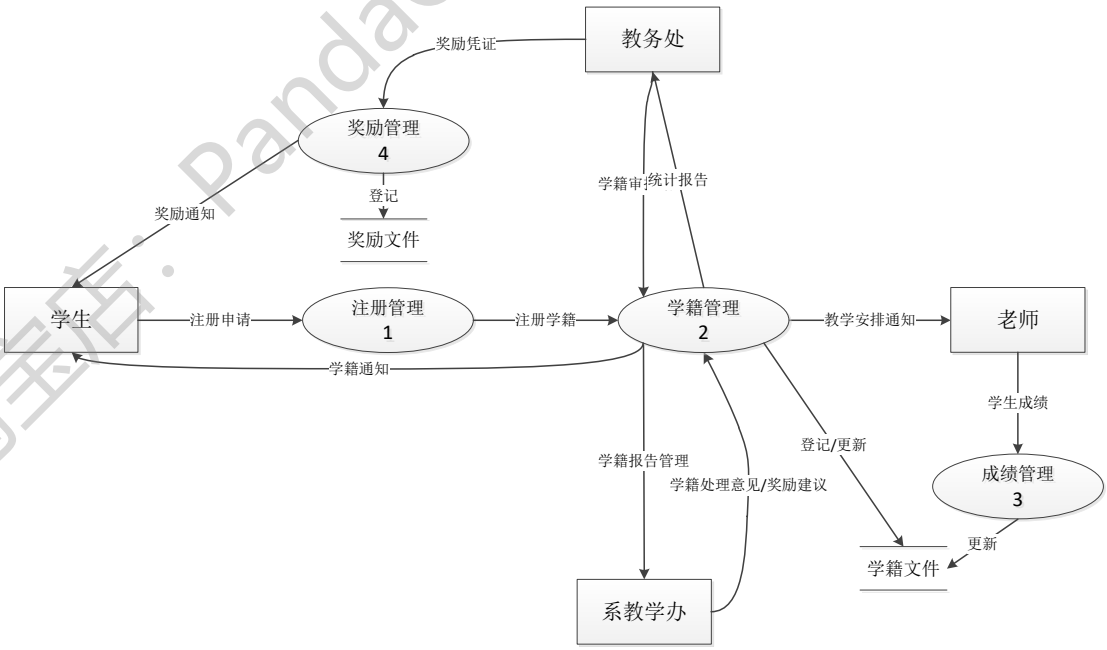
(1) 顶层 DFD 图（系统环境图）



(2) 数据文件存储：

1. 学籍文件 - 记录学生学籍信息。
2. 奖励文件 - 记录学生奖励信息。

(3) 第一层 DFD 图（数据流图），主要功能是注册管理、学籍管理、成绩管理、奖励管理。



淘宝店：Pandacode精文档源码库，原创

10. 问题陈述:

下面是一个简化的 Web 商品购销存信息系统的描述。

● Web 用户:

- 注册/登录系统。用户输入用户名和密码，系统验证。如果正确，系统返回注册/登录成功信息，否则返回注册/登录失败信息。
- 在系统显示窗口上浏览商品和选购商品，系统显示购物车信息。
- 根据购物车信息确定订单，网上支付，系统显示交易成功信息。
- 得到发票。

●仓储主管:

- 获取商品库存清单。
- 根据订单，确认出库单。
- 根据到货单，确认入库单。

●销售经理:

- 确认系统制作出的销售报表。

●采购经理:

- 根据销售报表和商品库存清单，确定采购清单。
- 提交到货单。

试回答:

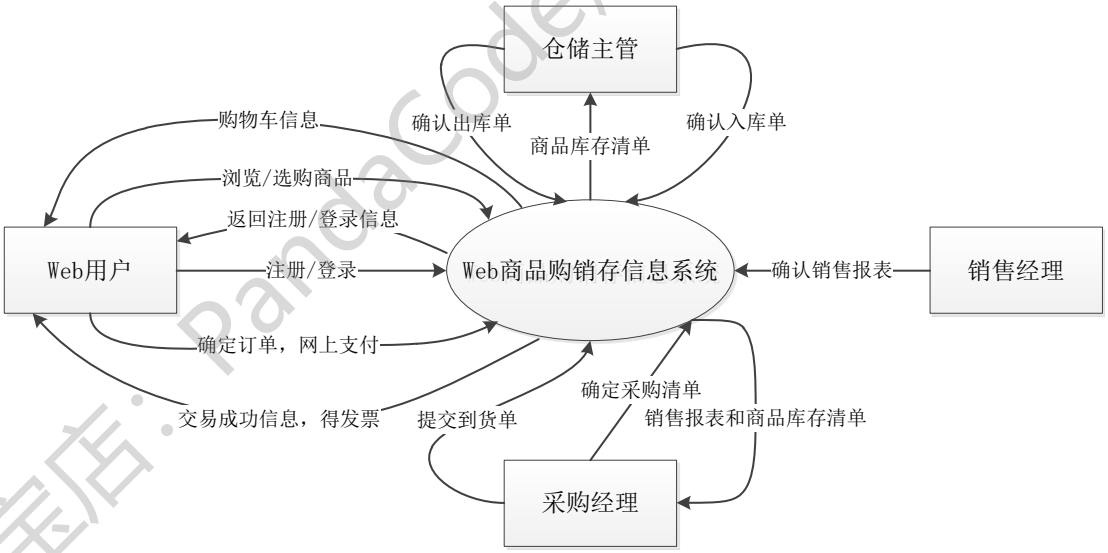
(1) (3 分) 用结构化方法给出系统的顶层数据流图。

(2) (2 分) 简述系统中可能用到的数据存储文件。

(3) (3 分) 用 UML 顺序图给出 Web 用户登录到系统的一个交互。

答: 考察软件设计

(1) 该系统顶层 DFD 数据流图。



(2) 该系统中用到的数据存储文件。

用户信息文件: 记录注册用户信息;

商品信息文件: 记录商品信息;

库存清单文件: 记录商品库存信息;

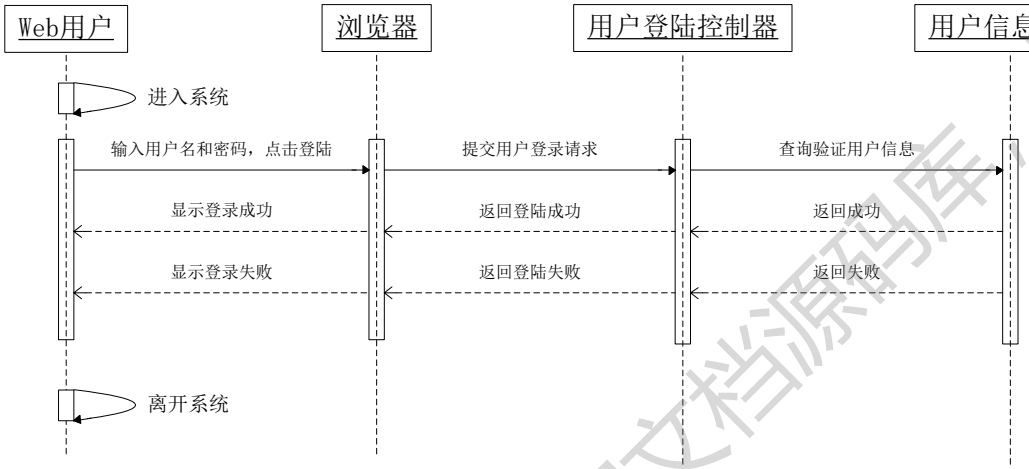
购物车文件: 记录用户选购商品信息;

订单文件: 记录用户提交订单信息;

交易记录文件: 记录支付信息;

淘宝店：PandaCode辉文档源码库，原创

销售报表文件：记录销售统计数据；
采购清单文件：记录采购清单信息；
到货单文件：记录到货单信息；
入库单文件：记录入库单信息；
出库单文件：记录出库单信息。
(3) 用 UML 顺序图给出 Web 用户登录到系统的一个交互。

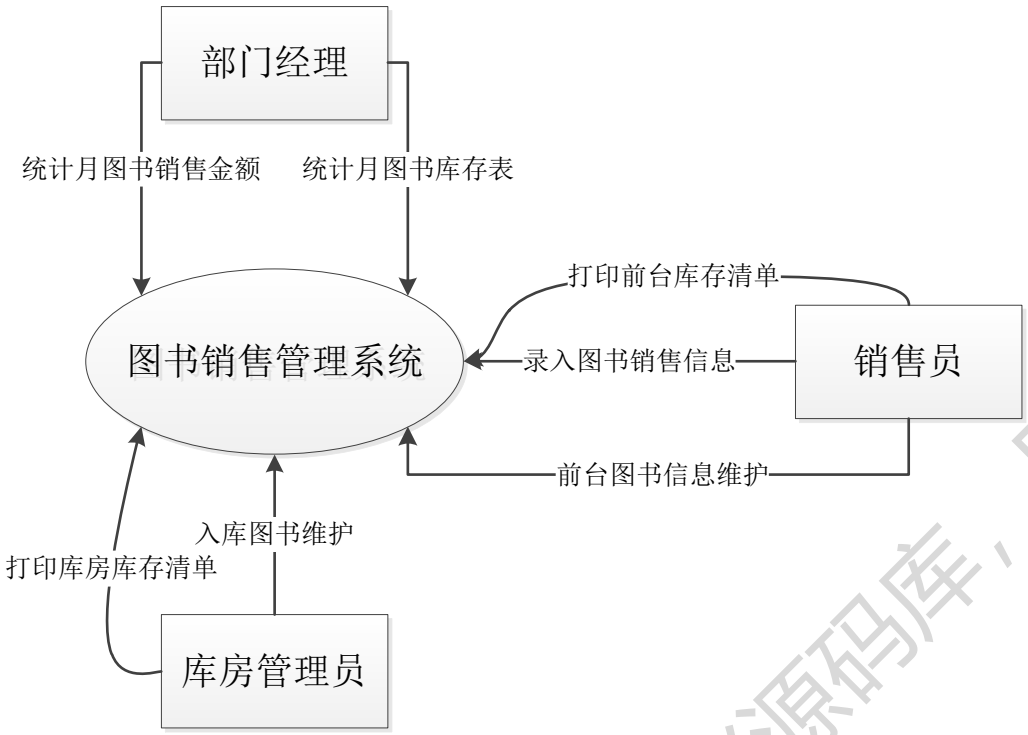


11. 问题陈述：在一简化的图书销售管理系统中，

- 库房管理员负责：
  - 输入、修改、删除入库图书信息（图书名， 图书编号， 数量， 单价，入库日期）；
  - 打印库房图书库存清单（图书名， 图书编号，库存量， 库存金额）。
- 销售员负责：
  - 录入图书销售信息（图书名，图书编号，销售量， 单价）；
  - 输入、修改、删除前台图书信息（图书名，图书编号，数量， 单价）；
  - 打印前台图书库存清单（图书名， 图书编号，库存量，库存金额）。
- 部门经理负责：
  - 统计月图书销售金额；
  - 统计月图书库存情况（图书名， 图书编号， 库存量， 库存金额）。

试回答：
(1) (3 分) 用结构化分析方法给出该系统的顶层 DFD；
(2) (2 分) 给出顶层 DFD 的数据字典；
(3) (3 分) 选择该图书销售管理系统中的一个交互，并用顺序图来描述。
解析：考察软件设计
(1) 顶层 DFD 图（系统环境图）

淘宝店 PandaCode辉文档源码库 原创



(2) 顶层 DFD 数据字典:

在结构化分析方法中，**数据字典**（Data Dictionary, DD）用于对数据流图（DFD）中出现的所有**数据流、数据存储、外部实体和加工**进行精确定义。

由于是**顶层 DFD**（Context Diagram），系统被抽象为一个整体处理（通常记为 P0），因此数据字典主要包括:

- **外部实体（External Entities）;**
- **数据流（Data Flows）;**
- （注：顶层 DFD 中无内部数据存储和子加工，故不列数据存储项）。

顶层 DFD 数据字典如下：

- **外部实体：**
  - 库房管理员
  - 销售员
  - 部门经理
- **数据流：**
  - 入库图书信息 = 图书名 + 图书编号 + 数量 + 单价 + 入库日期
  - 图书销售信息 = 图书名 + 图书编号 + 销售量 + 单价
  - 前台图书信息 = 图书名 + 图书编号 + 数量 + 单价
  - 库房图书库存清单 = {图书名 + 图书编号 + 库存量 + 库存金额}
  - 前台图书库存清单 = {图书名 + 图书编号 + 库存量 + 库存金额}
  - 月图书销售金额报表 = 月份 + 总销售金额
  - 月图书库存情况报表 = {图书名 + 图书编号 + 库存量 + 库存金额}

(3) 选择该图书销售管理系统中的"录入图书销售信息"交互，并用次序图（顺序图）来描述:

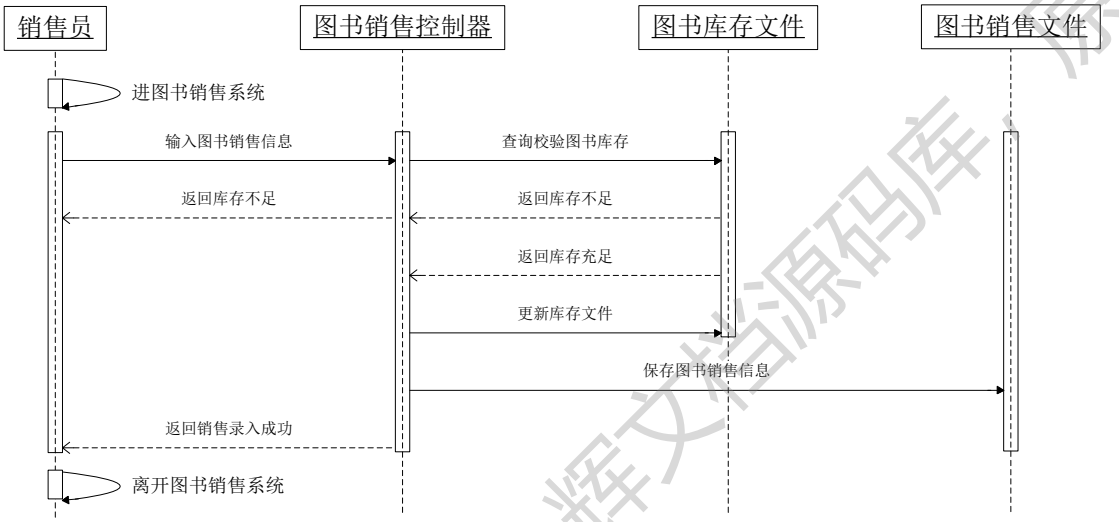
系统行为（交互）的建模工具-顺序图

定义：顺序图是一种交互图，即由一组对象以及这些对象之间的关系组成，其中还包含这些对象之间被发送的消息。

文字描述交互顺序如下:

淘宝店：PandaCode辉煌资源码库，原创

① 销售员向图书销售系统发起录入图书销售信息请求，输入图书销售信息。
② 通过图书销售控制器，查询图书库存文件，校验库存是否充足。
③ 若库存不足，则图书销售控制器返回库存不足。
④ 销售员收到库存不足，结束本次交互。
⑤ 若库存充足，则图书销售控制器更新图书库存文件，然后保存图书销售信息到图书销售文件中。
⑥ 销售员收到销售录入成功，结束本次交互。



淘宝店：PandaCode辉文档源码库，原创